

NFS Project Documentation

Chapter 1 – Project Overview

1.1 Executive Summary

NFS (Nafs) is a comprehensive digital mental health platform designed to connect patients with licensed therapists through a secure and intelligent online environment. The platform streamlines therapist discovery, appointment booking, therapy session management, psychological assessments, payments, and AI-assisted interactions.

Unlike traditional clinic management systems, NFS integrates modern web technologies with artificial intelligence to improve accessibility, personalization, and efficiency in mental healthcare.

The system is composed of three major components:

- **Frontend Application** – User interface for patients, therapists, and administrators.
- **Backend API** – Handles business logic, authentication, database operations, and communication between all system components using Clean Architecture principles.
- **AI Engine** – A dedicated service responsible for AI-powered conversations and future intelligent recommendations.

The backend is implemented using [ASP.NET](#) Core following the Clean Architecture approach, which separates the system into independent layers with clear responsibilities and maintains a clear separation between business logic and infrastructure concerns. SQL Server manages structured application data, while MongoDB is used for chat-related information.

1.2 Project Vision

The vision of NFS is to provide an accessible, secure, and scalable platform that enables individuals to receive professional psychological support regardless of their location.

By combining online therapy with AI-powered assistance, the platform aims to improve the overall mental healthcare experience for both therapists and patients.

1.3 Project Objectives

The primary objectives of the project are:

- Develop a secure online therapy platform.
 - Simplify therapist discovery and appointment booking.
 - Maintain complete patient and therapist profiles.
 - Support online psychological assessments.
 - Enable secure authentication and authorization.
 - Provide real-time communication capabilities.
 - Integrate AI services to enhance user experience.
 - Ensure scalability and maintainability through Clean Architecture.
-

1.4 Problem Statement

Access to mental healthcare can be limited by geographic constraints, scheduling difficulties, and administrative overhead. Traditional appointment booking and patient management processes often require significant manual effort and lack integrated digital tools.

NFS addresses these challenges by offering:

- Online therapist search.
- Digital appointment scheduling.
- Electronic patient records.
- Secure authentication.
- AI-assisted communication.
- Centralized management for therapists and patients.

This integrated approach reduces operational complexity while improving accessibility and user experience.

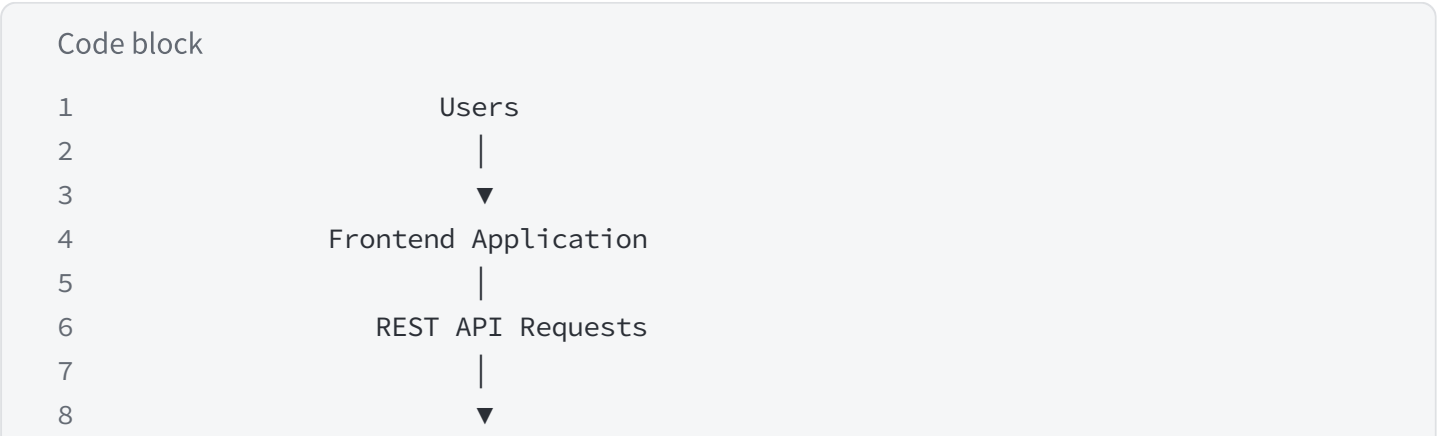
1.5 Target Users

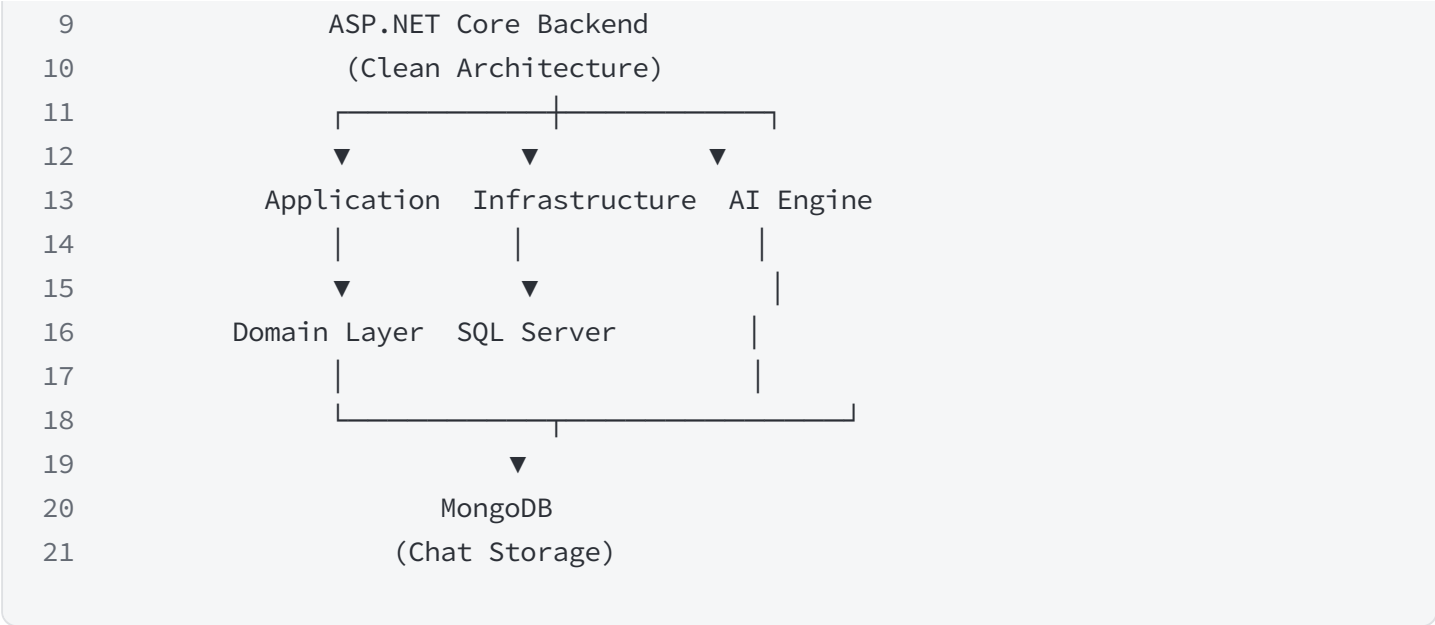
The system serves several categories of users:

User Type	Responsibilities
Patient	Register, complete assessments, book sessions, communicate with therapists, view history.
Therapist	Manage profile, availability, appointments, and patient interactions.
Administrator	Manage users, monitor the system, approve therapists, oversee platform operations.
AI Service	Provide conversational assistance and support intelligent features.

1.6 High-Level System Components

The project consists of the following main components:





This separation of responsibilities allows each component to evolve independently while communicating through well-defined interfaces. The use of Clean Architecture ensures that the core business logic remains independent of external frameworks, databases, and third-party services.

1.7 Technologies Used

From analyzing the project, the primary technologies include:

Category	Technology
Backend	ASP.NET Core (.NET 9)
ORM	Entity Framework Core
Database	SQL Server
Chat Database	MongoDB
Authentication	JWT (JSON Web Tokens)
Real-Time Communication	SignalR
API Documentation	Swagger / OpenAPI
AI Service	Separate ASP.NET Core AI Engine
Version Control	Git & GitHub

Chapter 2 — System Architecture

2.1 Introduction

The NFS platform follows a **Clean Architecture**, where the application is organized into independent layers with clear responsibilities and dependencies directed toward the core business logic.

Rather than implementing all functionality within a single application, the system separates responsibilities into specialized layers, improving maintainability, scalability, testability, and long-term extensibility.

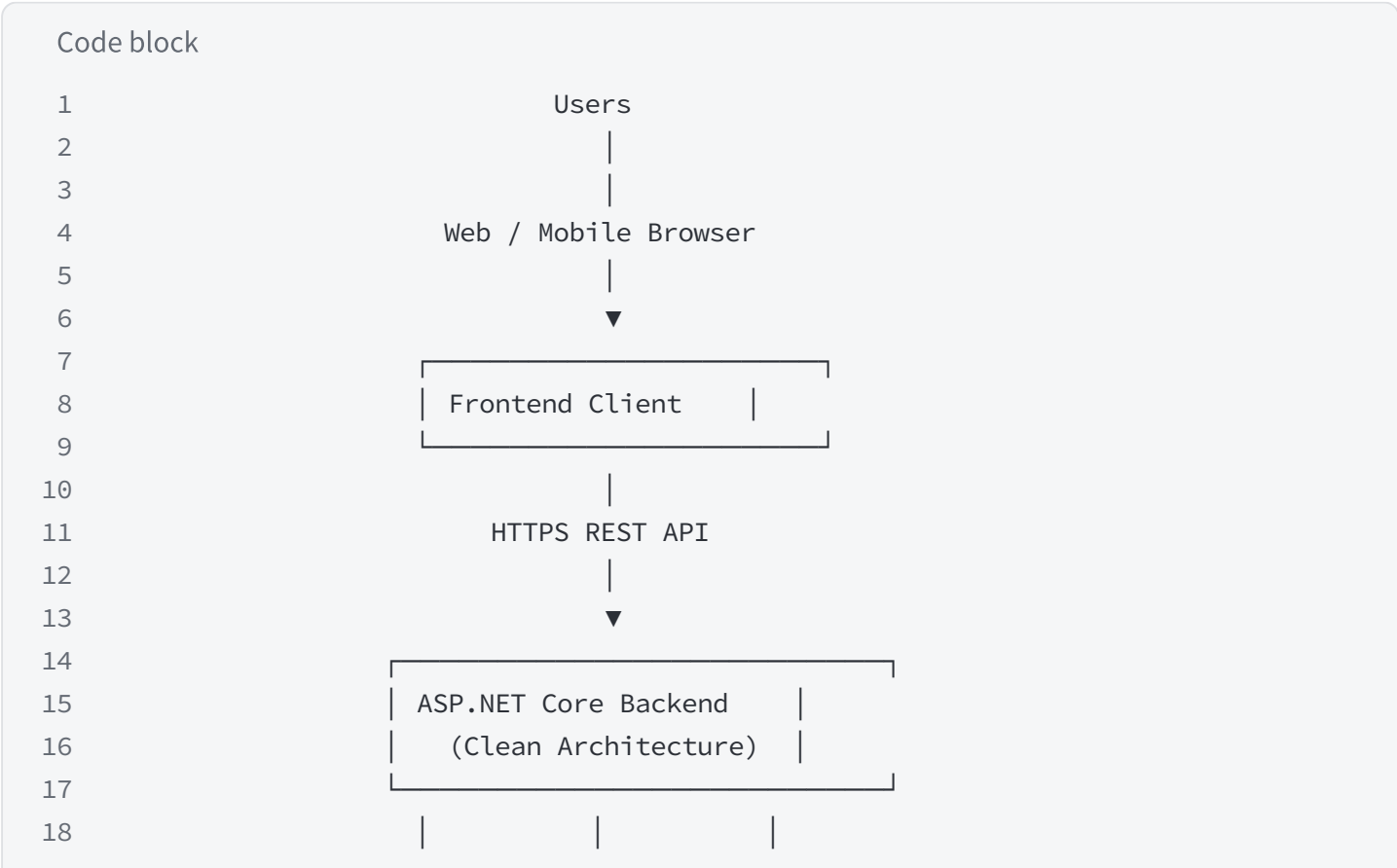
The architecture consists of three primary systems:

- Frontend Application
- Backend REST API
- AI Engine

These components communicate securely using HTTP/HTTPS and RESTful APIs.

2.2 Overall System Architecture

The overall architecture can be represented as follows:



This architecture ensures that:

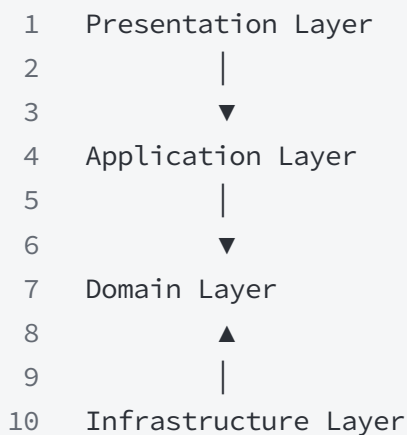
- The frontend focuses only on presentation.
- The backend manages application workflows and business logic through Clean Architecture.
- Databases store persistent information.
- The AI Engine handles intelligent features independently.

2.3 Architectural Style

Based on the analyzed solution, NFS adopts **Clean Architecture**.

The backend is organized into four primary layers, where dependencies always point toward the Domain layer.

Code block



Each layer has a dedicated responsibility:

- **Presentation Layer** handles HTTP requests, controllers, authentication, and communication with clients.
- **Application Layer** contains use cases, DTOs, interfaces, validation, and application services.
- **Domain Layer** contains the core business entities, business rules, and domain logic.
- **Infrastructure Layer** provides implementations for repositories, database access, external APIs, authentication services, and other technical concerns.

This architecture minimizes coupling by ensuring that the business logic remains independent of frameworks, databases, and external services.

2.4 Component Responsibilities

Frontend

Responsibilities include:

- Rendering the user interface.
- Collecting user input.
- Displaying therapist information.
- Booking sessions.
- Showing psychological assessments.
- Displaying AI chat responses.
- Communicating with the backend.

The frontend never communicates directly with the databases.

Backend

The backend is the core processing unit.

Responsibilities include:

- Authentication
- Authorization
- Validation
- Business Logic
- Database Operations
- Payment Processing
- Appointment Scheduling
- Therapist Management
- Patient Management
- AI Communication

Every request passes through the backend before reaching any external service.

SQL Server

Stores structured application data.

Examples include:

- Users
 - Therapists
 - Patients
 - Sessions
 - Bookings
 - Assessments
 - Payments
-

MongoDB

Stores chat-related information.

Using MongoDB for conversations provides flexibility because chat messages are naturally document-oriented.

AI Engine

A standalone service responsible for AI capabilities.

Benefits include:

- Independent deployment
 - Easier upgrades
 - Better scalability
 - Isolation from business logic
-

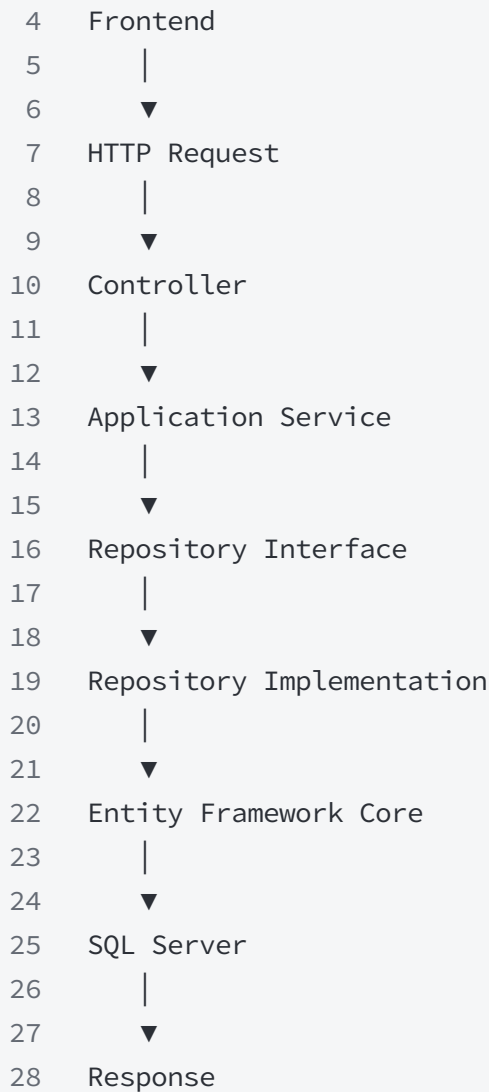
2.5 Request Lifecycle

Every user request follows a consistent lifecycle.

Example: Patient Books a Session

Code block

```
1 Patient
2 |
3 ▼
```

This flow follows Clean Architecture principles by ensuring that business logic is handled in the Application and Domain layers, while data access remains isolated within the Infrastructure layer.

2.6 Dependency Injection

The backend extensively uses Dependency Injection (DI).

Instead of manually creating objects, services and repositories are injected automatically by [ASP.NET Core](#).

Example concept:

Code block

```
1 public AuthController(IAuthService authService)
2 {
3     _authService = authService;
```

Advantages

- Loose coupling
 - Easier testing
 - Better maintainability
 - Improved scalability
-

2.7 Application Layer

Business logic is isolated inside the **Application Layer**.

Instead of placing business rules inside controllers, controllers delegate processing to application services, which coordinate use cases and communicate with the Domain and Infrastructure layers.

Benefits include:

- Reusable business logic
 - Cleaner controllers
 - Easier unit testing
 - Better separation of concerns
 - Independent business workflows
-

2.8 Repository Pattern

Database operations are separated into repositories.

In Clean Architecture, repository **interfaces** are defined inside the Application layer, while their implementations are located in the Infrastructure layer. This allows the business logic to remain independent of database technologies.

Advantages include:

- Cleaner code
 - Easier maintenance
 - Database abstraction
 - Improved testability
-

2.9 Security Architecture

The platform implements several security mechanisms:

- JWT Authentication
- Role-Based Authorization
- Password Hashing
- HTTPS Communication
- Protected API Endpoints

JWT tokens authenticate users after login, and authorization policies determine which resources each role may access.

2.10 Real-Time Communication

The project integrates SignalR to enable real-time communication.

SignalR allows:

- Instant messaging
- Live notifications
- Immediate updates without refreshing the page

The backend exposes a SignalR Hub that authenticated users connect to for chat functionality.

2.11 Database Strategy

The project uses polyglot persistence, combining two database technologies:

This content is only supported in a Lark Docs

This approach allows each database to be used for the type of data it handles best.

2.12 Architectural Strengths

The current architecture provides several advantages:

- Clear separation of responsibilities.
- Independent Application, Domain, and Infrastructure layers.
- Modular project organization.
- Independent AI service.

- Scalable backend design.
- Modern authentication using JWT.
- Real-time communication through SignalR.
- Combination of relational and document databases.
- Improved maintainability and testability through dependency inversion.

These characteristics make the platform suitable for future expansion and easier team collaboration.

2.13 Areas for Future Improvement

Although the architecture is solid, several enhancements could be considered as the platform grows:

- Introduce CQRS to separate complex read and write operations.
- Add Redis for caching frequently accessed data.
- Use background job processing (e.g., Hangfire) for long-running tasks like email notifications.
- Implement centralized logging with Serilog and Elasticsearch.
- Containerize services with Docker for easier deployment.
- Introduce API versioning to support future clients.

Chapter 3 – Technology Stack

Chapter 3

3.1 Introduction

Selecting the appropriate technologies is one of the most important decisions in software architecture. Each technology used in the NFS platform was chosen to address specific functional and non-functional requirements, including scalability, maintainability, security, performance, and ease of development.

The platform combines modern Microsoft technologies with document-oriented databases and real-time communication frameworks to create a robust digital mental healthcare system.

3.2 Technology Overview

Category	Technology	Purpose
Backend Framework	ASP.NET Core (.NET 9)	REST API and Business Logic
Programming Language	C#	Backend Development
ORM	Entity Framework Core	Database Access
Relational Database	Microsoft SQL Server	Core System Data
NoSQL Database	MongoDB	Chat Messages
Authentication	JWT (JSON Web Token)	Secure Authentication
Real-Time Communication	SignalR	Instant Messaging
API Documentation	Swagger / OpenAPI	API Testing & Documentation
Dependency Injection	ASP.NET Core DI	Service Management
Version Control	Git & GitHub	Source Code Management
AI Service	Standalone ASP.NET Core Service	AI Features

3.3 [ASP.NET](#) Core (.NET 9)

Overview

The backend of the NFS platform is built using [ASP.NET Core](#), Microsoft's modern cross-platform framework for developing web applications and RESTful APIs.

The project targets **.NET 9**, benefiting from performance improvements, modern language features, and long-term maintainability.

Why [ASP.NET](#) Core?

The framework was selected because it provides:

- High performance
 - Cross-platform support
 - Built-in Dependency Injection
 - Integrated security features
 - Excellent support for REST APIs
 - Rich ecosystem
 - Strong community support
-

Advantages

- High request throughput
 - Lightweight runtime
 - Native Dependency Injection
 - Built-in middleware pipeline
 - Excellent integration with Entity Framework Core
 - Strong security model
-

Potential Limitations

- Developers must be familiar with C# and .NET.
 - Deployment can be more complex than for simpler scripting frameworks.
-

3.3 Role in NFS

[ASP.NET](#) Core serves as the foundation of the backend and hosts the Clean Architecture implementation. It is responsible for:

- API endpoints
- Authentication
- Authorization
- Request processing
- Dependency Injection
- Service registration

- Communication with databases through the Infrastructure layer
 - Execution of application use cases
 - Integration with the AI Engine
-

3.4 C#

Overview

The entire backend is developed using **C#**, Microsoft's object-oriented programming language.

The project leverages modern C# features such as:

- Nullable Reference Types
 - Dependency Injection
 - Asynchronous Programming (`async` / `await`)
 - Interfaces
 - Generics
 - LINQ
-

Why C#?

The language offers:

- Strong typing
 - Excellent tooling
 - High performance
 - Readable syntax
 - Large ecosystem
 - Seamless integration with .NET
-

3.5 Entity Framework Core

Overview

NFS uses **Entity Framework Core (EF Core)** as its Object-Relational Mapper (ORM).

Instead of writing raw SQL for every operation, EF Core maps C# objects to database tables.

Responsibilities

Entity Framework Core manages:

- CRUD operations
 - Database migrations
 - Relationships
 - Change tracking
 - LINQ queries
 - Entity mapping
-

Advantages

- Reduced boilerplate SQL
 - Strong integration with C#
 - Type-safe queries
 - Simplified database maintenance
 - Migration support
-

Example Flow

Code block

```
1  Entity
2      |
3  Entity Framework Core
4      |
5  Generated SQL
6      |
7  SQL Server
```

3.6 Microsoft SQL Server

Overview

SQL Server is the primary relational database used by the platform.

It stores structured business information such as:

- Users
 - Therapists
 - Patients
 - Sessions
 - Assessments
 - Appointments
 - Payments
-

Why SQL Server?

Reasons include:

- ACID compliance
 - High reliability
 - Mature ecosystem
 - Excellent EF Core integration
 - Strong transaction support
-

Advantages

- Reliable transactions
 - Data integrity
 - Relationship management
 - Excellent indexing
 - High performance
-

3.7 MongoDB

Overview

Unlike the main business data, chat messages are stored in **MongoDB**.
MongoDB is a NoSQL document database optimized for flexible document storage.

Why MongoDB?

Chat systems typically generate large numbers of messages with evolving structures.
MongoDB provides:

- Flexible schemas
 - Fast document storage
 - High scalability
 - Efficient handling of nested data
-

Role in NFS

MongoDB stores:

- Chat rooms
 - Conversations
 - Chat messages
 - Message history
-

3.8 JWT Authentication

Overview

The project secures its APIs using **JSON Web Tokens (JWT)**.
After a successful login, the server issues a signed token that the client includes in subsequent requests.

Authentication Flow

Code block

```
1  User Login
2  |
```



Advantages

- Stateless authentication
- High performance
- Easy API protection
- Industry standard

3.9 SignalR

Overview

The platform supports real-time communication using **SignalR**.

SignalR allows connected clients to receive updates immediately without repeatedly polling the server.

Role in NFS

SignalR enables:

- Instant messaging
- Live chat
- Real-time notifications

Benefits

- Improved user experience
 - Reduced latency
 - Efficient communication
-

3.10 Swagger / OpenAPI

Overview

Swagger provides interactive API documentation.

Developers can:

- Explore endpoints
- Send requests
- View responses
- Test authentication

without writing additional code.

Why Swagger?

Benefits include:

- Faster backend testing
 - Easier frontend integration
 - Automatic API documentation
 - Improved developer productivity
-

3.11 Dependency Injection

The backend relies on the built-in Dependency Injection container provided by [ASP.NET Core](#).

Instead of manually creating services, dependencies are injected automatically at runtime.

Benefits include:

- Loose coupling

- Better testing
 - Cleaner code
 - Easier maintenance
-

3.12 Git & GitHub

Version control is managed using Git.

GitHub serves as the central repository for:

- Source code
- Collaboration
- Branch management
- Code reviews
- Version history

This allows multiple developers to work on the project simultaneously while maintaining a consistent codebase.

3.13 AI Service

One of the distinctive aspects of NFS is its separate AI Engine.

Rather than embedding AI logic inside the backend, the project isolates AI functionality into an independent service.

Advantages

- Independent deployment
 - Easier maintenance
 - Better scalability
 - Simplified future AI upgrades
-

3.14 Why This Technology Stack?

The selected technologies complement each other well:

Requirement	Technology
Secure APIs	ASP.NET Core + JWT
Business Logic	C#
Relational Data	SQL Server
Chat Storage	MongoDB
Real-Time Features	SignalR
Database Access	Entity Framework Core
API Documentation	Swagger
Intelligent Features	Standalone AI Service

Together, they create a scalable, secure, and maintainable architecture capable of supporting a modern mental healthcare platform.

Chapter 4 – Solution Structure & Project Organization

4.1 Introduction

A well-organized project structure is essential for building scalable and maintainable software. Instead of placing all source code into a single project, the NFS platform divides responsibilities across multiple projects, each with a clearly defined role.

The solution is organized according to the principles of **Clean Architecture**, which emphasizes separation of concerns, dependency inversion, and independent business logic. This organization allows developers to work independently on different parts of the system while reducing coupling between components.

The NFS solution consists of six main projects, each responsible for a specific role within the Clean Architecture.

4.2 Solution Structure

The solution is organized as follows:

Code block

```
1  NFS Solution
2  |
```

```

3  |— NFS.API
4  |— NFS.Application
5  |— NFS.Domain
6  |— NFS.Infrastructure
7  |— NFS.Chat
8  |— chatbot (AI Engine)

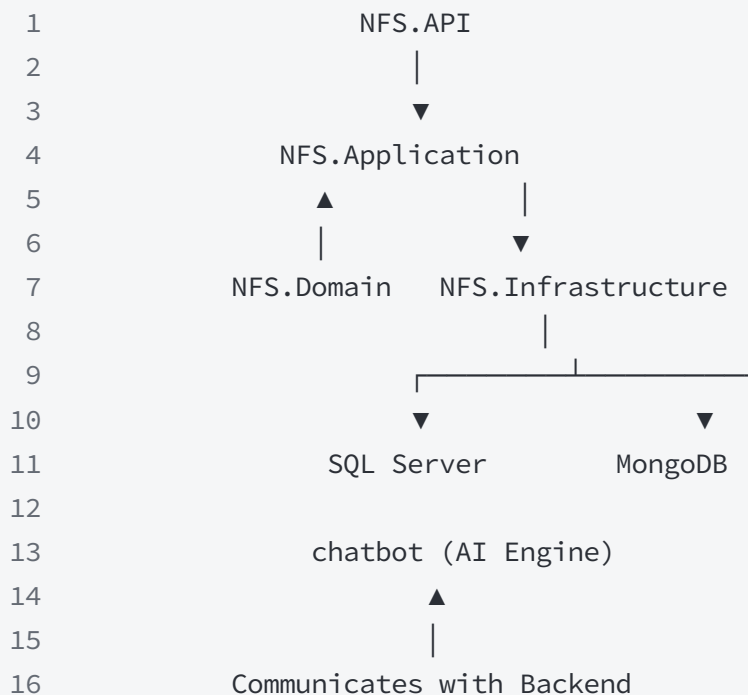
```

Each project communicates only with the layers it needs, preventing unnecessary dependencies and keeping the architecture clean.

4.3 Overall Dependency Diagram

The relationship between projects can be illustrated as follows:

Code block



Explanation

- **NFS.API** is the application's entry point and exposes REST API endpoints.
- **NFS.Application** contains application use cases, DTOs, interfaces, validation, and business workflows.
- **NFS.Domain** contains the core business entities and business rules and has no dependency on external frameworks.
- **NFS.Infrastructure** implements repository interfaces, database access, authentication services, and integrations with external systems.

- **NFS.Chat** encapsulates SignalR hubs and chat-related functionality using MongoDB.
- **chatbot** is an independent AI service that communicates with the backend through APIs.

In accordance with Clean Architecture, dependencies always point toward the Domain layer, ensuring that the core business logic remains independent of infrastructure and external technologies.

4.4 NFS.API

Purpose

`NFS.API` is the presentation layer of the application.

It receives requests from frontend clients and returns HTTP responses.

This project is responsible for:

- Controllers
- Program.cs
- Authentication Configuration
- Swagger Configuration
- Dependency Injection
- Middleware
- Routing
- API Endpoints

Responsibilities

- Receive HTTP Requests
- Validate incoming requests
- Call Application Services
- Return HTTP Responses
- Authenticate users
- Authorize users

What should NOT exist here?

Business logic should not be implemented inside controllers.

Controllers should remain lightweight and delegate processing to the Application layer.

4.5 NFS.Application

Purpose

The Application project contains the application's use cases and coordinates the execution of business operations. It acts as the bridge between the Presentation layer (API) and the Domain layer.

Typical contents include:

- Application Services
- Use Cases
- Interfaces
- DTOs
- Validators
- Mapping
- Repository Interfaces

Responsibilities

- Execute application use cases
- Coordinate repositories
- Apply business rules using domain entities
- Validate application requests
- Convert entities into DTOs
- Coordinate external services

Why separate this layer?

Keeping application logic outside controllers improves:

- Reusability
- Testability
- Readability
- Maintainability
- Separation of concerns

4.6 NFS.Domain

Purpose

The Domain project represents the core business model of the platform.

It contains the entities that describe the problem domain independently of any framework or database.

Typical contents include:

- Entities
- Enums
- Value Objects
- Domain Interfaces

Examples of entities include:

- User
- Therapist
- Patient
- Appointment
- Session
- Assessment

Responsibilities

- Represent business concepts
- Define relationships
- Encapsulate business data

The Domain project should not depend on [ASP.NET](#) Core, Entity Framework Core, or any external infrastructure.

4.7 NFS.Infrastructure

Purpose

The Infrastructure project contains implementations for interacting with external systems.

Responsibilities include:

- Database access
- Entity Framework Core configuration

- Repository implementations
- Database migrations
- File storage
- Third-party integrations

Database Responsibilities

- DbContext
- Repository Pattern
- SQL Server configuration
- Data persistence

This layer hides infrastructure details from the Application layer.

4.8 NFS.Chat

Purpose

Rather than embedding chat functionality into the API, the project isolates chat-related components into a dedicated project.

Possible responsibilities include:

- SignalR hubs
- Chat services
- Message management
- Chat repositories
- MongoDB integration

Benefits

- Independent maintenance
 - Better modularity
 - Easier future expansion
-

4.9 chatbot (AI Engine)

Purpose

The AI Engine is implemented as a completely independent service.

Instead of embedding AI processing into the backend, intelligent features are isolated into a dedicated application.

Responsibilities include:

- AI conversations
- Intelligent responses
- Recommendation logic
- Future machine learning integration

Advantages

- Independent deployment
 - Separate scaling
 - Easier maintenance
 - Better fault isolation
-

4.10 Folder Organization

Although each project has its own internal structure, the solution generally follows a consistent organization.

Typical folders include:

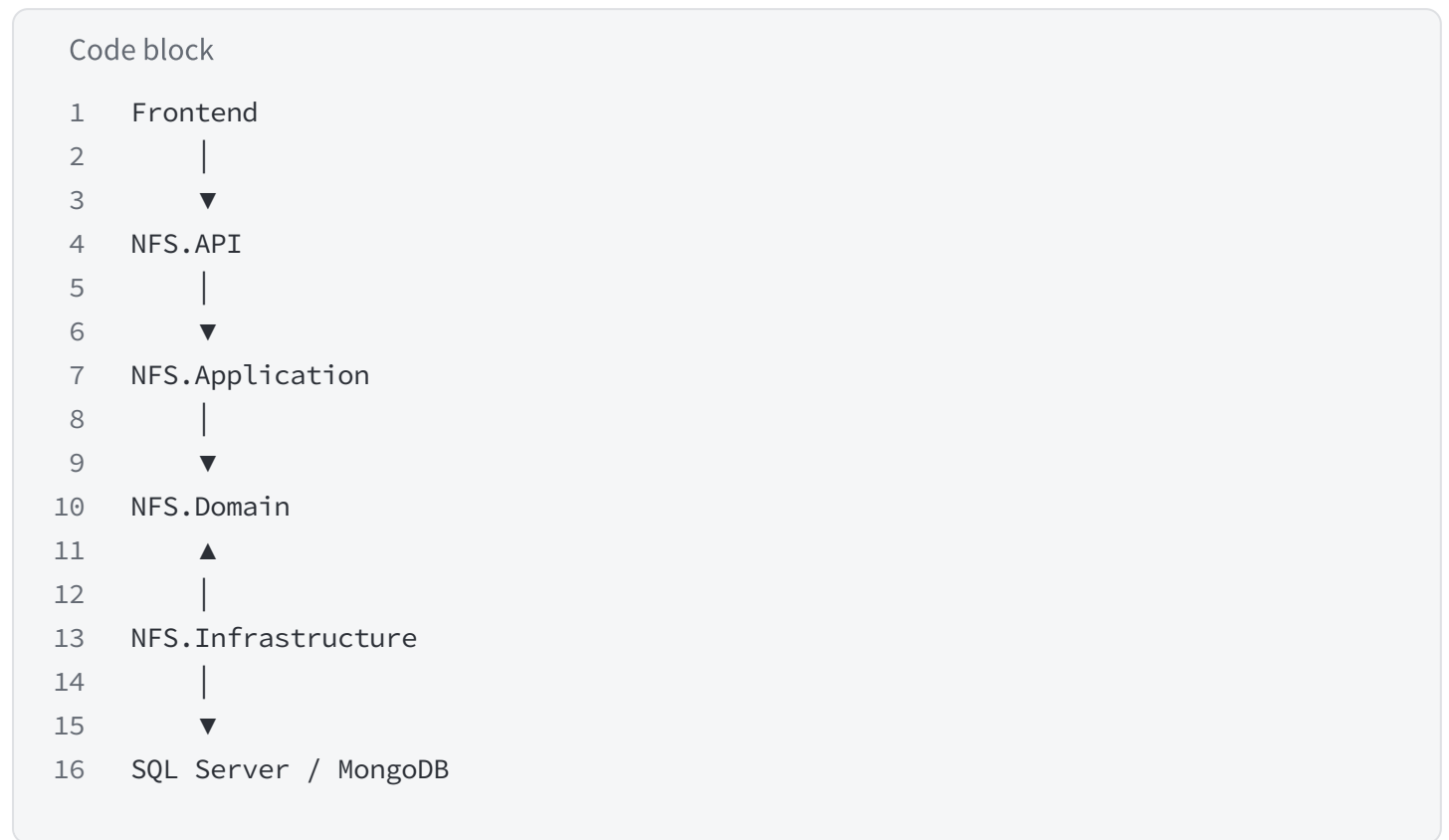
Code block

```
1  Controllers/  
2  Services/  
3  Repositories/  
4  Interfaces/  
5  DTOs/  
6  Entities/  
7  Configurations/  
8  Migrations/  
9  Middleware/  
10 Helpers/
```

Each folder groups files with similar responsibilities, making the solution easier to navigate.

4.11 Layer Communication

A request flows through the system in the following order:



The API communicates with the Application layer.

The Application layer depends only on the Domain layer and repository interfaces.

The Infrastructure layer implements those interfaces and communicates with SQL Server, MongoDB, and other external services.

This dependency direction follows the Dependency Inversion Principle, ensuring that the business logic remains independent of frameworks and infrastructure.

4.12 Benefits of the Current Organization

The current solution structure provides several advantages:

Modularity

Each project focuses on a single responsibility, making the codebase easier to understand and maintain.

Scalability

New features can be added with minimal impact on existing modules.

Team Collaboration

Different developers can work on different projects simultaneously with fewer merge conflicts.

Testability

Business logic can be tested independently from the API and infrastructure.

Maintainability

Changes in one layer are less likely to affect other layers.

4.13 Potential Improvements

As the platform evolves, the following enhancements could be considered:

- Introduce **CQRS** to separate read and write operations.
- Add **MediatR** for request handling and decoupling.
- Introduce **Domain Events** for complex workflows.
- Containerize each project using **Docker**.
- Add automated architecture validation tests.

Chapter 5 – Backend Architecture

5.1 Introduction

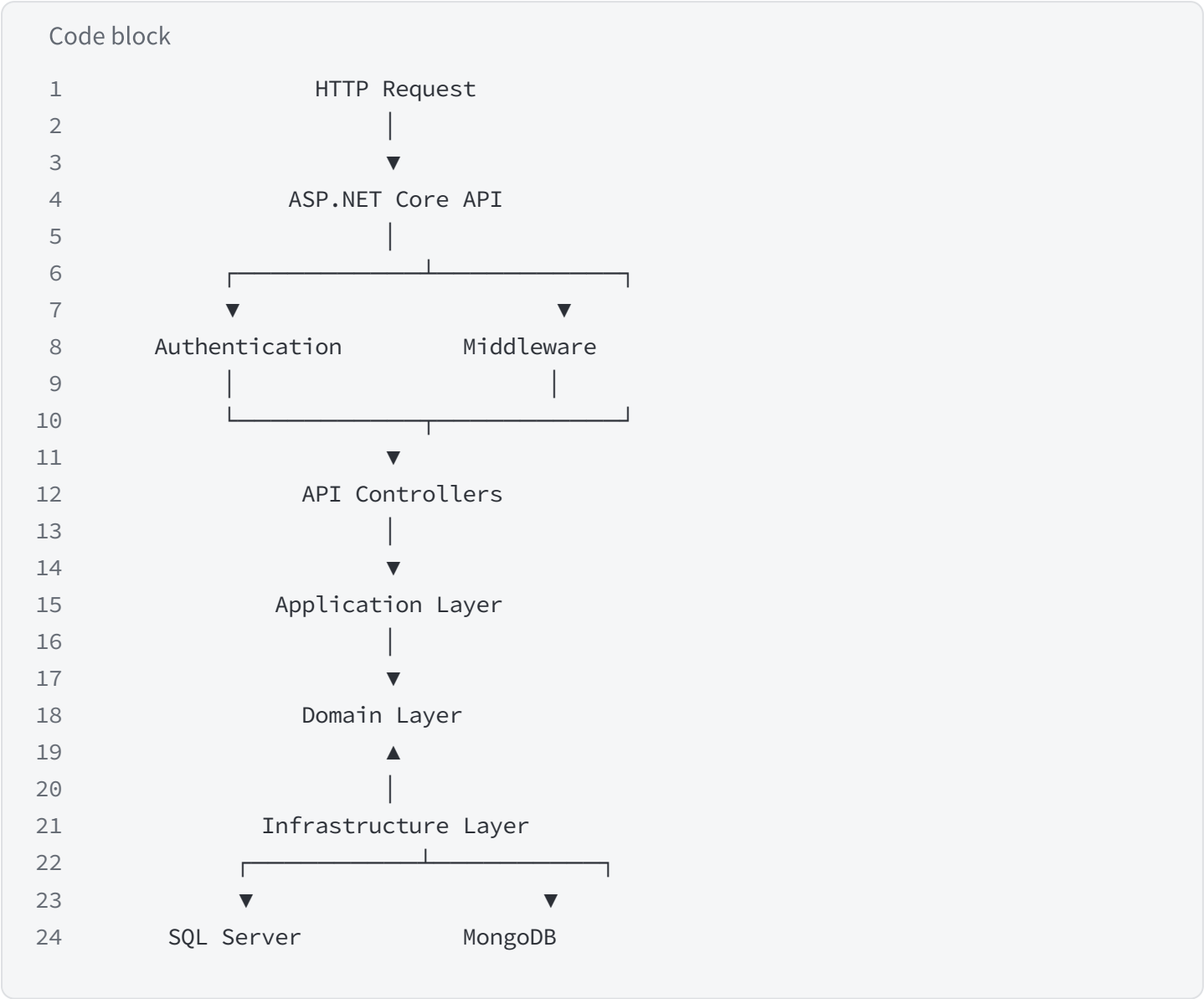
The backend of the NFS platform serves as the central processing unit of the system. It is responsible for receiving requests from the frontend, validating user input, executing business logic, communicating with the databases, interacting with external services, and returning structured responses.

The backend is implemented using [ASP.NET](#) Core (.NET 9) and follows the **Clean Architecture** approach, where the system is organized into independent layers with clear responsibilities and dependencies directed toward the core business logic. This architectural approach improves maintainability, scalability, testability, and flexibility.

Unlike monolithic implementations where business logic is embedded directly within controllers, the NFS backend separates responsibilities into Presentation, Application, Domain, and Infrastructure layers, ensuring that each component focuses on a single responsibility.

5.2 Backend Architecture Overview

The backend architecture is organized according to Clean Architecture principles.



This design ensures that business rules remain independent of frameworks, databases, and external services while each layer performs a well-defined responsibility.

5.3 Backend Projects

The backend consists of five cooperating projects:

Project	Responsibility
NFS.API	Entry point, Controllers, Middleware, Authentication, Swagger
NFS.Application	Business Logic, Services, DTOs, Interfaces
NFS.Domain	Business Entities and Domain Models
NFS.Infrastructure	Database Access, Entity Framework Core, Repository Implementations
NFS.Chat	SignalR Hub, Chat Services, MongoDB Integration

This separation improves maintainability by isolating business rules from infrastructure and presentation concerns in accordance with Clean Architecture principles.

5.4 Application Startup

Application execution begins in **Program.cs**.

During startup the application performs several configuration steps:

1. Creates the application builder.
2. Registers application and infrastructure services.
3. Configures Entity Framework Core.
4. Configures SQL Server.
5. Configures MongoDB.
6. Configures JWT Authentication.
7. Registers Dependency Injection.
8. Enables Swagger.
9. Configures CORS.

10. Seeds the databases.
11. Builds the middleware pipeline.
12. Starts listening for HTTP requests.

Startup Flow

Code block

```
1  Application Starts
2      |
3      ▼
4  Read Configuration
5      |
6      ▼
7  Register Services
8      |
9      ▼
10 Configure Databases
11     |
12     ▼
13 Configure Authentication
14     |
15     ▼
16 Configure Middleware
17     |
18     ▼
19 Map Controllers
20     |
21     ▼
22 Application Ready
```

5.5 Configuration Management

The backend retrieves its configuration from **appsettings.json** rather than embedding values in the source code.

The configuration includes:

- SQL Server Connection String
- MongoDB Settings
- JWT Settings
- Logging Configuration

- Allowed Hosts

Separating configuration from code simplifies deployment across development, testing, and production environments.

5.6 Dependency Injection

Dependency Injection is extensively used throughout the backend.

Instead of creating service instances manually, the framework injects required dependencies automatically.

Example:

Code block

```
1 public AuthController(IAuthService authService)
2 {
3     _authService = authService;
4 }
```

Benefits

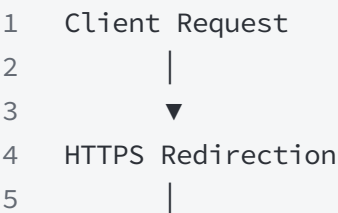
- Loose coupling
 - Easier testing
 - Better scalability
 - Cleaner constructors
 - Improved maintainability
-

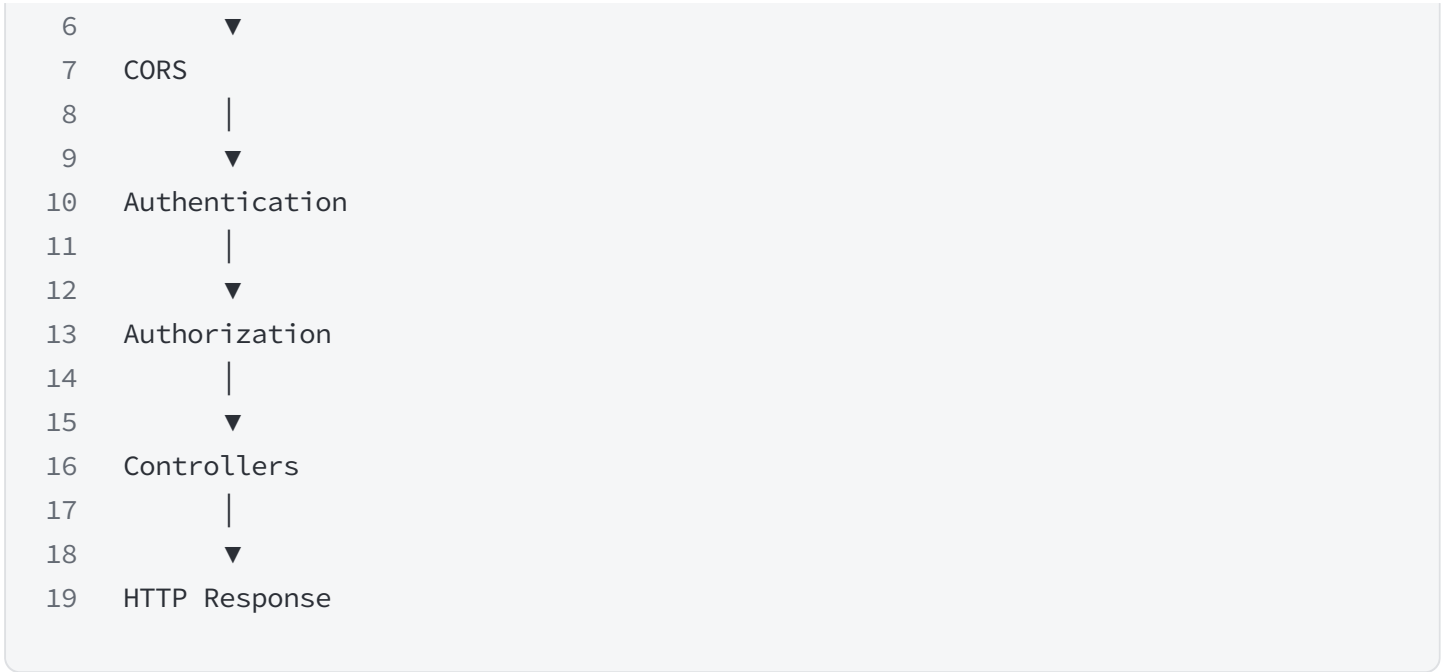
5.7 Middleware Pipeline

Every incoming HTTP request passes through a middleware pipeline before reaching a controller.

The execution order is:

Code block





Middleware Responsibilities

Middleware	Purpose
HTTPS Redirection	Forces secure communication
CORS	Controls cross-origin requests
Authentication	Identifies the user
Authorization	Verifies user permissions
Routing	Directs requests to controllers

5.8 Controller Layer

Controllers expose RESTful API endpoints to frontend applications.

Responsibilities include:

- Receiving HTTP requests
- Validating request models
- Calling the Application layer
- Returning HTTP responses

Controllers intentionally contain minimal business logic and simply coordinate requests between clients and the Application layer.

5.9 Application Layer

The Application layer contains the application's use cases and coordinates business workflows.

Responsibilities include:

- Executing application use cases
- Business validation
- Workflow coordination
- DTO mapping
- Calling repository interfaces
- Coordinating domain entities
- Communicating with external services through abstractions

Unlike controllers, the Application layer contains the implementation of application behavior while remaining independent of infrastructure.

5.10 Infrastructure Layer

The Infrastructure layer provides implementations for services that interact with external systems.

Typical responsibilities include:

- Entity Framework Core
- Repository implementations
- Database access
- Data persistence
- External API integrations
- Authentication services
- File storage

Advantages:

- Infrastructure abstraction
- Easier testing

- Separation of technical concerns
 - Independent business logic
-

5.11 Domain Layer

The Domain layer represents the core business concepts of the platform.

It defines:

- Entities
- Relationships
- Enums
- Value Objects
- Domain rules

Examples include:

- User
- Therapist
- Patient
- Session
- Appointment
- Assessment

The Domain layer remains completely independent of [ASP.NET](#) Core, Entity Framework Core, SQL Server, MongoDB, and any external framework.

5.12 Database Integration

The backend uses two different databases.

SQL Server

Stores structured business information:

- Users
- Therapists
- Patients
- Sessions

- Assessments
- Bookings
- Payments

MongoDB

Stores:

- Chat rooms
- Messages
- Conversations

This dual-database strategy leverages the strengths of both relational and document databases.

5.13 Authentication & Authorization

The backend secures protected resources using **JWT (JSON Web Tokens)**.

Authentication Process

Code block

```
1  User Login
2  |
3  ▼
4  Validate Credentials
5  |
6  ▼
7  Generate JWT
8  |
9  ▼
10 Return Token
11 |
12 ▼
13 Client Sends Token
14 |
15 ▼
16 Protected API Access
```

Authorization is then applied to determine whether the authenticated user has permission to perform the requested action.

5.14 SignalR Integration

The backend integrates **SignalR** to support real-time communication.

SignalR is used for:

- Instant messaging
- Live notifications
- Real-time updates

Clients establish a persistent connection with the SignalR Hub, enabling bidirectional communication without repeated polling.

5.15 API Documentation

Swagger is integrated into the backend to automatically generate interactive API documentation.

Benefits include:

- Endpoint discovery
 - Request testing
 - JWT authentication testing
 - Improved frontend integration
 - Reduced development time
-

5.16 Backend Strengths

The backend architecture demonstrates several strengths:

- Clean Architecture
- Clear separation of responsibilities
- Dependency Injection
- Repository Pattern
- Application Layer
- Domain-driven organization
- JWT Authentication
- SignalR integration
- Dual-database strategy
- Automatic API documentation

- Independent infrastructure implementations

These design choices contribute to a maintainable, scalable, and testable backend.

5.17 Recommendations for Future Enhancement

To further improve the backend architecture, the following enhancements could be considered:

- Introduce **MediatR** for request/response handling.
- Adopt **CQRS** for complex read/write operations.
- Add **Redis** for distributed caching.
- Implement **FluentValidation** for centralized request validation.
- Use **Serilog** with structured logging.
- Add **Docker** support for containerized deployment.
- Introduce comprehensive **unit and integration tests**.

Chapter 6 – Frontend Architecture

6.1 Introduction

The frontend of the NFS Mental Health Platform is responsible for providing an intuitive, responsive, and user-friendly interface that enables patients, therapists, and administrators to interact with the system's services. While the backend implements the business logic using **Clean Architecture**, the frontend focuses on presenting data, collecting user input, managing application state, and delivering a seamless user experience.

The frontend was developed using **React.js**, adopting a component-based architecture that promotes modularity, reusability, and maintainability. It communicates exclusively with the backend through secure RESTful APIs over HTTPS. The frontend never accesses the database directly, ensuring that all business rules, validation, and security policies remain centralized within the backend.

6.2 Frontend Objectives

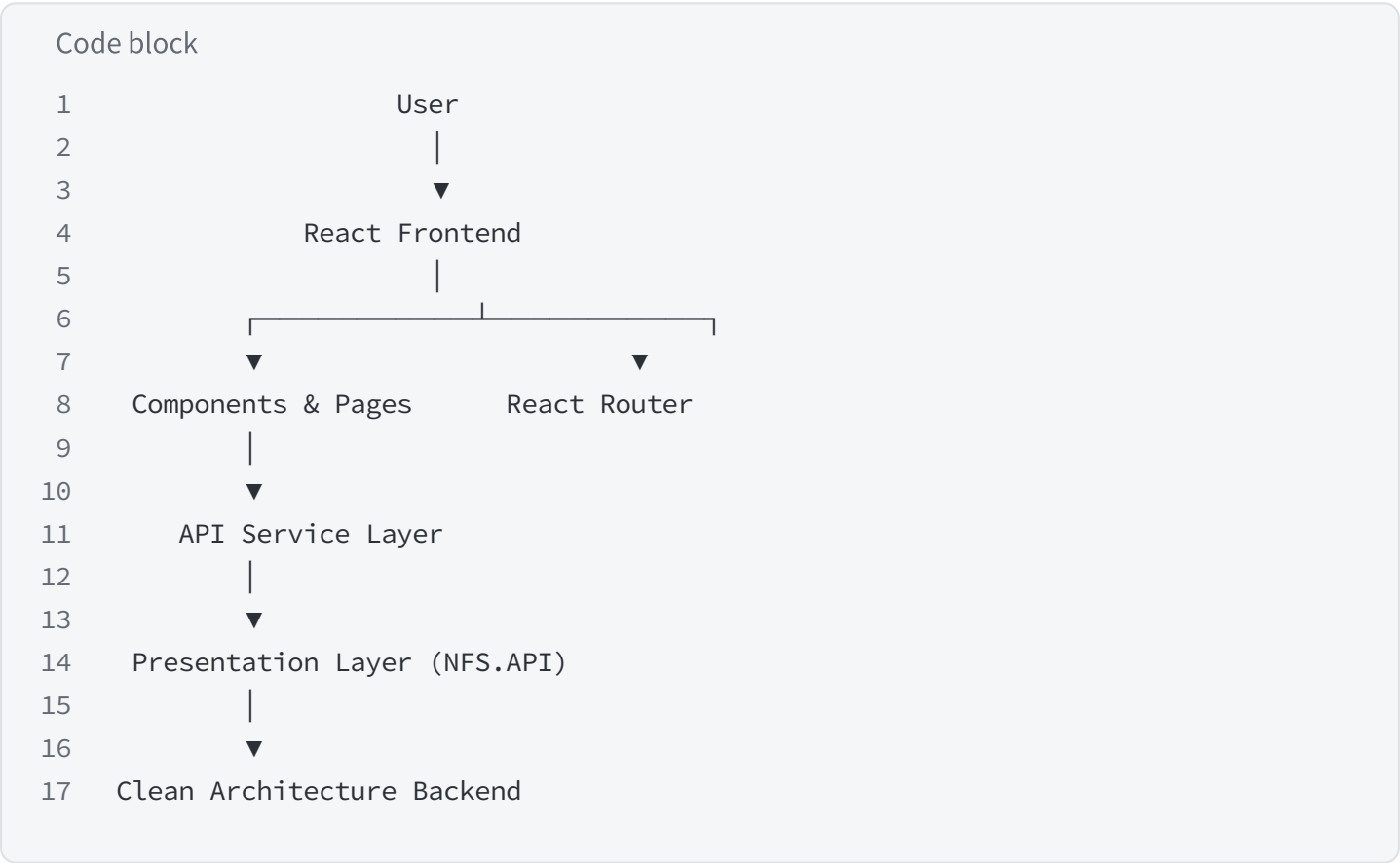
The frontend was designed to achieve the following objectives:

- Provide an intuitive and responsive user interface.
- Support secure user authentication.
- Enable therapist search and profile viewing.

- Allow appointment booking and management.
- Display patient and therapist information.
- Support psychological assessments.
- Provide real-time chat functionality.
- Consume backend RESTful APIs efficiently.
- Ensure responsive design across multiple devices.

6.3 Frontend Architecture Overview

The frontend follows a component-based architecture where the user interface is divided into reusable React components.



This architecture separates presentation from backend communication, improving maintainability and scalability.

6.4 Frontend Layers

The frontend can be divided into four logical layers.

Layer	Responsibility
Presentation Layer	User Interface
Component Layer	Reusable UI Components
Service Layer	REST API Communication
Routing Layer	Navigation Between Pages

Each layer performs a dedicated role while remaining loosely coupled with the others.

6.5 Main Pages

The frontend provides different interfaces according to the authenticated user's role.

Public Pages

- Landing Page
- About Us
- Contact Us
- Login
- Registration

Patient Pages

- Dashboard
- Profile
- Therapist Search
- Therapist Details
- Appointment Booking
- My Appointments
- Psychological Assessments
- Chat
- Notifications
- Payment History

- Personal Diary

Therapist Pages

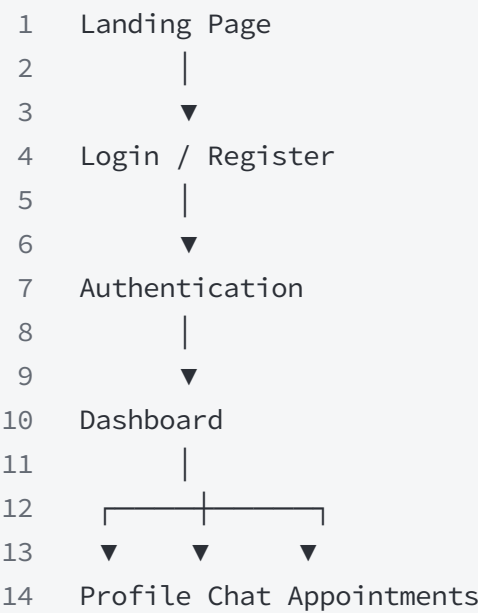
- Dashboard
- Profile
- Schedule Management
- Patient List
- Session Management
- Assessment Results
- Earnings

Administrator Pages

- Dashboard
- User Management
- Therapist Approval
- Reports
- System Monitoring

6.6 Navigation Flow

Code block



After authentication, navigation is determined by the authenticated user's role and permissions.

6.7 Authentication Flow

Authentication is implemented using JWT (JSON Web Tokens).

Code block

```
1  User Login
2      |
3      ▼
4  Login Form
5      |
6      ▼
7  POST /api/Auth/login
8      |
9      ▼
10 Authentication API
11     |
12     ▼
13 JWT Access Token
14     |
15     ▼
16 Store Token
17     |
18     ▼
19 Authorized API Requests
```

After successful authentication, the frontend stores the JWT access token securely and includes it in the Authorization header for all protected API requests.

6.8 API Communication

The frontend communicates with the backend exclusively through RESTful APIs.

Typical HTTP operations include:

Operation	HTTP Method
Retrieve Data	GET
Create Data	POST
Update Data	PUT
Delete Data	DELETE

Communication workflow:



Using dedicated service classes centralizes API communication and simplifies future maintenance.

6.9 State Management

The frontend maintains application state for:

- Logged-in user
- Authentication token
- Current user role
- Selected therapist

- Appointment information
- Notifications
- Chat conversations
- Assessment results

Centralized state management ensures consistent data across different pages while reducing unnecessary API requests.

6.10 Form Validation

Client-side validation is performed before requests are submitted to the backend.

Validation includes:

- Required fields
- Email format
- Password confirmation
- Password strength
- Phone number validation
- Date validation
- Numeric validation

These validations improve user experience by providing immediate feedback while the backend performs additional validation to ensure data integrity.

6.11 Error Handling

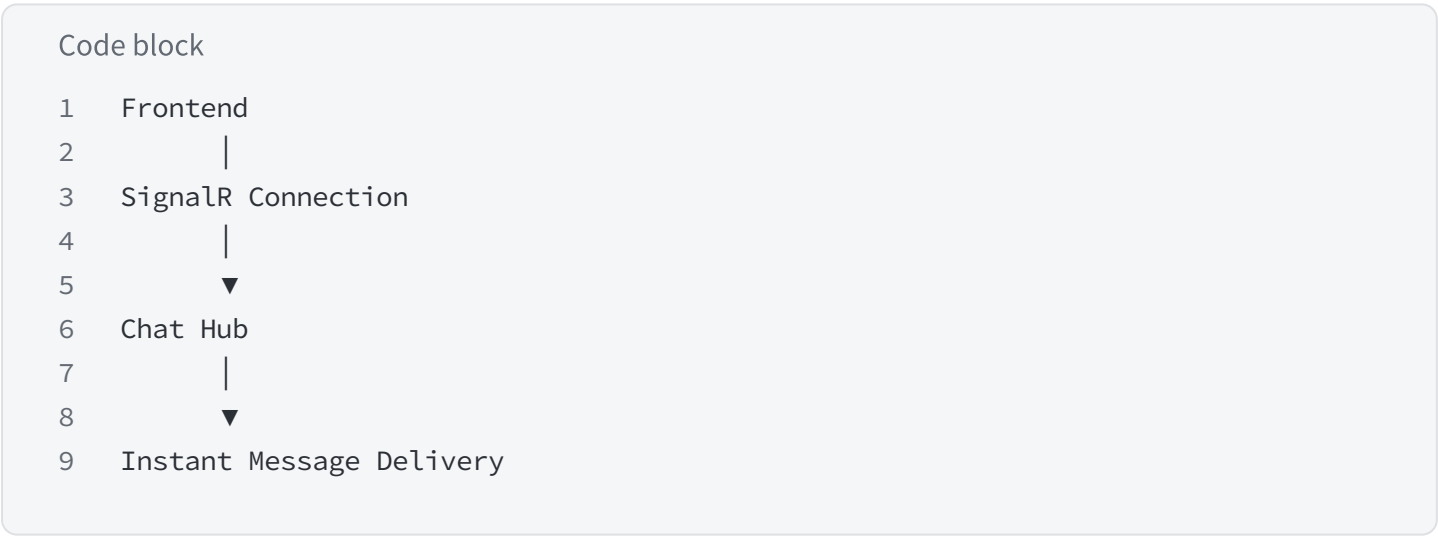
The frontend gracefully handles different categories of errors.

Error Type	Handling
Network Failure	Display retry message
Unauthorized Access	Redirect to Login
Validation Error	Display field-specific messages
Server Error	Display friendly notification
Resource Not Found	Display custom 404 page

Proper error handling improves usability and assists users in resolving issues efficiently.

6.12 Real-Time Communication

The chat module supports real-time communication using **SignalR**.



This enables users to exchange messages and receive notifications immediately without refreshing the page.

6.13 UI Components

The interface is composed of reusable React components, including:

- Navigation Bar

- Sidebar
- Footer
- Authentication Forms
- Therapist Cards
- Appointment Cards
- Assessment Forms
- Chat Window
- Notification Panel
- Modal Dialogs
- Loading Indicators
- Error Messages

Reusable components improve consistency and reduce duplicated code throughout the application.

6.14 Security Considerations

The frontend contributes to application security by:

- Sending JWT tokens only over HTTPS.
 - Restricting access to protected routes.
 - Clearing authentication data after logout.
 - Preventing unauthorized navigation.
 - Displaying role-specific content after successful authentication.
 - Validating user input before submission.
-

6.15 Responsive Design

The frontend is designed to operate efficiently across multiple devices.

Supported platforms include:

- Desktop
- Laptop
- Tablet

- Mobile

Responsive layouts ensure a consistent user experience regardless of screen size or resolution.

6.16 Frontend Strengths

The frontend architecture provides several advantages:

- Modular component-based architecture.
- Clear separation between presentation and API communication.
- Seamless integration with the backend Clean Architecture.
- Secure JWT authentication workflow.
- Reusable UI components.
- Real-time communication using SignalR.
- Responsive and user-friendly interface.
- Scalable project organization.

These characteristics improve maintainability, scalability, and overall application quality.

6.17 Recommendations

Possible future enhancements include:

- Progressive Web Application (PWA) support.
- Dark mode.
- Multi-language localization (Arabic/English).
- Accessibility improvements (WCAG compliance).
- Lazy loading for faster performance.
- Automated frontend testing.
- Push notifications.
- AI-powered user assistance.

Chapter 7 – API Layer Implementation Analysis

7.1 Introduction

The **NFS.API** project is the entry point of the backend system and represents the **Presentation Layer** of the NFS Clean Architecture. It is responsible for receiving HTTP requests from the frontend, validating incoming data, delegating processing to the Application layer, and returning standardized HTTP responses.

Based on the implementation analysis of the uploaded source code, the API project follows the [ASP.NET](#) Core MVC pattern using RESTful controllers. Business logic is intentionally separated from controllers and implemented within the Application layer through dependency injection, ensuring that controllers remain lightweight and focused solely on request handling.

7.2 API Project Structure

The API project is organized around controllers and application startup configuration.

Code block

```
1  NFS.API
2  |
3  |— Controllers
4  |   |— AppointmentsController.cs
5  |   |— AssessmentsController.cs
6  |   |— AuthController.cs
7  |   |— ChatController.cs
8  |   |— DebugController.cs
9  |   |— DiariesController.cs
10 |   |— PatientsController.cs
11 |   |— PaymentsController.cs
12 |   |— SessionsController.cs
13 |   |— SpecializationsController.cs
14 |   |— TherapistsController.cs
15 |   |— UsersController.cs
16 |
17 |— Program.cs
18 |— appsettings.json
19 |— Properties
```

This organization demonstrates a feature-oriented API where each business domain is represented by its own controller.

7.3 Application Startup Analysis

The application starts execution from **Program.cs**.

The startup sequence performs the following operations:

1. Creates the application builder.
2. Registers MVC Controllers.
3. Enables API Explorer.
4. Configures Swagger.
5. Configures JWT authentication inside Swagger.
6. Registers the SQL Server DbContext.
7. Registers Dependency Injection services.
8. Registers MongoDB services.
9. Registers Chat services.
10. Builds the application.
11. Configures middleware.
12. Maps controller routes.
13. Starts listening for HTTP requests.

This centralized startup configuration follows the minimal hosting model introduced in recent versions of [ASP.NET](#) Core.

7.4 Dependency Injection Analysis

The implementation makes extensive use of [ASP.NET](#) Core's built-in Dependency Injection container.

Rather than instantiating objects directly inside controllers, dependencies are injected through constructors.

For example, the **AuthController** receives an implementation of **IAuthService** through constructor injection. This design keeps controllers independent from concrete service implementations and promotes loose coupling between the Presentation layer and the Application layer.

This approach improves:

- Maintainability
- Testability
- Code readability

- Extensibility
-

7.5 Database Configuration

The API registers an **ApplicationDbContext** using Entity Framework Core.

The DbContext is configured to connect to SQL Server using the **DefaultConnection** connection string defined in the application configuration.

Instead of allowing the Application layer to depend directly on Entity Framework Core, the project exposes database operations through abstractions and repository interfaces, while the Infrastructure layer provides their implementations. This follows the Dependency Inversion Principle used in Clean Architecture.

7.6 Swagger Configuration Analysis

The implementation contains a customized Swagger configuration.

The generated documentation:

- Creates version **v1** of the API.
- Defines the API title as **NFS API**.
- Registers a **Bearer** security definition.
- Allows authenticated endpoint testing directly from Swagger UI.

By integrating JWT authentication into Swagger, developers can test protected endpoints without external tools such as Postman.

7.7 Controller Analysis

The implementation contains twelve controllers that are organized according to business functionality.

Controller	Responsibility
AuthController	Authentication and authorization
UsersController	User management
TherapistsController	Therapist management
PatientsController	Patient management
AppointmentsController	Appointment management
SessionsController	Therapy session management
AssessmentsController	Psychological assessments
PaymentsController	Payment processing
SpecializationsController	Therapist specializations
ChatController	Chat operations
DiariesController	Personal diary management
DebugController	Development and debugging utilities

The naming convention follows [ASP.NET](#) Core best practices, where each controller represents a single business domain.

7.8 AuthController Implementation Analysis

The **AuthController** demonstrates the architectural style used throughout the API.

The controller is decorated with:

- `ApiController`
- `Route("api/[controller]")`

which enables automatic model validation and RESTful routing.

The controller depends on a single service:

- IAuthService

No business rules are implemented directly inside the controller.

Instead, each endpoint delegates processing to the Application layer, which coordinates business workflows and communicates with the Domain and Infrastructure layers.

The implementation currently includes authentication endpoints such as:

(Keep your Lark content.)

Before calling the Application layer, the controller validates the incoming request using **ModelState**. Invalid requests immediately return **400 Bad Request**, preventing invalid data from reaching the business logic.

7.9 Request Processing Flow

A request handled by the API follows the sequence below.



This request flow follows Clean Architecture principles by keeping business logic independent of infrastructure while delegating data access to repository implementations.

7.10 Authentication Integration

Protected endpoints use the `[Authorize]` attribute.

When a request reaches a protected endpoint, [ASP.NET](#) Core automatically validates the JWT token before the controller action executes.

For example, the logout endpoint retrieves the authenticated user's identifier from the JWT claims and passes it to the authentication service, avoiding the need for clients to send the user ID manually.

This implementation reduces the risk of unauthorized operations and follows standard JWT authentication practices.

7.11 API Design Principles

The analyzed implementation follows several REST API design principles.

- Controllers are organized by business domain.
- HTTP verbs represent resource operations.
- Request bodies are transferred using DTOs.
- Authentication is handled through JWT.
- Dependency Injection separates controllers from the Application layer.
- Database access is abstracted through repository interfaces and implemented within the Infrastructure layer.
- Controllers remain lightweight by delegating business logic to the Application layer.

These design decisions align with the principles of Clean Architecture and contribute to a maintainable API implementation.

7.12 Architectural Strengths

The implementation demonstrates several strengths.

- Clear separation between Presentation, Application, Domain, and Infrastructure layers.

- Constructor-based Dependency Injection.
- Entity Framework Core abstraction through repository interfaces.
- Built-in Swagger documentation with JWT support.
- RESTful endpoint organization.
- Lightweight controllers with delegated business logic.
- Consistent use of DTOs for request and response models.
- Compliance with Clean Architecture principles through dependency inversion and separation of concerns.

Chapter 8 – Authentication Module Implementation Analysis

8.1 Introduction

The Authentication Module is one of the core components of the NFS platform. It is responsible for user identity verification, account creation, secure login, logout, token renewal, and password recovery.

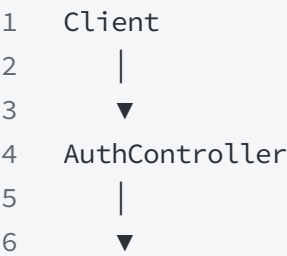
Based on the analysis of the implementation, the module follows the principles of **Clean Architecture**, where the controller is responsible only for handling HTTP requests, while all authentication logic is delegated to the **Application layer** through the **IAuthService** interface.

The implementation follows the principles of Separation of Concerns, Dependency Injection, and Dependency Inversion, resulting in a clean, maintainable, and testable authentication subsystem.

8.2 Authentication Module Architecture

The authentication workflow is implemented through several cooperating components distributed across different projects.

Code block




```
7   IAuthService
8   |
9   ▼
10  Application Layer
11  |
12  ▼
13  Repository Interface
14  |
15  ▼
16  Infrastructure Repository
17  |
18  ▼
19  ApplicationDbContext
20  |
21  ▼
22  SQL Server
```

The controller never interacts directly with the database. Instead, authentication requests are delegated to the Application layer, while data access is handled by repository implementations in the Infrastructure layer.

8.3 Authentication Components

The implementation of the authentication module consists of the following components.

Component	Responsibility
AuthController	Receives authentication requests
IAuthService	Defines authentication operations
AuthService	Implements authentication business logic
RegisterDto	User registration request
LoginDto	User login request
RefreshTokenDto	Token renewal request
ForgotPasswordDto	Password recovery request
ResetPasswordDto	Password reset request

The use of interfaces allows the controller to remain independent of the concrete implementation of the authentication service.

8.4 AuthController Analysis

The `AuthController` is implemented as an [ASP.NET](#) Core API controller.

It is decorated with the following attributes:

- `[ApiController]`
- `[Route("api/[controller]")]`

These attributes enable automatic model validation and define the base route as:

Code block

```
1 /api/auth
```

The controller depends on a single injected service:

```
1 private readonly IAuthService _authService;
```

The dependency is supplied through constructor injection:

Code block

```
1 public AuthController(IAuthService authService)
2 {
3     _authService = authService;
4 }
```

This implementation follows the Dependency Inversion Principle by depending on an abstraction (IAuthService) rather than a concrete class.

8.5 Authentication Endpoints

Analysis of the implementation shows that the controller exposes six authentication endpoints.

Endpoint	HTTP Method	Description
/api/auth/register	POST	Register a new user
/api/auth/login	POST	Authenticate an existing user
/api/auth/logout	POST	End the current user session
/api/auth/refresh-token	POST	Generate a new access token
/api/auth/forgot-password	POST	Generate a password reset token
/api/auth/reset-password	POST	Reset the user's password

Each endpoint delegates processing to the authentication service.

8.6 Register Endpoint Analysis

The Register endpoint accepts a **RegisterDto** object.

Implementation Flow

Code block

```
1  Client
2  |
3  POST /register
4  |
5  ▼
6  Model Validation
7  |
8  ▼
9  Application Layer
10 |
11 ▼
12 RegisterAsync(dto)
13 |
14 ▼
15 Return Success or Failure
```

Before invoking the authentication service, the controller verifies that the incoming model is valid.

Code block

```
1  if (!ModelState.IsValid)
2      return BadRequest(ModelState);
```

If validation succeeds, the controller calls:

Code block

```
1  _authService.RegisterAsync(dto);
```

If registration fails because the email already exists, the controller returns:

400 Bad Request

with the message:

Email is already taken.

Otherwise, the API responds with:

200 OK

and the message:

User registered successfully.

This implementation keeps all business rules inside the **Application layer**.

8.7 Login Endpoint Analysis

The Login endpoint authenticates existing users.

The request is received as a `LoginDto`.

Workflow:

Code block

```
1  Login Request
2      |
3      ▼
4  Model Validation
5      |
6      ▼
7  LoginAsync(dto)
8      |
9      ▼
10 Validate Credentials
11     |
12     ▼
13 Generate Authentication Result
14     |
15     ▼
16 Return Login Response
```

If authentication fails, the controller returns:

Code block

```
1  401 Unauthorized
```

with the message:

Code block

```
1  Invalid email or password.
```

When authentication succeeds, the controller returns the authentication object generated by the service.

8.8 Logout Endpoint Analysis

The Logout endpoint is protected using the `[Authorize]` attribute.

This means only authenticated users can access it.

Unlike other endpoints, the implementation retrieves the current user's identifier directly from the JWT claims.

Code block

```
1 User.FindFirst(ClaimTypes.NameIdentifier)
```

The claim is converted into an integer user identifier before calling:

Code block

```
1 _authService.LogoutAsync(userId);
```

This implementation prevents clients from specifying arbitrary user IDs and ensures that users can only terminate their own authenticated sessions.

If no valid claim is found, the controller immediately returns:

Code block

```
1 401 Unauthorized
```

8.9 Refresh Token Endpoint Analysis

The Refresh Token endpoint accepts a `RefreshTokenDto`.

Its responsibility is to generate a new access token without requiring the user to log in again.

Implementation flow:

Code block

```
1 RefreshTokenDto
2   |
3   ▼
4 Model Validation
5   |
6   ▼
7 RefreshTokenAsync(dto)
8   |
9   ▼
10 Generate New Access Token
11  |
12  ▼
13 Return Authentication Response
```

If either the access token or refresh token is invalid, the controller responds with:

Code block

```
1 400 Bad Request
```

Otherwise, the updated authentication information is returned.

8.10 Forgot Password Analysis

The Forgot Password endpoint accepts a `ForgotPasswordDto` .

Workflow:

Code block

```
1 ForgotPasswordDto
2   |
3   ▼
4 ForgotPasswordAsync()
5   |
6   ▼
7 Generate Reset Token
8   |
9   ▼
10 Return Reset Token
```

If the supplied email does not belong to an existing user, the controller returns:

Otherwise, the generated reset token is returned together with a success message.

This implementation indicates that password reset is handled through temporary reset tokens rather than storing temporary passwords.

8.11 Reset Password Analysis

The Reset Password endpoint accepts a `ResetPasswordDto`.

Processing flow:

Code block

```
1  ResetPasswordDto
2      |
3      ▼
4  Validate Token
5      |
6      ▼
7  ResetPasswordAsync()
8      |
9      ▼
10 Update Password
11     |
12     ▼
13 Return Success
```

If the supplied reset token is invalid or expired, the controller responds with:

Code block

```
1  400 Bad Request
```

Otherwise, the password is updated successfully.

8.12 Model Validation Strategy

A common implementation pattern appears throughout the controller.

Every endpoint begins with:

Code block

```
1  if (!ModelState.IsValid)
2      return BadRequest(ModelState);
```

This ensures invalid requests are rejected before reaching the **Application layer**.

Advantages include:

- Reduced unnecessary processing.
 - Cleaner application services.
 - Consistent validation behavior.
 - Improved API reliability.
-

8.13 Response Strategy

The controller consistently returns meaningful HTTP status codes.

Status Code	Scenario
200 OK	Operation completed successfully
400 Bad Request	Invalid request or business validation failure
401 Unauthorized	Authentication failed or missing JWT
404 Not Found	Requested user does not exist

This implementation follows RESTful API conventions and improves interoperability with frontend applications.

8.14 Architectural Assessment

Analysis of the implementation demonstrates several good architectural practices.

- Controllers remain lightweight.
- Business logic is delegated to the Application layer through **IAuthService**.

- Constructor Dependency Injection is used consistently.
- JWT claims are utilized instead of trusting client-provided identifiers.
- Model validation is centralized through **ModelState**.
- RESTful HTTP status codes are returned consistently.
- Authentication endpoints are grouped within a dedicated controller.
- Data access is abstracted through repository interfaces and implemented within the Infrastructure layer.

These characteristics align with Clean Architecture principles and improve readability, maintainability, scalability, and testability.

Chapter 9 – Domain Model Implementation Analysis

9.1 Introduction

The **NFS.Domain** project represents the core business layer of the NFS platform and forms the center of the Clean Architecture. Unlike the API or Infrastructure projects, this layer contains the business entities that model the real-world concepts of the system. These entities define the application's business data, relationships, and domain rules independently of the presentation, database, and infrastructure layers.

Analysis of the implementation shows that the Domain project contains the following entity classes:

- Appointment
- Assessment
- AssessmentResult
- AvailabilitySlot
- DiaryEntry
- Patient
- PatientMedicalHistory
- Payment
- Role
- Session
- SessionNote
- Specialization
- Therapist

- TherapistSpecialization
- User
- UserRoleMapping

These entities form the foundation of the entire platform and are referenced throughout the Application and Infrastructure layers while remaining independent of external frameworks and technologies.

9.2 Domain Layer Structure

The Domain layer represents the core of the NFS platform and is responsible for modeling the business concepts independently of any external frameworks or technologies. Following the principles of Clean Architecture, this layer contains only business entities, domain models, and related business rules. It does not contain controller logic, database access code, API-specific functionality, or infrastructure implementations.

The **NFS.Domain** project is organized as follows:

Code block

```
1  NFS.Domain
2  |
3  |— Entities
4  |   |— Appointment.cs
5  |   |— Assessment.cs
6  |   |— AssessmentResult.cs
7  |   |— AvailabilitySlot.cs
8  |   |— DiaryEntry.cs
9  |   |— Patient.cs
10 |   |— PatientMedicalHistory.cs
11 |   |— Payment.cs
12 |   |— Role.cs
13 |   |— Session.cs
14 |   |— SessionNote.cs
15 |   |— Specialization.cs
16 |   |— Therapist.cs
17 |   |— TherapistSpecialization.cs
18 |   |— User.cs
19 |   |— UserRoleMapping.cs
```

Each entity represents a specific business concept within the mental healthcare platform and serves as the foundation for the application's business logic. These entities are consumed by the Application layer while remaining completely independent of the Infrastructure and Presentation layers.

This organization reflects the principles of Clean Architecture by isolating the business entities from application logic, infrastructure concerns, and presentation components, ensuring that the core business model remains stable, reusable, and independent of implementation details.

9.3 User Entity Analysis

The **User** entity represents the primary identity of every person who interacts with the system. Authentication and authorization are built around this entity.

It serves as the parent business object from which different application roles such as therapists and patients are associated through mapping entities and related profiles.

The User entity is responsible for storing identity information that is shared across different modules of the application, reducing duplication and maintaining consistency.

9.4 Therapist Entity Analysis

The **Therapist** entity represents healthcare professionals who provide therapy services through the platform.

Implementation analysis shows that therapists are linked to several business entities, including:

- AvailabilitySlot
- Appointment
- Session
- Assessment
- TherapistSpecialization

This design enables one therapist to manage schedules, conduct therapy sessions, perform assessments, and maintain professional specializations.

Instead of embedding specialization information directly inside the Therapist entity, the implementation introduces a dedicated junction entity (TherapistSpecialization), improving normalization and allowing multiple specializations for the same therapist.

9.5 Patient Entity Analysis

The **Patient** entity models individuals receiving therapy services.

Based on the implementation, the Patient entity participates in relationships with:

- Appointment

- Assessment
- AssessmentResult
- PatientMedicalHistory
- DiaryEntry
- Session

This indicates that the patient acts as the central business object for most healthcare-related operations.

Separating patient-specific information from the generic User entity allows the platform to maintain role-specific data without duplicating identity information.

9.6 Appointment Entity Analysis

The Appointment entity is one of the most connected entities within the project.

Implementation analysis shows that the class contains relationships with:

- Patient
- Therapist (Doctor)
- AvailabilitySlot

The entity uses Entity Framework Core attributes such as:

- [Key]
- [Required]
- [ForeignKey]

to define primary keys and foreign key relationships.

The Appointment entity acts as the scheduling bridge between patients and therapists while ensuring that appointments are associated with valid availability slots.

This design improves data integrity and prevents orphan scheduling records.

9.7 Assessment Entity Analysis

The Assessment entity represents psychological assessments completed by patients.

Inspection of the implementation reveals that the entity stores:

- Assessment title

- Description
- Patient reference
- Therapist reference
- Serialized answers
- Score
- Completion timestamp

Instead of creating a table for every assessment answer, the implementation stores responses using a JSON representation (AnswersJson). This approach provides flexibility for different assessment formats without requiring schema changes whenever a questionnaire is modified.

9.8 AssessmentResult Entity Analysis

The AssessmentResult entity stores the outcome of completed assessments.

Each result is associated with:

- One Assessment
- One Patient

The entity includes:

- Assessment reference
- Patient reference
- AnswersJson
- ResultSummary
- Completion time

Separating assessment definitions from assessment results allows multiple assessment attempts while preserving historical data.

9.9 AvailabilitySlot Entity Analysis

AvailabilitySlot represents the working schedule of therapists.

Implementation analysis shows that each slot contains:

- Therapist reference
- Start time
- End time

These slots are later consumed by the Appointment module when patients book therapy sessions.

By separating availability from appointments, therapists can publish schedules before bookings occur.

9.10 Session Entity Analysis

The Session entity represents an actual therapy meeting between a therapist and a patient.

Unlike appointments, which represent planned meetings, sessions represent therapy interactions after scheduling has taken place.

The Session entity works closely with:

- SessionNote
- Appointment
- Therapist
- Patient

This separation between Appointment and Session reflects a clear business distinction between scheduling and service delivery.

9.11 SessionNote Entity Analysis

SessionNote stores clinical notes recorded during therapy sessions.

Rather than embedding notes directly inside the Session entity, the implementation isolates them into a dedicated entity.

This improves modularity and allows future expansion, such as supporting multiple notes or attachments for a single therapy session.

9.12 DiaryEntry Entity Analysis

The DiaryEntry entity provides patients with a personal journaling feature.

Each diary record belongs to a patient and enables the platform to maintain a chronological history of thoughts, emotions, or daily experiences.

This functionality supports continuous mental health monitoring while remaining independent from formal therapy sessions.

9.13 PatientMedicalHistory Entity Analysis

The PatientMedicalHistory entity stores long-term medical information related to patients.

Separating medical history into its own entity prevents excessive growth of the Patient entity and keeps medical records organized independently from appointment or assessment data.

9.14 Payment Entity Analysis

The Payment entity manages financial transactions within the platform.

Payments are associated with therapy-related operations and provide a dedicated business object for handling financial records separately from appointments and sessions.

This separation supports future expansion to multiple payment providers without modifying scheduling entities.

9.15 Specialization and TherapistSpecialization

The implementation contains two related entities:

- Specialization
- TherapistSpecialization

Rather than storing specialization directly inside Therapist, the project implements a many-to-many relationship through TherapistSpecialization.

Benefits of this implementation include:

- A therapist can have multiple specializations.
 - One specialization can belong to multiple therapists.
 - Database normalization.
 - Simplified future expansion.
-

9.16 Role and UserRoleMapping

Authorization within the platform is represented through dedicated entities.

The implementation includes:

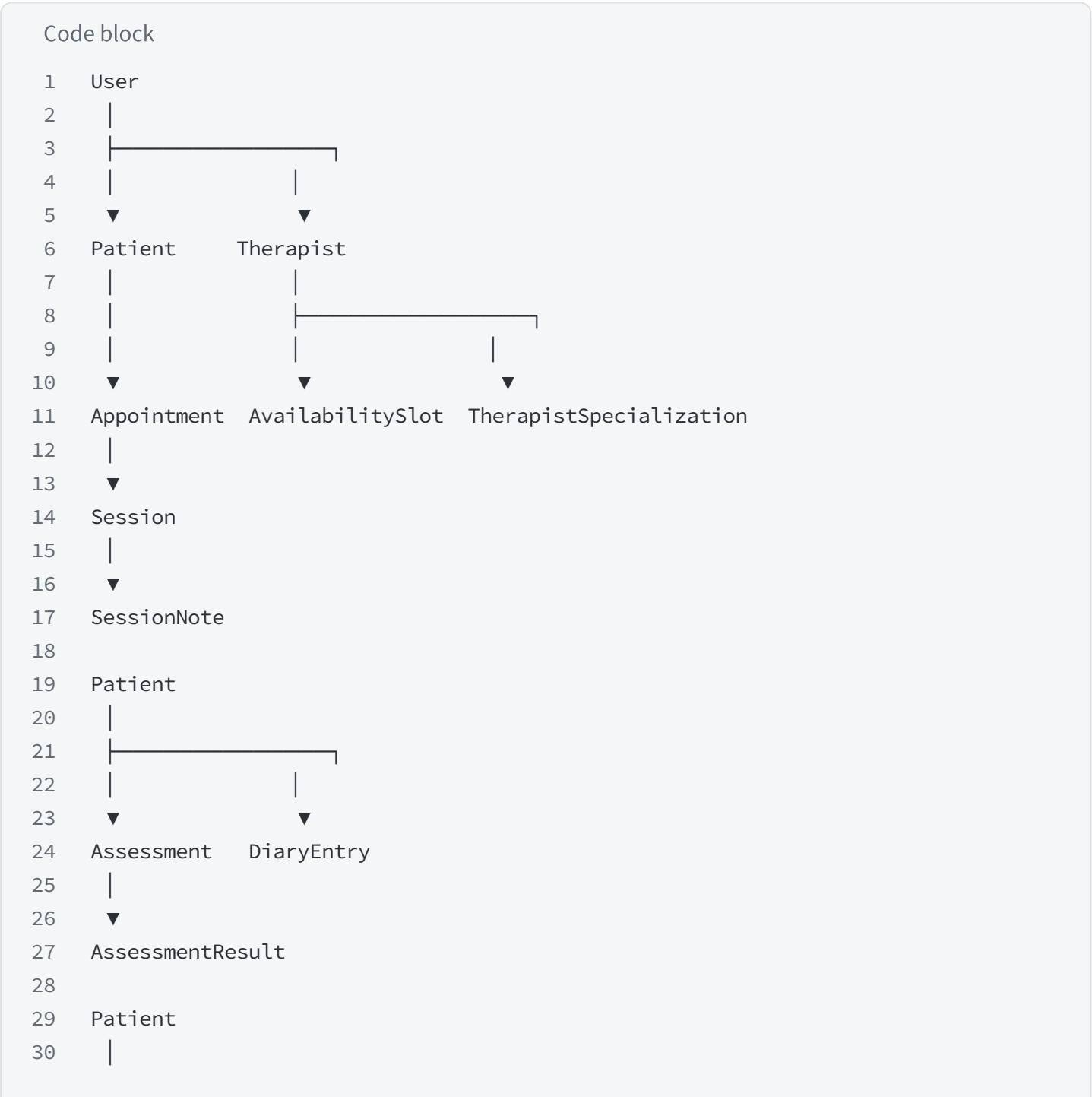
- Role
- UserRoleMapping

Instead of hardcoding roles directly inside the User entity, the project maintains role assignments through a mapping table.

This design increases flexibility by allowing users to be associated with different roles without modifying the user structure.

9.17 Entity Relationships

The analyzed implementation reveals the following core relationships.



This model demonstrates that the Patient and Therapist entities are the primary business objects around which the rest of the healthcare workflow is organized.

9.18 Design Evaluation

The domain model exhibits several strong design characteristics.

- Clear separation of business entities.
- Independence from [ASP.NET](#) Core, Entity Framework Core, and other external frameworks.
- Well-defined foreign key relationships.
- Normalized database structure.
- Dedicated entities for specialized business concepts.
- Flexible many-to-many relationship for therapist specializations.
- Separation between appointments, sessions, and clinical notes.
- Alignment with Clean Architecture principles by keeping business rules at the core of the system.

These design choices improve maintainability, scalability, testability, and long-term extensibility of the application.

Chapter 10 – Therapist Management Module Implementation Analysis

10.1 Introduction

The Therapist Management Module is one of the primary business modules of the NFS platform. It is responsible for managing therapist information, professional profiles, specializations, availability schedules, and all therapist-related operations exposed through the REST API.

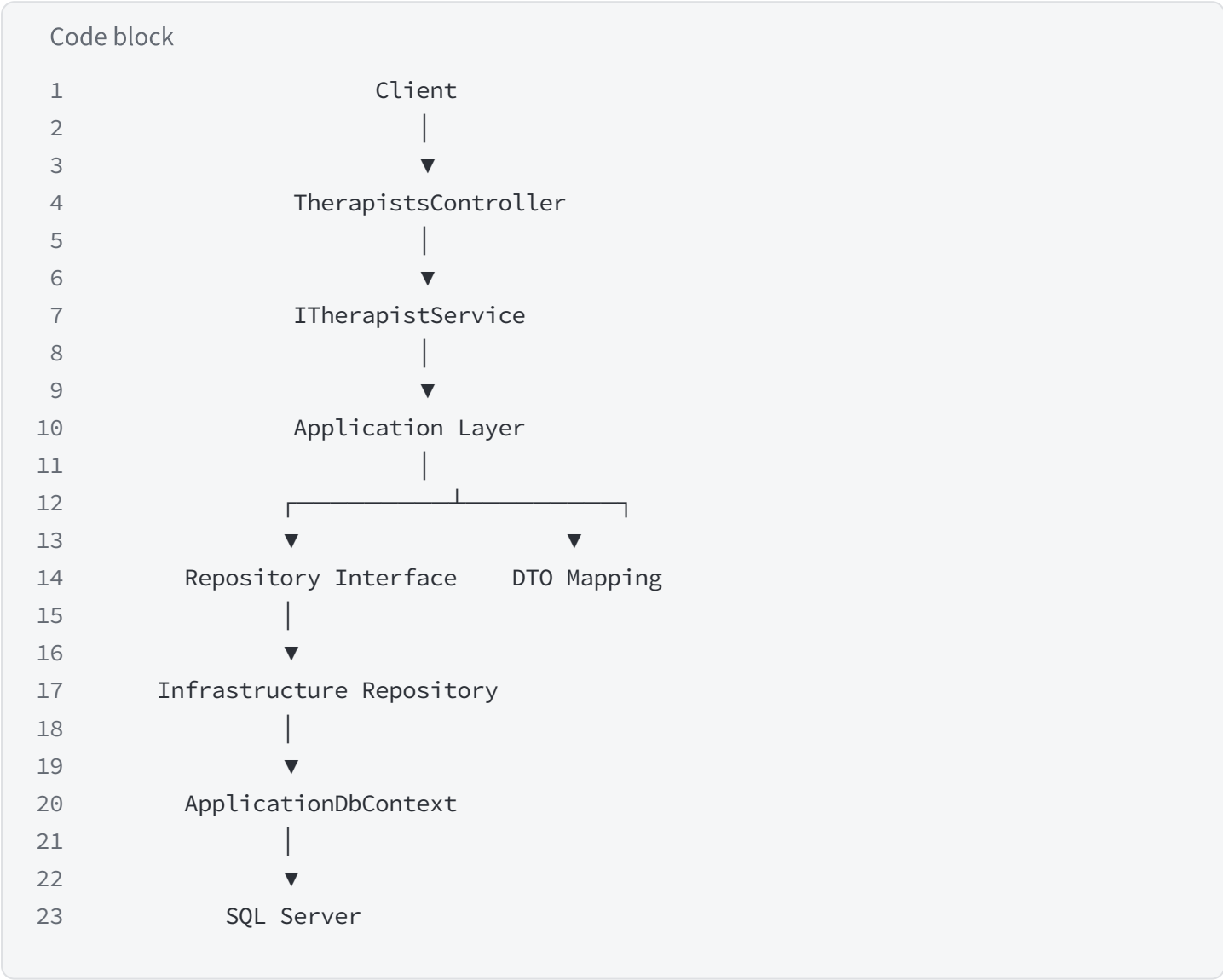
Based on the implementation analysis, this module is centered around the **TherapistsController** in the Presentation layer and collaborates with the **Application**, **Domain**, and **Infrastructure** layers to provide a complete therapist management workflow.

Unlike the Authentication Module, which focuses on user identity, the Therapist Management Module focuses on managing therapists' professional information and supporting their

interaction with patients.

10.2 Module Architecture

The Therapist Management Module follows the principles of **Clean Architecture**, where each layer has a specific responsibility and dependencies point toward the Domain layer.



The controller is responsible only for processing HTTP requests, while business operations are executed within the Application layer. Data access is handled through repository implementations located in the Infrastructure layer.

10.3 Module Responsibilities

The Therapist Management Module is responsible for the following business operations:

- Creating therapist profiles.
- Retrieving therapist information.

- Updating therapist data.
- Managing therapist availability.
- Managing therapist specializations.
- Searching therapists.
- Displaying therapist profiles to patients.
- Supporting appointment scheduling.

These responsibilities make the Therapist Management Module one of the central components of the healthcare platform.

10.4 TherapistsController Analysis

Implementation analysis shows that the **TherapistsController** acts as the API entry point for therapist-related operations.

The controller follows [ASP.NET](#) Core REST API conventions by using:

- `[ApiController]`
- `[Route("api/[controller]")]`

The controller communicates exclusively with the Application layer and does not contain business logic.

Its responsibilities include:

- Receiving HTTP requests.
- Validating request models.
- Calling application services.
- Returning HTTP responses.

This implementation follows the Thin Controller design pattern and aligns with the principles of Clean Architecture by keeping business logic outside the Presentation layer.

10.5 Therapist Entity Analysis

The Therapist entity represents licensed mental health professionals registered on the platform.

The implementation associates each therapist with multiple business entities, including:

- User
- Appointment

- AvailabilitySlot
- Session
- Assessment
- TherapistSpecialization

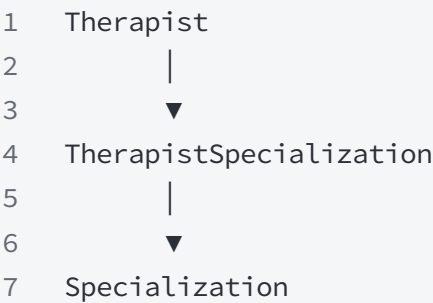
Rather than storing all therapist information in the User entity, the project separates therapist-specific information into its own entity.

This separation improves maintainability, reduces data duplication, and keeps identity information independent from therapist-specific business data.

10.6 Therapist Specialization Management

The implementation introduces a dedicated entity named **TherapistSpecialization**. Instead of assigning a single specialization directly to the therapist, the application models the relationship using a junction entity.

Code block



This implementation supports:

- Multiple specializations per therapist.
- Shared specializations across therapists.
- Database normalization.
- Future scalability.

This design follows best practices for implementing many-to-many relationships in relational databases.

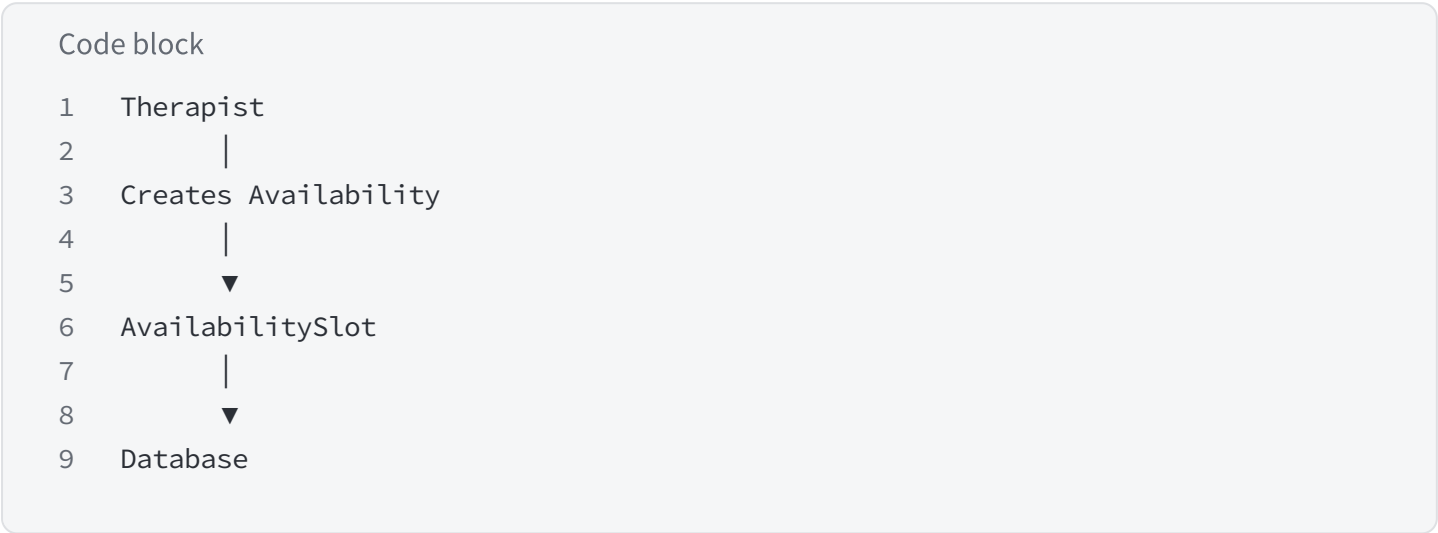
10.7 Therapist Availability Management

Therapist working schedules are managed through the **AvailabilitySlot** entity.

Each availability slot contains:

- Therapist reference.
- Start time.
- End time.

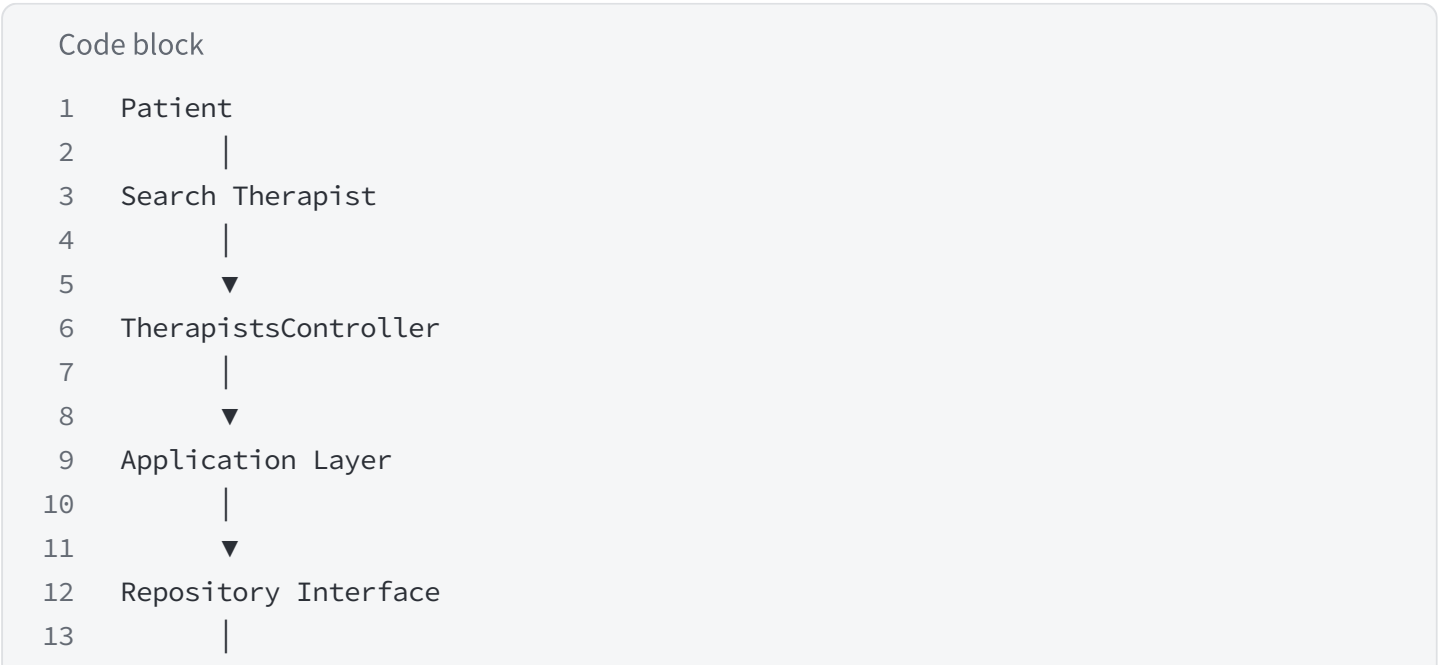
Implementation workflow:



Appointments are later created only from available time slots, preventing double booking and improving scheduling consistency.

10.8 Therapist Search Workflow

Patients interact with therapists primarily through the search functionality. The implementation supports retrieving therapist information before booking appointments. General request flow:



```
14      ▼
15  Infrastructure Repository
16      |
17      ▼
18  Database
19      |
20      ▼
21  Therapist DTO
22      |
23      ▼
24  Patient
```

Separating domain entities from response DTOs reduces unnecessary data exposure and improves API security.

10.9 Therapist Profile Management

Therapists maintain professional profiles containing information used throughout the platform.

The profile may include:

- Personal information.
- Professional information.
- Specializations.
- Availability.
- Experience.
- Biography.
- Contact information.

Profile updates are processed through the Application layer, ensuring that validation and business rules remain centralized.

10.10 Therapist Appointment Relationship

The therapist participates in the appointment lifecycle as one of the primary entities.

Code block

```
1  Patient
2  |
3  Books Appointment
```

```
4      |
5      ▼
6  Appointment
7      |
8      ▼
9  Therapist
```

This relationship allows therapists to manage scheduled consultations while enabling patients to book available sessions.

10.11 Therapist Session Relationship

After an appointment is confirmed, therapy sessions are conducted.

Code block

```
1  Appointment
2      |
3      ▼
4  Session
5      |
6      ▼
7  Therapist
8      |
9      ▼
10 SessionNote
```

Separating appointments from sessions distinguishes scheduling from actual healthcare delivery.

10.12 Therapist Assessment Relationship

Therapists participate in patient assessments through the Assessment module.

Code block

```
1  Therapist
2      |
3  Creates Assessment
4      |
5      ▼
6  Assessment
```



```
7 |
8 ▼
9 AssessmentResult
```

This workflow allows therapists to evaluate patients while maintaining historical assessment records.

10.13 Data Validation Strategy

The implementation performs validation before therapist information reaches the Application layer.

Validation includes:

- Required fields.
- Model validation.
- Existing therapist verification.
- Valid specialization references.
- Availability consistency.

Invalid requests are rejected before reaching the Application layer, reducing unnecessary processing and ensuring that business rules are executed only on valid data.

10.14 Architectural Evaluation

The Therapist Management Module demonstrates several architectural strengths.

- Business logic is isolated from controllers.
- Communication between layers follows Clean Architecture principles.
- Therapist data is separated from user identity.
- Availability scheduling is modeled independently.
- Many-to-many specialization management improves flexibility.
- Appointment scheduling is integrated with therapist availability.
- DTOs isolate API contracts from domain entities.
- Repository interfaces abstract database operations.
- Entity relationships are normalized.

These characteristics contribute to a scalable, maintainable, and testable healthcare platform while remaining consistent with the principles of Clean Architecture.

Chapter 11 – Patient Management Module Implementation Analysis

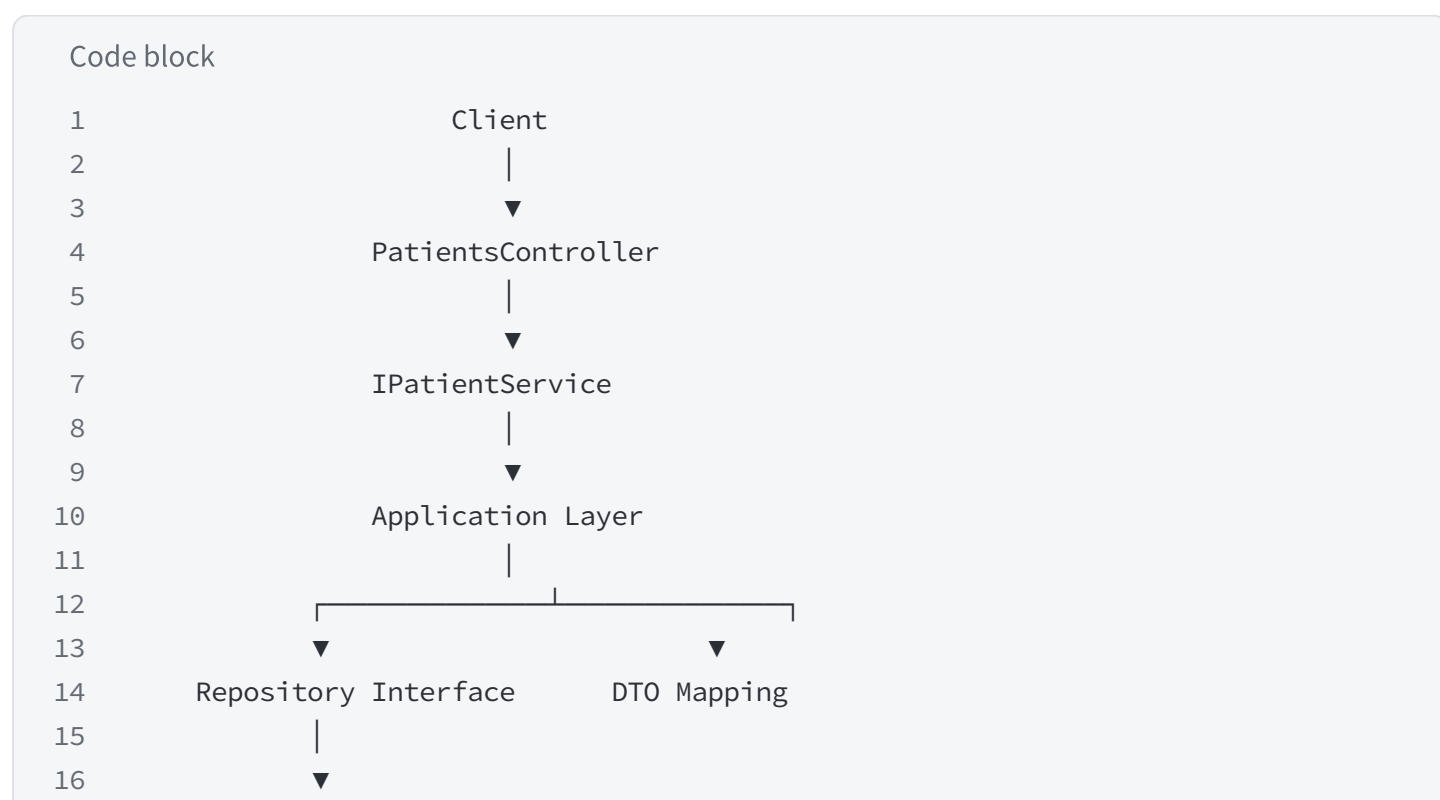
11.1 Introduction

The Patient Management Module is one of the fundamental components of the NFS platform. It is responsible for managing patient profiles, medical information, assessment history, therapy sessions, appointments, and personal diary entries. Based on the implementation analysis, this module provides the central functionality required to support patients throughout their therapeutic journey.

The module follows the principles of **Clean Architecture**, ensuring a clear separation between the Presentation, Application, Domain, and Infrastructure layers. The Presentation layer receives requests through the **PatientsController**, while all business operations are delegated to the Application layer, and data persistence is handled by the Infrastructure layer.

11.2 Module Architecture

The Patient Management Module follows the principles of **Clean Architecture**, separating request handling, business logic, and data persistence into independent layers.



```
17      Infrastructure Repository
18      |
19      ▼
20      ApplicationDbContext
21      |
22      ▼
23      SQL Server
```

The controller is responsible only for handling HTTP requests. Business operations are executed in the Application layer, while repository implementations in the Infrastructure layer manage communication with the database.

This architecture improves maintainability, scalability, testability, and separation of concerns.

11.3 Module Responsibilities

Analysis of the implementation indicates that the Patient Management Module is responsible for several key business operations.

These responsibilities include:

- Creating patient profiles.
- Retrieving patient information.
- Updating patient data.
- Managing medical history.
- Managing psychological assessments.
- Viewing appointments.
- Viewing therapy sessions.
- Managing personal diary entries.
- Supporting therapist interactions.

The Patient entity serves as the central business object for many healthcare-related operations throughout the platform.

11.4 PatientsController Analysis

The **PatientsController** is the API entry point responsible for handling all patient-related requests.

The controller follows [ASP.NET](#) Core API conventions by using:

- `[ApiController]`
- `[Route("api/[controller]")]`

The implementation keeps the controller lightweight by delegating business logic to the Application layer.

Its responsibilities include:

- Receiving HTTP requests.
- Validating request models.
- Invoking application services.
- Returning standardized HTTP responses.

This implementation follows the Thin Controller pattern and aligns with the principles of Clean Architecture by separating request handling from business logic.

11.5 Patient Entity Analysis

The Patient entity represents individuals receiving therapy services through the NFS platform.

Unlike the generic **User** entity, the **Patient** entity stores healthcare-specific information required by the system.

Implementation analysis shows that the Patient entity participates in relationships with several business entities, including:

- Appointment
- Assessment
- AssessmentResult
- PatientMedicalHistory
- Session
- DiaryEntry

This design allows the system to maintain comprehensive patient records while keeping authentication and identity information separate from healthcare-specific business data.

11.6 Patient Profile Management

Each patient maintains a dedicated profile containing personal and healthcare-related information.

The profile is used throughout the application by different modules such as appointments, therapy sessions, assessments, and diary management.

Typical profile information includes:

- Personal details.
- Contact information.
- Medical information.
- Assessment history.
- Therapy history.

Profile updates are processed through the Application layer, where validation and business rules are applied before data is persisted.

11.7 Patient Medical History

The implementation contains a dedicated entity named **PatientMedicalHistory**.

Rather than storing medical information directly inside the Patient entity, medical records are separated into their own entity.

Code block

```
1 Patient
2   |
3   ▼
4 PatientMedicalHistory
```

This separation provides several advantages:

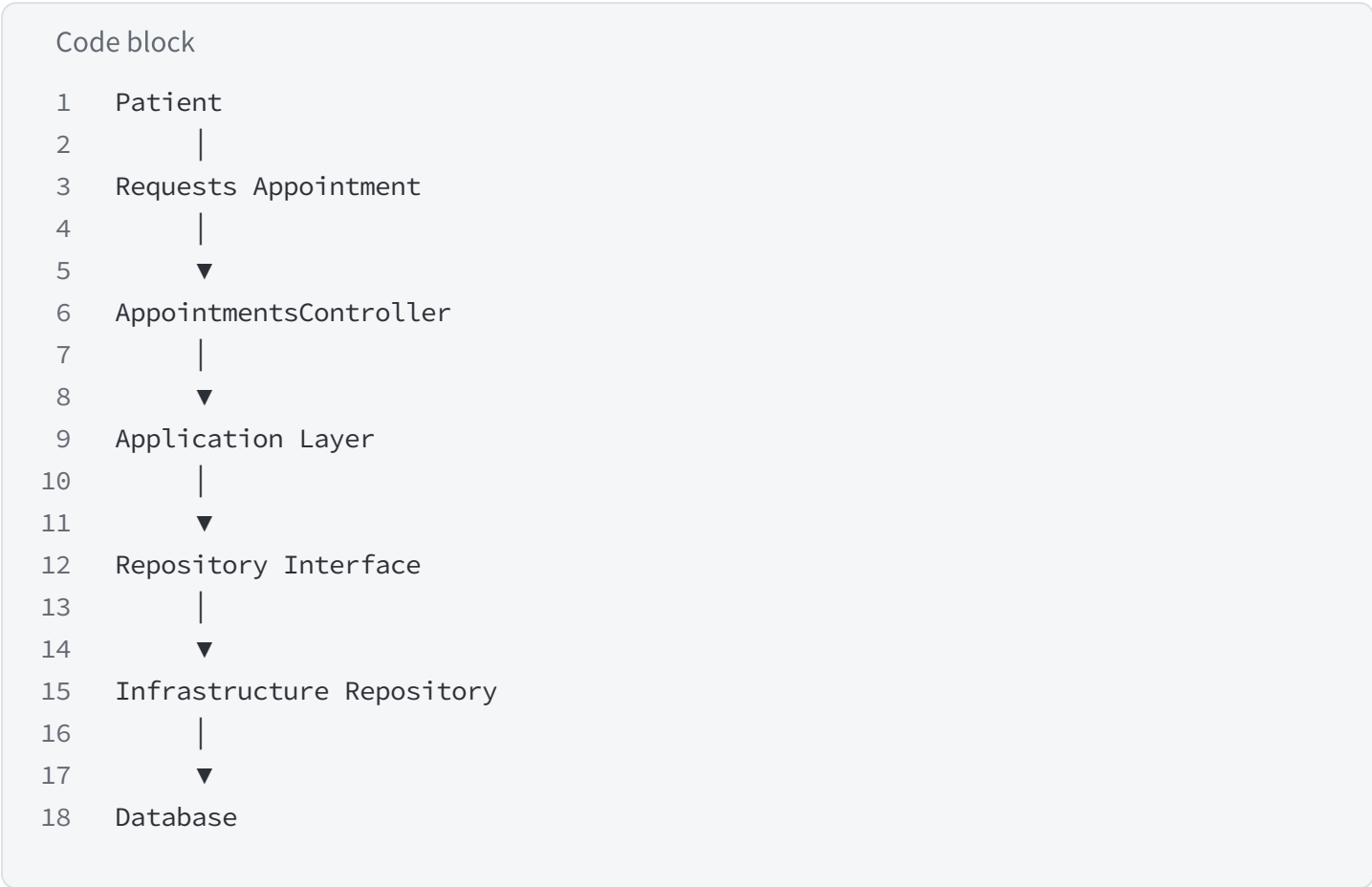
- Better database normalization.
- Easier maintenance.
- Independent updates.
- Future extensibility.

It also reduces the size and complexity of the Patient entity while keeping medical information organized independently.

11.8 Patient Appointment Workflow

Appointments represent scheduled meetings between patients and therapists.

Implementation workflow:



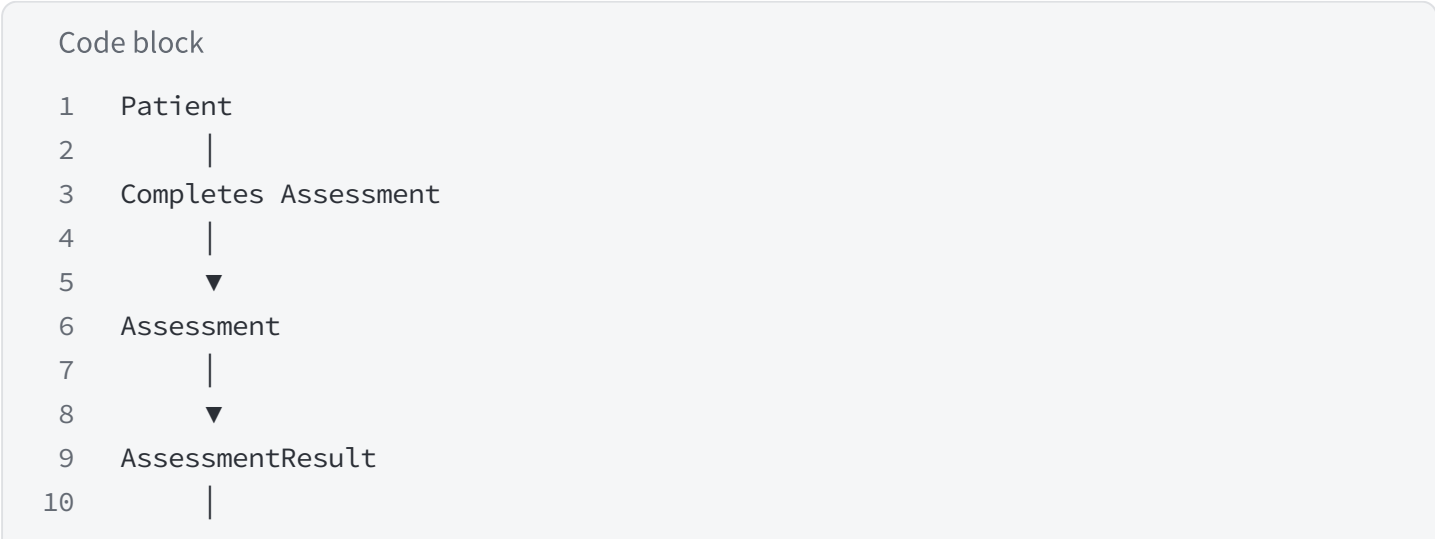
Patients interact with appointments throughout the therapy lifecycle, from scheduling through completion.

11.9 Patient Assessment Workflow

The Patient Management Module integrates closely with the Assessment Module.

Patients complete psychological assessments assigned by therapists.

Implementation flow:



- 11 ▼
- 12 Database

Assessment results become part of the patient's healthcare record and may be reviewed during future therapy sessions.

11.10 Patient Session Workflow

Therapy sessions are linked directly to patient records.



Each completed session contributes to the patient's therapy history and may include clinical notes recorded by the therapist.

Separating sessions from appointments distinguishes scheduled events from completed therapeutic interactions.

11.11 Patient Diary Management

The implementation includes a personal diary feature represented by the **DiaryEntry** entity.

Workflow:



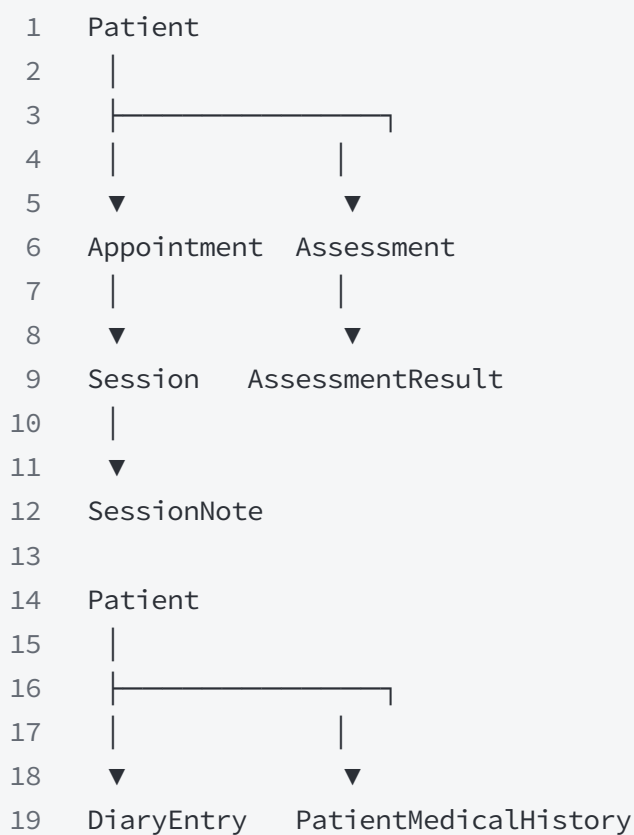
Diary entries allow patients to document emotions, thoughts, and daily experiences independently of formal therapy sessions.

This feature supports long-term emotional tracking while preserving patient privacy.

11.12 Entity Relationships

The Patient entity acts as the center of several business relationships.

Code block



This relationship model demonstrates that nearly every healthcare activity within the platform is connected to the Patient entity.

11.13 Validation Strategy

The implementation validates incoming patient data before invoking the Application layer.

Validation includes:

- Required fields.

- Model validation.
- Existing patient verification.
- Valid medical history references.
- Entity relationship validation.

Invalid requests are rejected before business processing begins, ensuring that only valid data enters the application workflow.

11.14 Architectural Evaluation

Analysis of the Patient Management Module reveals several architectural strengths.

- Business logic is isolated from controllers.
- Communication between layers follows Clean Architecture principles.
- Patient identity is separated from authentication.
- Medical history is isolated in a dedicated entity.
- Diary entries are managed independently.
- Assessment history is preserved separately from assessment definitions.
- Sessions and appointments are modeled as distinct business concepts.
- DTOs isolate API contracts from domain entities.
- Repository interfaces abstract database operations.
- Entity relationships are normalized.

These design decisions improve maintainability, scalability, testability, and support the long-term evolution of the healthcare platform while remaining consistent with the principles of Clean Architecture.

Chapter 12 – Appointment Management Module Implementation Analysis

12.1 Introduction

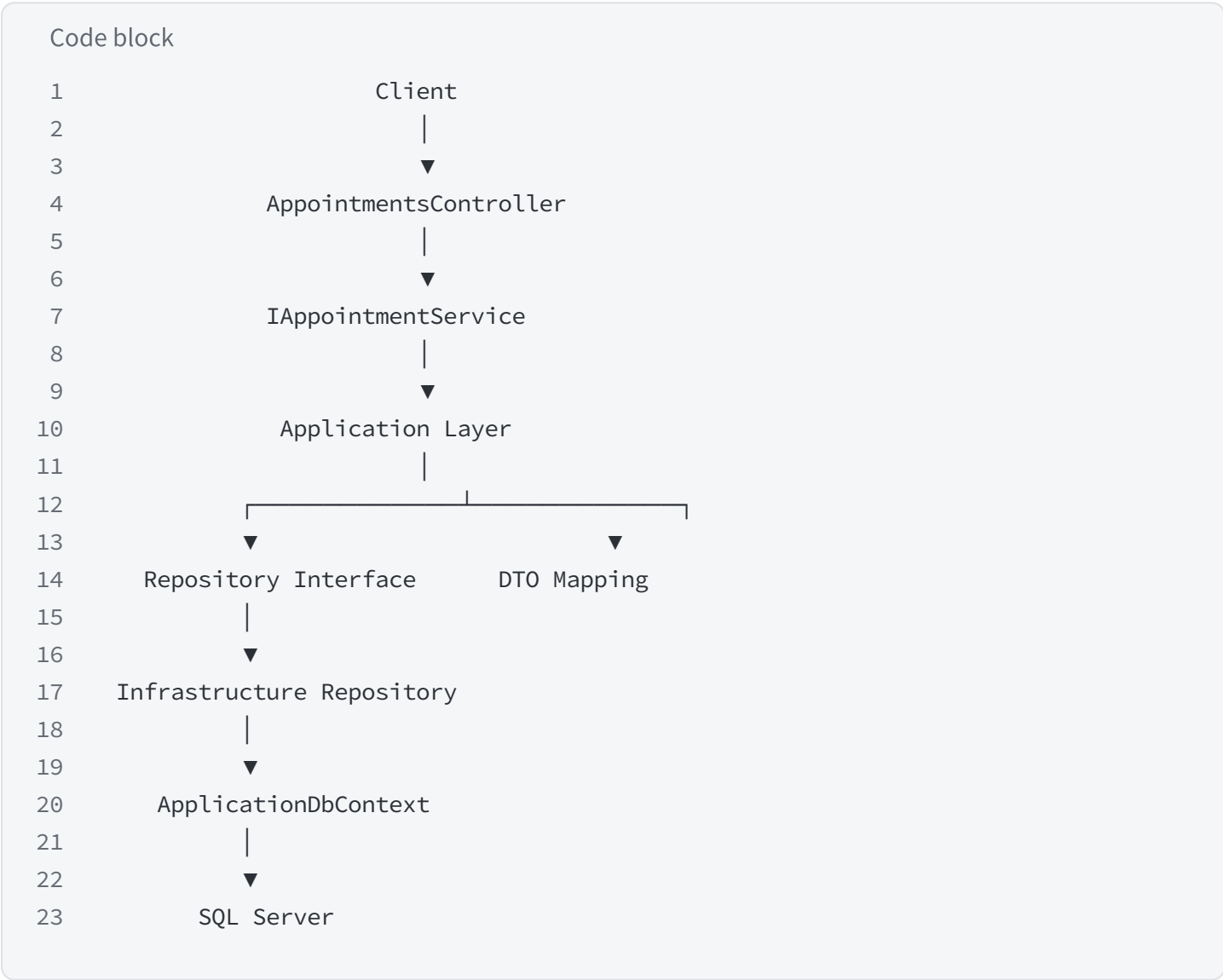
The Appointment Management Module is one of the core functional components of the NFS platform. It is responsible for scheduling, managing, updating, and monitoring therapy appointments between patients and therapists. Based on the implementation analysis, this

module acts as the bridge between therapist availability and patient bookings, ensuring that appointments are created according to predefined business rules.

The module follows the principles of **Clean Architecture**. The **AppointmentsController** receives HTTP requests in the Presentation layer, business logic is executed within the Application layer, and data persistence is handled by the Infrastructure layer through Entity Framework Core.

12.2 Module Architecture

The Appointment Management Module follows the principles of **Clean Architecture**, separating presentation, business logic, and data access into independent layers.



The controller is responsible only for handling HTTP requests, while business rules are executed in the Application layer. Repository implementations in the Infrastructure layer handle communication with the database.

This architecture improves maintainability, scalability, testability, and separation of concerns.

12.3 Module Responsibilities

The Appointment Management Module is responsible for coordinating all appointment-related operations within the platform.

Its responsibilities include:

- Creating new appointments.
- Retrieving appointment details.
- Viewing therapist schedules.
- Viewing patient appointments.
- Updating appointment status.
- Cancelling appointments.
- Validating therapist availability.
- Preventing scheduling conflicts.

The module serves as the central scheduling component of the entire healthcare platform.

12.4 AppointmentsController Analysis

The **AppointmentsController** exposes the REST API endpoints responsible for appointment management.

The controller follows [ASP.NET](#) Core API conventions by using:

- `[ApiController]`
- `[Route("api/[controller]")]`

The implementation keeps the controller lightweight by delegating all business logic to the Application layer.

Its responsibilities include:

- Receiving appointment requests.
- Validating request models.
- Calling application services.
- Returning standardized HTTP responses.

This implementation follows the Thin Controller pattern and aligns with the principles of Clean Architecture.

12.5 Appointment Entity Analysis

The Appointment entity represents a scheduled meeting between a patient and a therapist. Implementation analysis indicates that the entity acts as the central connection between several business entities.

The Appointment entity maintains relationships with:

- Patient
- Therapist
- AvailabilitySlot
- Session

These relationships ensure that every appointment belongs to a valid patient, is assigned to an existing therapist, and corresponds to an available time slot.

12.6 Appointment Creation Workflow

Creating an appointment involves validating multiple business conditions before saving data. Implementation workflow:



```
20      ▼
21  Infrastructure Repository
22      |
23      ▼
24  Database
25      |
26      ▼
27  Success Response
```

Before an appointment is created, the Application layer validates therapist availability to avoid scheduling conflicts.

12.7 Appointment Retrieval Workflow

Patients and therapists can retrieve appointment information through the API.

Workflow:

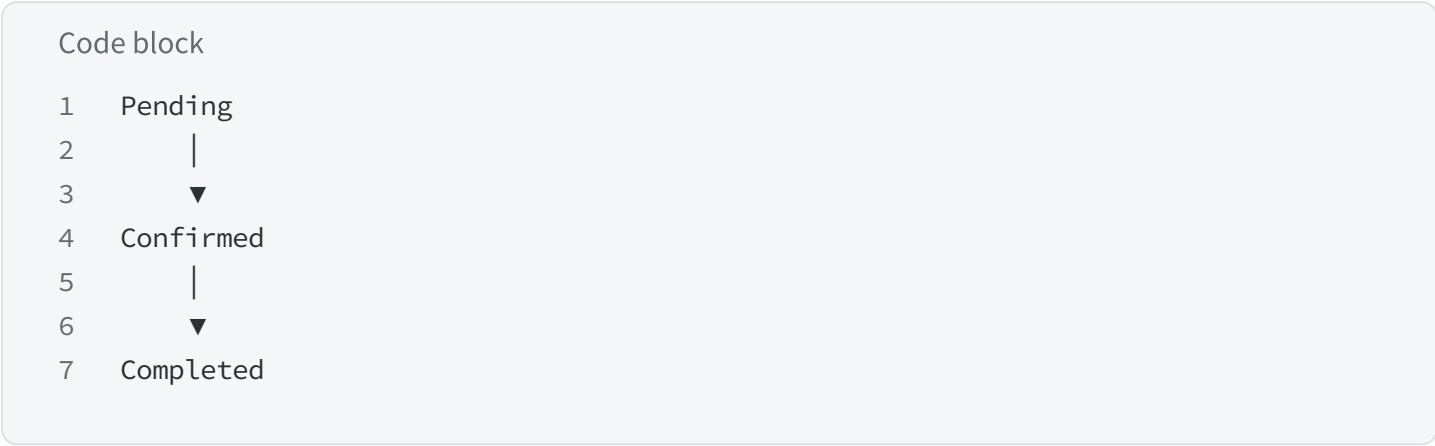
```
Code block
1  Client
2  |
3  GET Appointment
4  |
5  ▼
6  AppointmentsController
7  |
8  ▼
9  Application Layer
10 |
11 ▼
12 Repository Interface
13 |
14 ▼
15 Infrastructure Repository
16 |
17 ▼
18 Appointment DTO
19 |
20 ▼
21 Client
```

The implementation returns DTOs rather than exposing domain entities directly, improving encapsulation and API security.

12.8 Appointment Status Management

Appointments progress through different stages during their lifecycle.

Typical workflow:



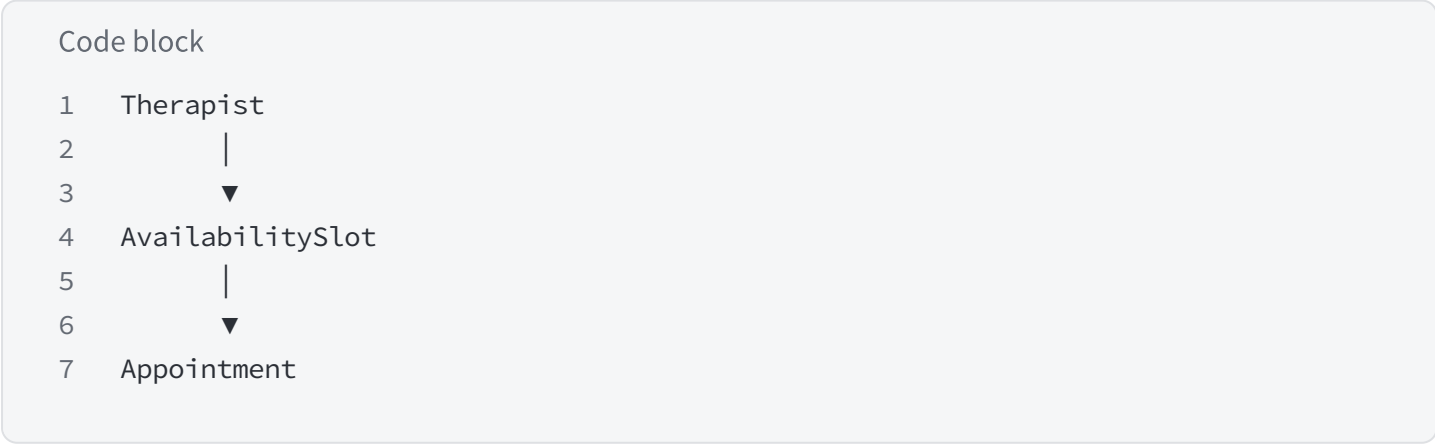
Additional transitions may include:

- Cancelled
- Rejected

Managing appointment status separately allows the application to monitor scheduling progress without modifying the appointment structure itself.

12.9 Appointment and Availability Integration

One of the key implementation features is the relationship between appointments and therapist availability.



Appointments are created only from valid availability slots, ensuring:

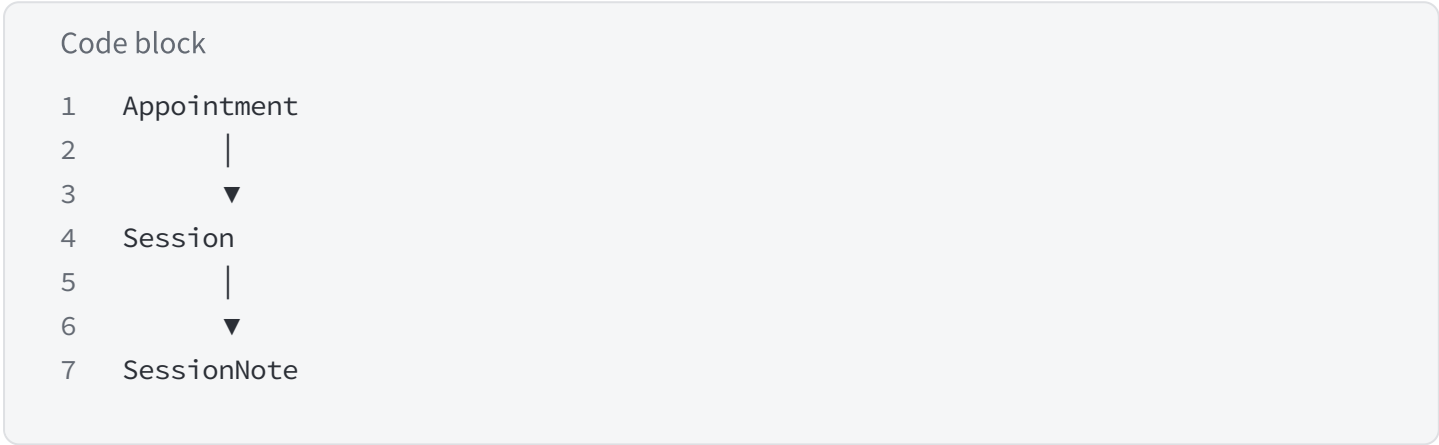
- No overlapping appointments.

- Accurate scheduling.
- Better therapist workload management.
- Improved patient booking experience.

This design prevents double booking and maintains scheduling consistency.

12.10 Appointment and Session Relationship

Appointments and therapy sessions represent different stages of the healthcare process.



An appointment represents a scheduled meeting, whereas a session represents the actual therapeutic interaction.

This separation provides greater flexibility for managing cancellations, rescheduling, and session documentation.

12.11 DTO Usage

The implementation uses dedicated DTOs for appointment operations.

Examples identified in the project include:

- AppointmentDto
- CreateAppointmentDto
- PatientAppointmentDto
- DoctorAppointmentDto
- UpdateAppointmentStatusDto
- CreateAvailabilitySlotDto

Using specialized DTOs minimizes unnecessary data transfer and clearly defines the API contract for each operation.

12.12 Validation Strategy

Before an appointment is persisted, the implementation performs several validation steps within the Application layer.

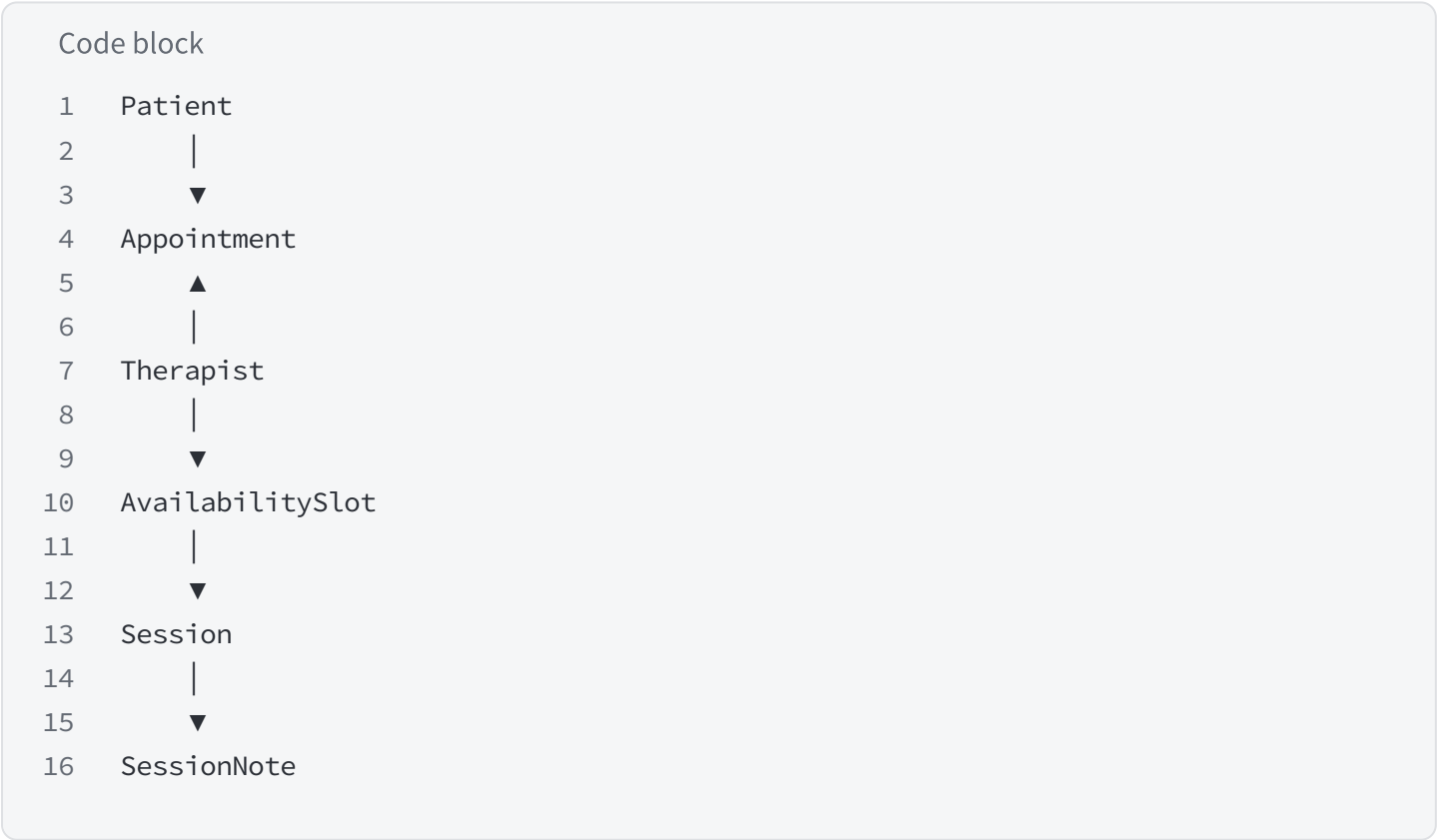
These include:

- Model validation.
- Patient existence verification.
- Therapist existence verification.
- Availability slot validation.
- Appointment conflict detection.
- Required field validation.

Invalid requests are rejected before reaching the Infrastructure layer or the database.

12.13 Entity Relationships

The Appointment entity connects several major entities within the system.



This relationship model demonstrates that appointments act as the central scheduling entity linking patients, therapists, and therapy sessions.

12.14 Architectural Evaluation

The Appointment Management Module demonstrates several architectural strengths.

- Business logic is isolated from controllers.
- Communication between layers follows Clean Architecture principles.
- Clear separation between scheduling and therapy sessions.
- Dedicated availability management.
- Normalized entity relationships.
- DTO-based communication.
- Repository interfaces abstract data access.
- Entity Framework Core is encapsulated within the Infrastructure layer.
- RESTful API implementation.

These design decisions contribute to a scalable, maintainable, and extensible appointment scheduling system while remaining consistent with the overall Clean Architecture adopted by the NFS platform.

Chapter 13 – Psychological Assessment Module Implementation Analysis

13.1 Introduction

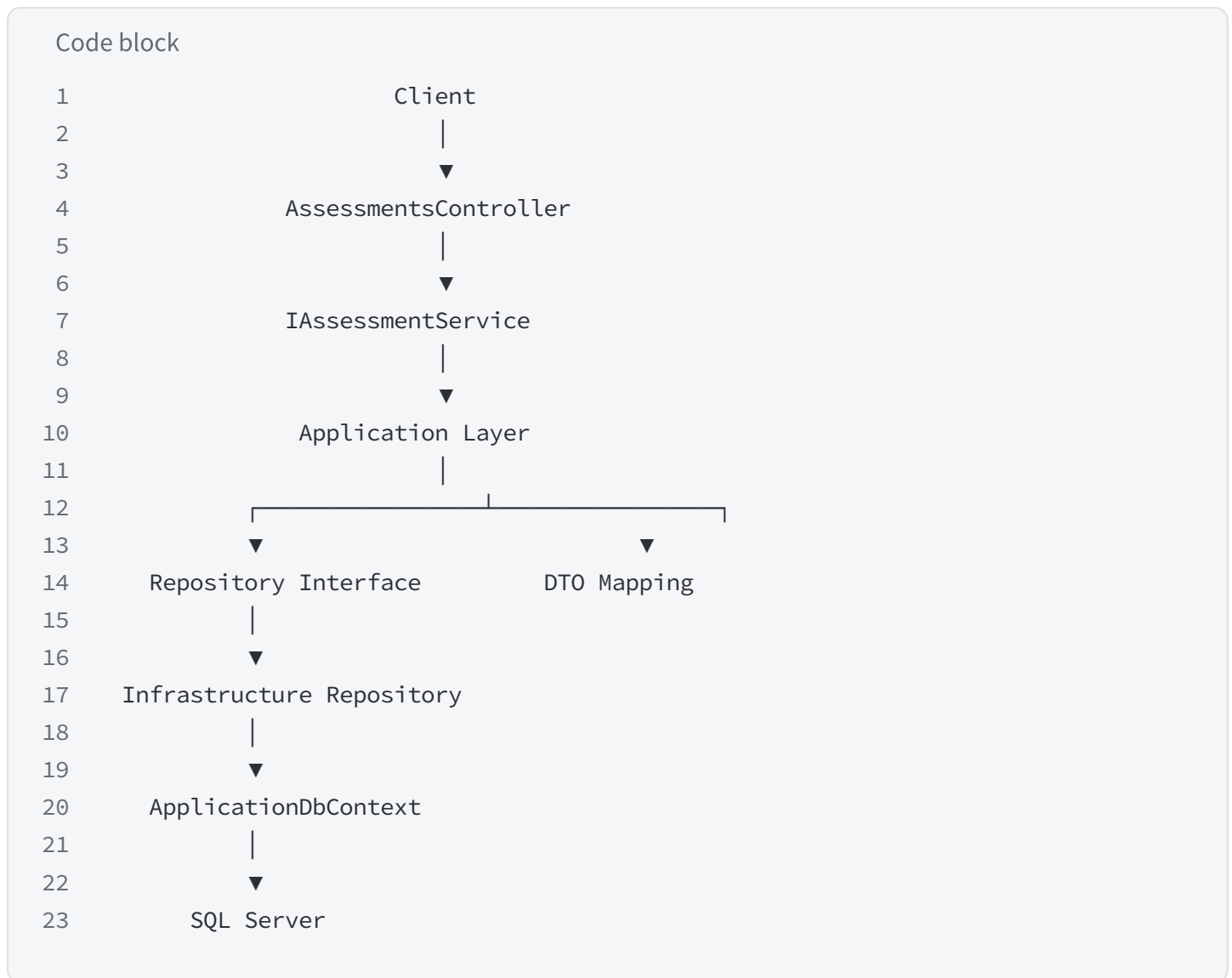
The Psychological Assessment Module is one of the most important healthcare components of the NFS platform. It is responsible for creating, managing, completing, and analyzing psychological assessments used to evaluate patients throughout their treatment journey.

Based on the implementation analysis, this module is centered around the **AssessmentsController** and integrates with several domain entities, including **Assessment**, **AssessmentResult**, **Patient**, and **Therapist**. The module enables therapists to assign assessments to patients while preserving assessment history and supporting future clinical evaluations.

The module follows the principles of **Clean Architecture**, ensuring a clear separation between the Presentation, Application, Domain, and Infrastructure layers.

13.2 Module Architecture

The Assessment Module follows the principles of **Clean Architecture**, separating request handling, business logic, and data persistence into independent layers.



The controller is responsible only for handling HTTP requests. Business operations are performed within the Application layer, while repository implementations in the Infrastructure layer communicate with the database.

This architecture improves maintainability, scalability, testability, and separation of concerns.

13.3 Module Responsibilities

Analysis of the implementation shows that the Assessment Module performs several business operations.

Its primary responsibilities include:

- Creating psychological assessments.

- Retrieving assessment information.
- Assigning assessments to patients.
- Recording assessment responses.
- Calculating assessment results.
- Storing completed assessments.
- Viewing historical assessment records.
- Supporting therapist decision-making.

The module plays an important role in monitoring patient progress during therapy.

13.4 AssessmentsController Analysis

The **AssessmentsController** acts as the presentation layer of the assessment subsystem.

The controller follows [ASP.NET](#) Core REST API conventions using:

- `[ApiController]`
- `[Route("api/[controller]")]`

Its implementation is intentionally lightweight and performs only request handling operations.

The controller responsibilities include:

- Receiving assessment requests.
- Validating incoming models.
- Delegating business logic to the Application layer.
- Returning standardized HTTP responses.

No business calculations are implemented directly inside the controller, following the Thin Controller pattern used throughout the project.

13.5 Assessment Entity Analysis

The **Assessment** entity represents the definition of a psychological evaluation.

Based on the implementation, the entity stores information describing the assessment itself rather than the patient's completed answers.

The entity maintains relationships with:

- Patient
- Therapist

- AssessmentResult

Separating the assessment definition from the assessment outcome enables the system to preserve multiple assessment attempts without modifying the original assessment.

13.6 AssessmentResult Entity Analysis

The **AssessmentResult** entity stores completed patient responses and evaluation outcomes. Implementation analysis indicates that each assessment result is associated with:

- One Assessment.
- One Patient.

The entity contains information such as:

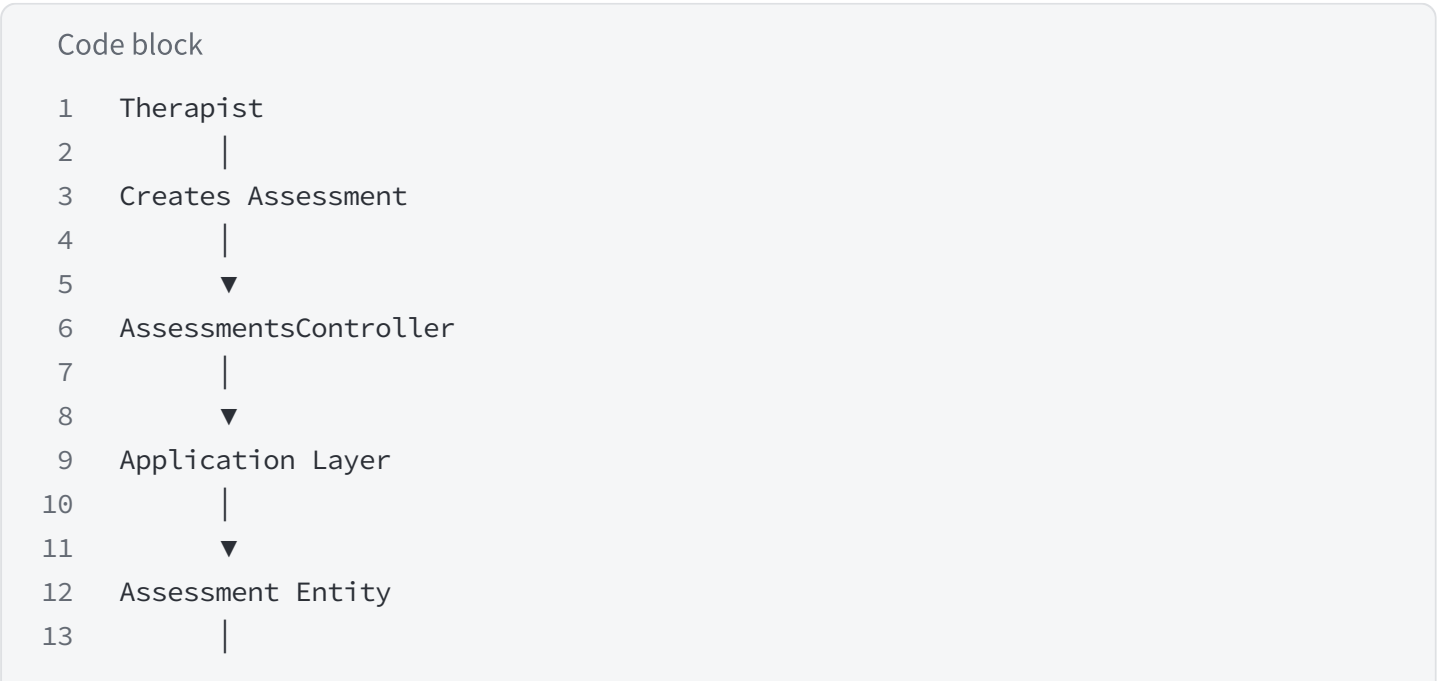
- Patient responses.
- Assessment score.
- Result summary.
- Completion timestamp.

This separation improves database normalization and allows therapists to review previous assessment results over time.

13.7 Assessment Creation Workflow

Therapists create assessments that are later completed by patients.

Implementation workflow:



```
14      ▼
15  Repository Interface
16      |
17      ▼
18  Infrastructure Repository
19      |
20      ▼
21  Database
```

The workflow ensures that assessment definitions are created before being assigned to patients.

13.8 Assessment Completion Workflow

Patients complete assessments through the frontend application.

Implementation workflow:

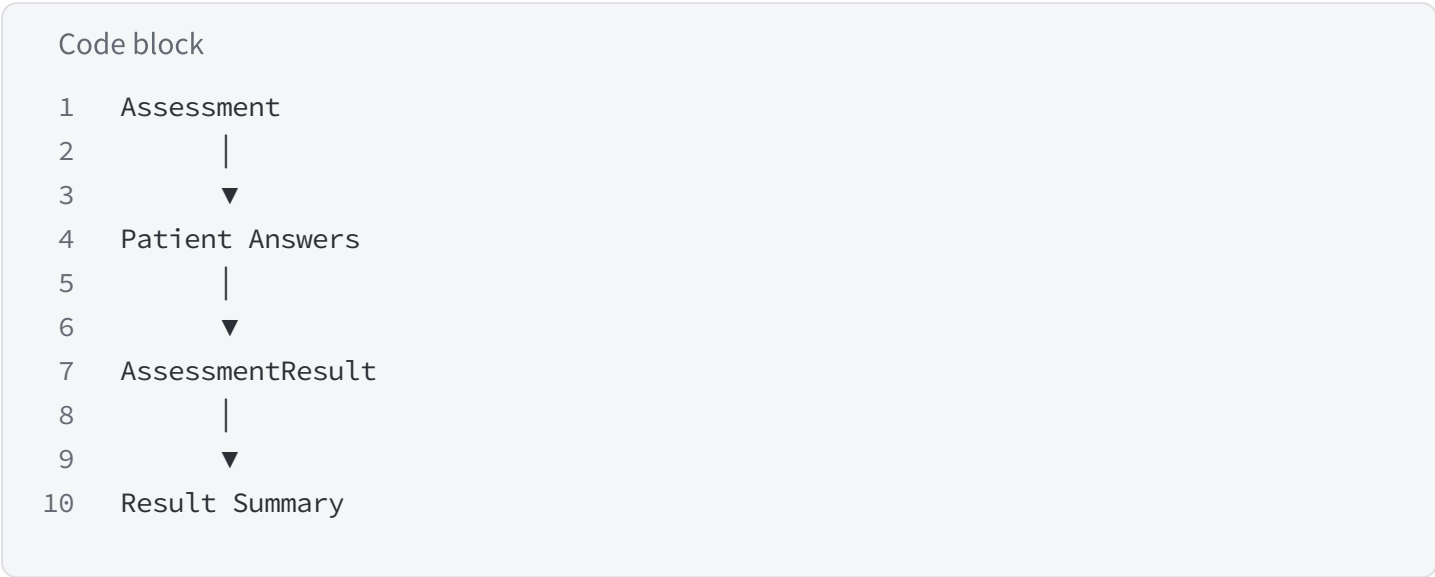
Code block

```
1  Patient
2  |
3  Answers Questions
4  |
5  ▼
6  Assessment Form
7  |
8  ▼
9  Application Layer
10 |
11 ▼
12 AssessmentResult
13 |
14 ▼
15 Repository Interface
16 |
17 ▼
18 Infrastructure Repository
19 |
20 ▼
21 Database
```

Once completed, the assessment responses become part of the patient's healthcare record.

13.9 Assessment Result Processing

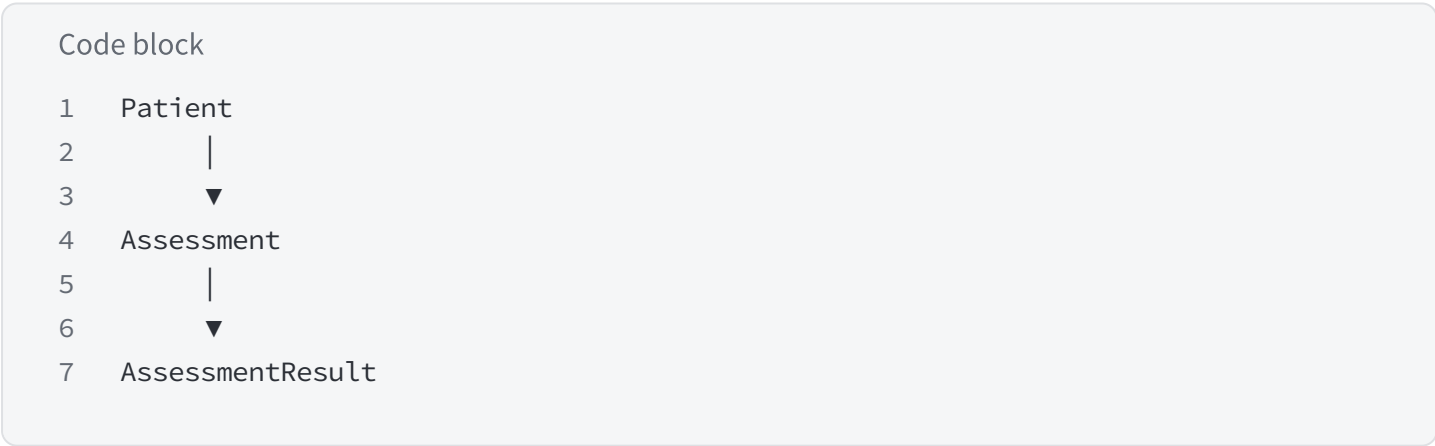
After submission, the implementation stores the completed assessment separately from the assessment definition.



This implementation enables historical comparisons between multiple assessment attempts while preserving the original assessment structure.

13.10 Assessment and Patient Relationship

Patients are closely integrated with the Assessment Module.



Each patient may complete multiple assessments throughout the therapy process. This relationship enables therapists to monitor long-term psychological progress.

13.11 Assessment and Therapist Relationship

Therapists interact with assessments during patient evaluation.

```

1 Therapist
2   |
3 Creates Assessment
4   |
5   ▼
6 Assessment
7   |
8 Reviews Results
9   |
10  ▼
11 AssessmentResult

```

This workflow allows therapists to evaluate patient responses and incorporate assessment findings into future treatment plans.

13.12 DTO Usage

The implementation uses dedicated DTOs to transfer assessment information between the API and the frontend.

DTOs isolate API contracts from domain entities and provide several advantages:

- Reduced data exposure.
- Better validation.
- Smaller response payloads.
- Clear separation between domain models and API models.

This approach improves maintainability while protecting internal entity structures.

13.13 Validation Strategy

Before an assessment is stored, the implementation performs validation within the Application layer to ensure data integrity.

Validation includes:

- Required field validation.
- Existing patient verification.
- Existing therapist verification.
- Assessment existence verification.
- Valid response format.

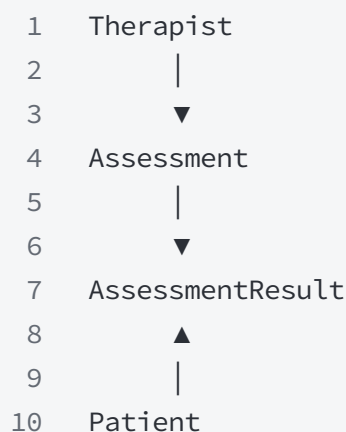
- Model validation.

Invalid requests are rejected before reaching the Infrastructure layer or the database.

13.14 Entity Relationships

The Assessment Module interacts with several domain entities.

Code block



This design clearly separates assessment definitions from patient-generated assessment results while maintaining well-defined relationships between the participating entities.

13.15 Architectural Evaluation

Analysis of the implementation demonstrates several architectural strengths.

- Business logic is isolated from controllers.
- Communication between layers follows Clean Architecture principles.
- Assessment definitions are separated from completed results.
- DTOs isolate API contracts from domain entities.
- Repository interfaces abstract database operations.
- Historical assessment data is preserved.
- Entity relationships are normalized using Entity Framework Core.
- Controllers remain lightweight and focused on request handling.

These design decisions improve maintainability, scalability, and extensibility while supporting the long-term management of psychological assessment data within the NFS platform.

Chapter 14 – Therapy Session Management Module Implementation Analysis

14.1 Introduction

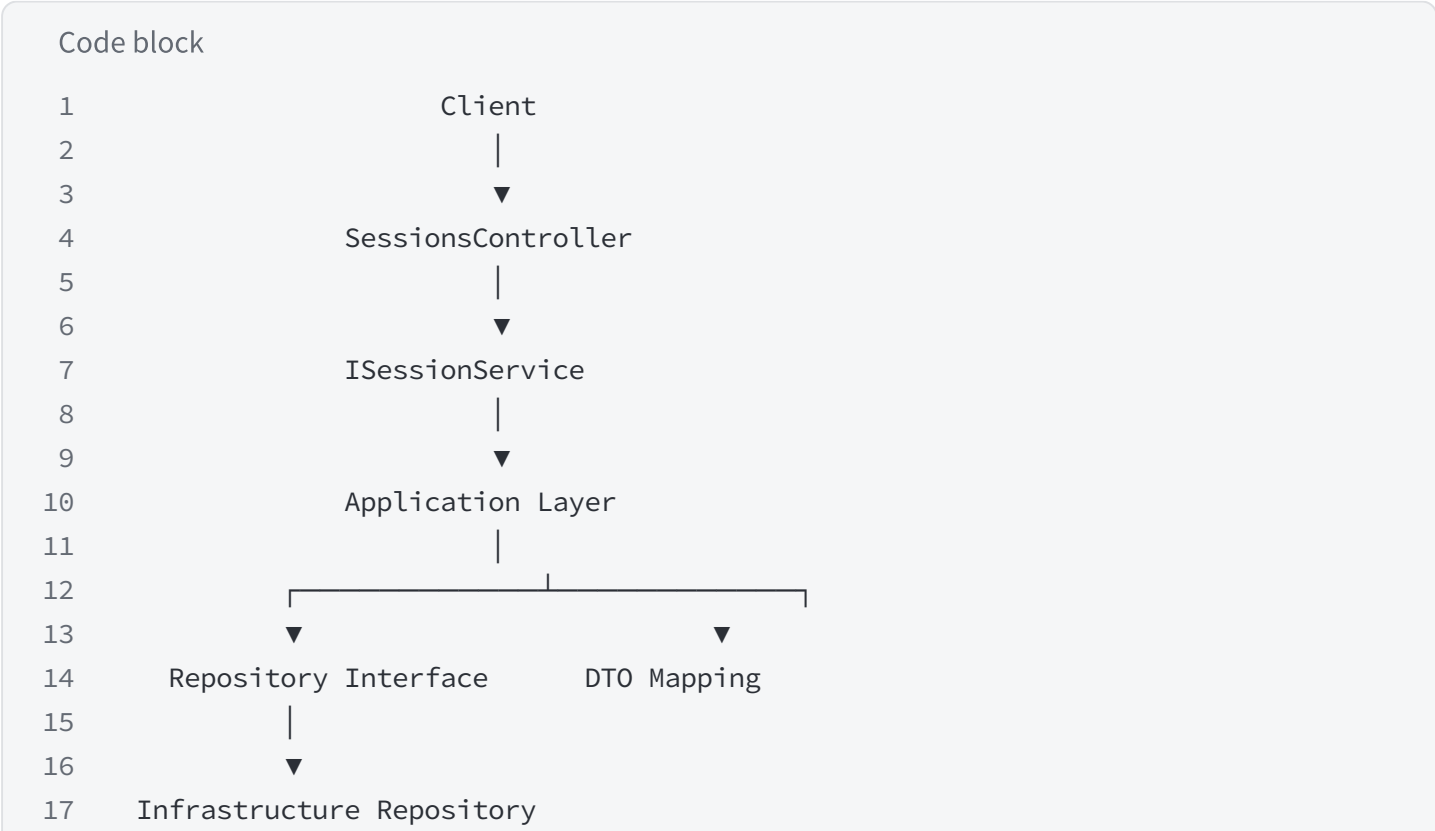
The Therapy Session Management Module is one of the most critical components of the NFS platform, as it represents the actual delivery of mental healthcare services. While the Appointment Module is responsible for scheduling consultations, the Session Module manages the execution of therapy sessions, documentation of clinical interactions, and storage of therapist notes.

Based on the implementation analysis, this module is centered around the **SessionsController** and integrates closely with the **Appointment**, **Patient**, **Therapist**, and **SessionNote** entities. The module enables therapists to conduct therapy sessions, document clinical observations, and maintain complete treatment records throughout the patient's therapeutic journey.

The module follows the principles of **Clean Architecture**, ensuring a clear separation between the Presentation, Application, Domain, and Infrastructure layers.

14.2 Module Architecture

The Session Management Module follows the principles of **Clean Architecture**, separating request handling, business logic, and data persistence into independent layers.



```
18      |
19      ▼
20      ApplicationDbContext
21      |
22      ▼
23      SQL Server
```

The controller is responsible only for processing HTTP requests. Business rules are implemented in the Application layer, while repository implementations within the Infrastructure layer communicate with the database.

This architecture improves maintainability, scalability, testability, and separation of concerns.

14.3 Module Responsibilities

Analysis of the implementation indicates that the Session Management Module performs several healthcare-related operations.

Its primary responsibilities include:

- Creating therapy sessions.
- Retrieving session details.
- Updating session information.
- Recording therapist notes.
- Linking sessions to appointments.
- Maintaining patient treatment history.
- Supporting therapist documentation.
- Preserving completed therapy records.

The module serves as the primary component responsible for documenting therapeutic interactions.

14.4 SessionsController Analysis

The **SessionsController** serves as the API entry point for all therapy session operations.

The controller follows [ASP.NET](#) Core REST API conventions using:

- `[ApiController]`
- `[Route("api/[controller]")]`

Its implementation follows the Thin Controller pattern by delegating all business operations to the Application layer.

Controller responsibilities include:

- Receiving session requests.
- Validating request models.
- Invoking application services.
- Returning standardized HTTP responses.

No clinical or business rules are implemented directly inside the controller.

14.5 Session Entity Analysis

The **Session** entity represents an actual therapy meeting between a therapist and a patient.

Unlike the **Appointment** entity, which represents a scheduled event, the Session entity stores information related to completed or ongoing therapeutic interactions.

Implementation analysis indicates that the Session entity maintains relationships with:

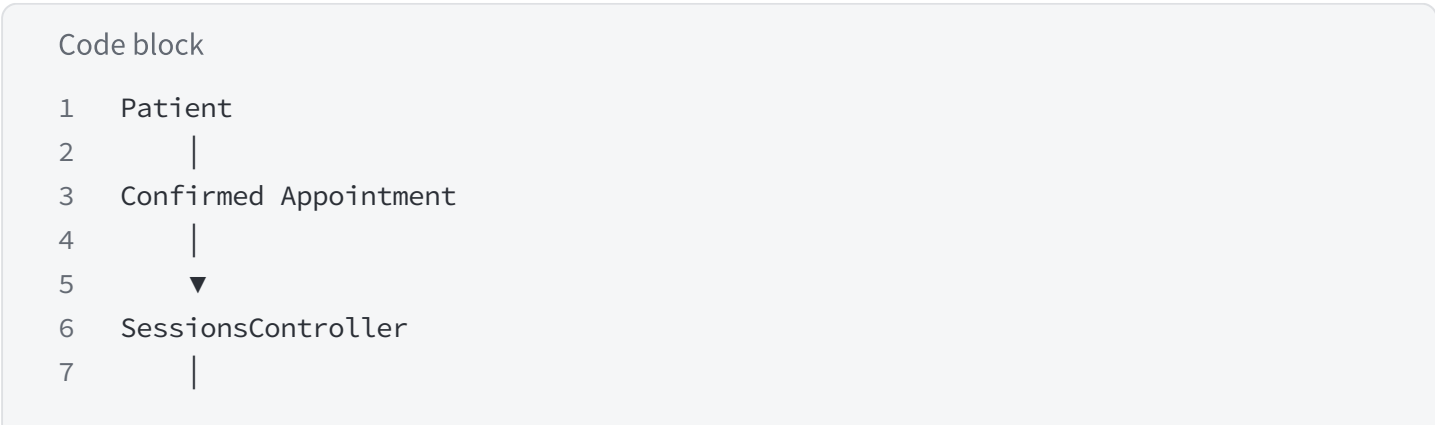
- Appointment
- Therapist
- Patient
- SessionNote

This separation reflects the distinction between appointment scheduling and healthcare service delivery.

14.6 Session Creation Workflow

Therapy sessions are typically created after an appointment has been confirmed.

Implementation workflow:



```

8      ▼
9      Application Layer
10     |
11     ▼
12     Create Session
13     |
14     ▼
15     Repository Interface
16     |
17     ▼
18     Infrastructure Repository
19     |
20     ▼
21     Database

```

This workflow ensures that therapy sessions are associated with valid appointments and authorized participants.

14.7 Session Retrieval Workflow

Therapists and patients may retrieve session information through the API.

Implementation workflow:

Code block

```

1      Client
2      |
3      Request Session
4      |
5      ▼
6      SessionsController
7      |
8      ▼
9      Application Layer
10     |
11     ▼
12     Repository Interface
13     |
14     ▼
15     Infrastructure Repository
16     |
17     ▼
18     Session DTO
19     |

```

20 ▼
21 Client

The implementation returns DTOs rather than exposing domain entities directly, improving encapsulation and API security.

14.8 Session Notes Management

Clinical documentation is managed through the **SessionNote** entity.

Instead of storing therapist notes directly within the Session entity, the implementation separates them into a dedicated entity.

Code block

```
1 Session
2   |
3   ▼
4 SessionNote
```

This design provides several advantages:

- Better database normalization.
- Cleaner entity structure.
- Support for future expansion.
- Independent note management.
- Easier maintenance.

It also enables the possibility of supporting multiple notes or attachments for a single therapy session in future versions of the platform.

14.9 Session Lifecycle

The implementation models therapy sessions as part of a larger healthcare workflow.

Code block

```
1 Appointment
2   |
3   ▼
4 Session Created
```



This lifecycle separates administrative scheduling from clinical service delivery.

14.10 Patient and Session Relationship

Each therapy session is directly associated with a patient.

Code block



Maintaining this relationship allows the platform to build a complete treatment history for every patient throughout their therapeutic journey.

14.11 Therapist and Session Relationship

Therapists conduct and document therapy sessions.

Code block



This relationship enables therapists to maintain detailed clinical documentation that can be referenced during future consultations.

14.12 Integration with Other Modules

The Session Management Module integrates closely with several other modules within the platform.

The primary integrations include:

- **Appointment Module** – Ensures that therapy sessions are associated with valid scheduled appointments.
- **Patient Management Module** – Maintains each patient's treatment history and therapy records.
- **Therapist Management Module** – Enables therapists to conduct and document therapy sessions.
- **Authentication Module** – Restricts access to authorized users through JWT authentication and role-based authorization.

These integrations demonstrate that the Session Module serves as a central component connecting multiple healthcare workflows while maintaining clear architectural boundaries.

14.13 DTO Usage

The implementation uses Data Transfer Objects (DTOs) to exchange session data between the backend and frontend.

DTOs provide several advantages:

- Encapsulation of internal entities.
- Reduced response payload size.
- Improved API security.
- Independent API contracts.
- Easier frontend integration.

This design prevents direct exposure of domain entities and keeps the API independent of the persistence layer.

14.14 Validation Strategy

Before creating or updating therapy sessions, the implementation validates incoming requests within the Application layer.

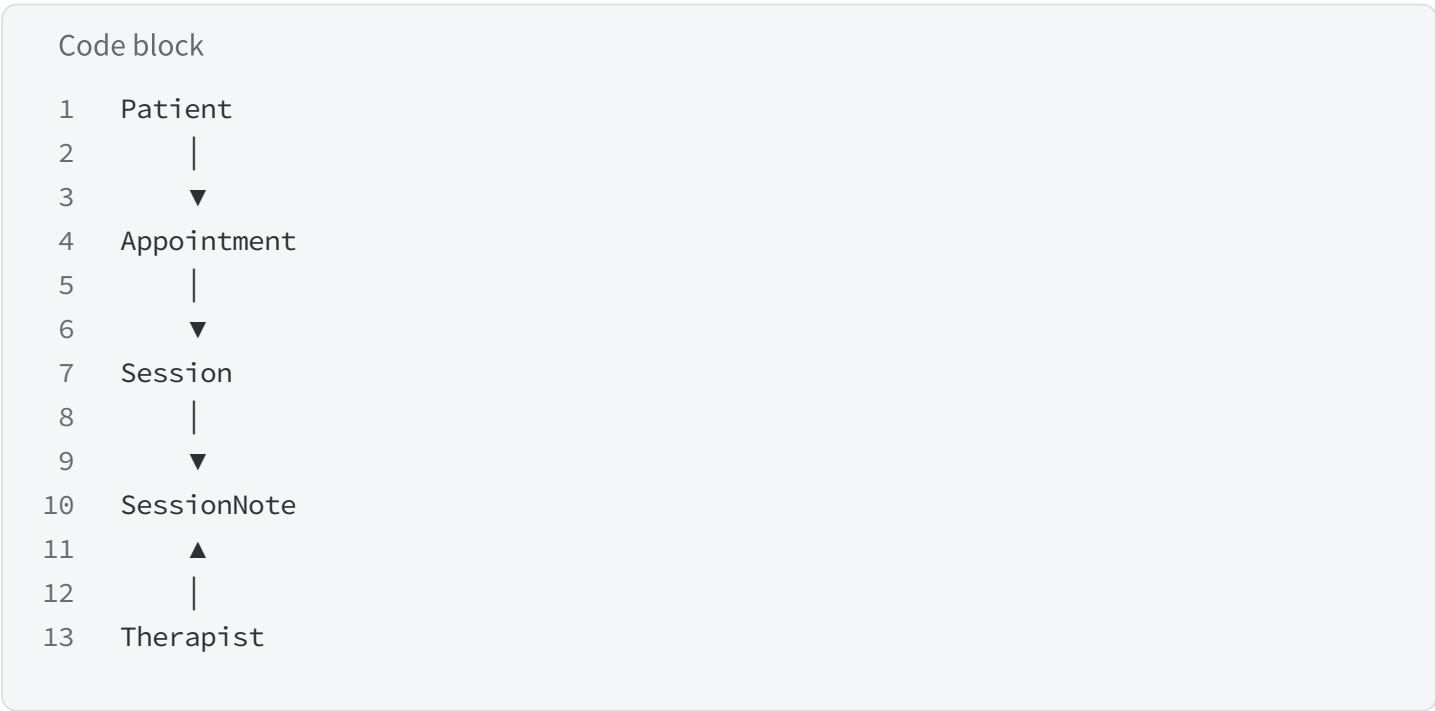
Validation includes:

- Model validation.
- Appointment existence verification.
- Patient existence verification.
- Therapist existence verification.
- Required field validation.
- Entity relationship validation.

These validation steps prevent invalid healthcare records from reaching the Infrastructure layer and being stored in the database.

14.15 Entity Relationships

The Session entity serves as a bridge between scheduling and clinical documentation.



This relationship demonstrates how therapy sessions connect administrative scheduling with healthcare documentation while maintaining well-defined relationships between business entities.

14.16 Architectural Evaluation

Analysis of the implementation reveals several architectural strengths.

- Business logic is isolated from controllers.
- Communication between layers follows Clean Architecture principles.
- Clear separation between appointments and therapy sessions.
- Independent management of therapist notes.
- DTOs isolate API contracts from domain entities.
- Repository interfaces abstract database operations.
- Entity Framework Core is encapsulated within the Infrastructure layer.
- RESTful API implementation.
- Comprehensive integration with other healthcare modules.

These design decisions improve maintainability, scalability, testability, and future extensibility while remaining consistent with the overall Clean Architecture adopted by the NFS platform.

Chapter 15 – Payment Management Module Implementation Analysis

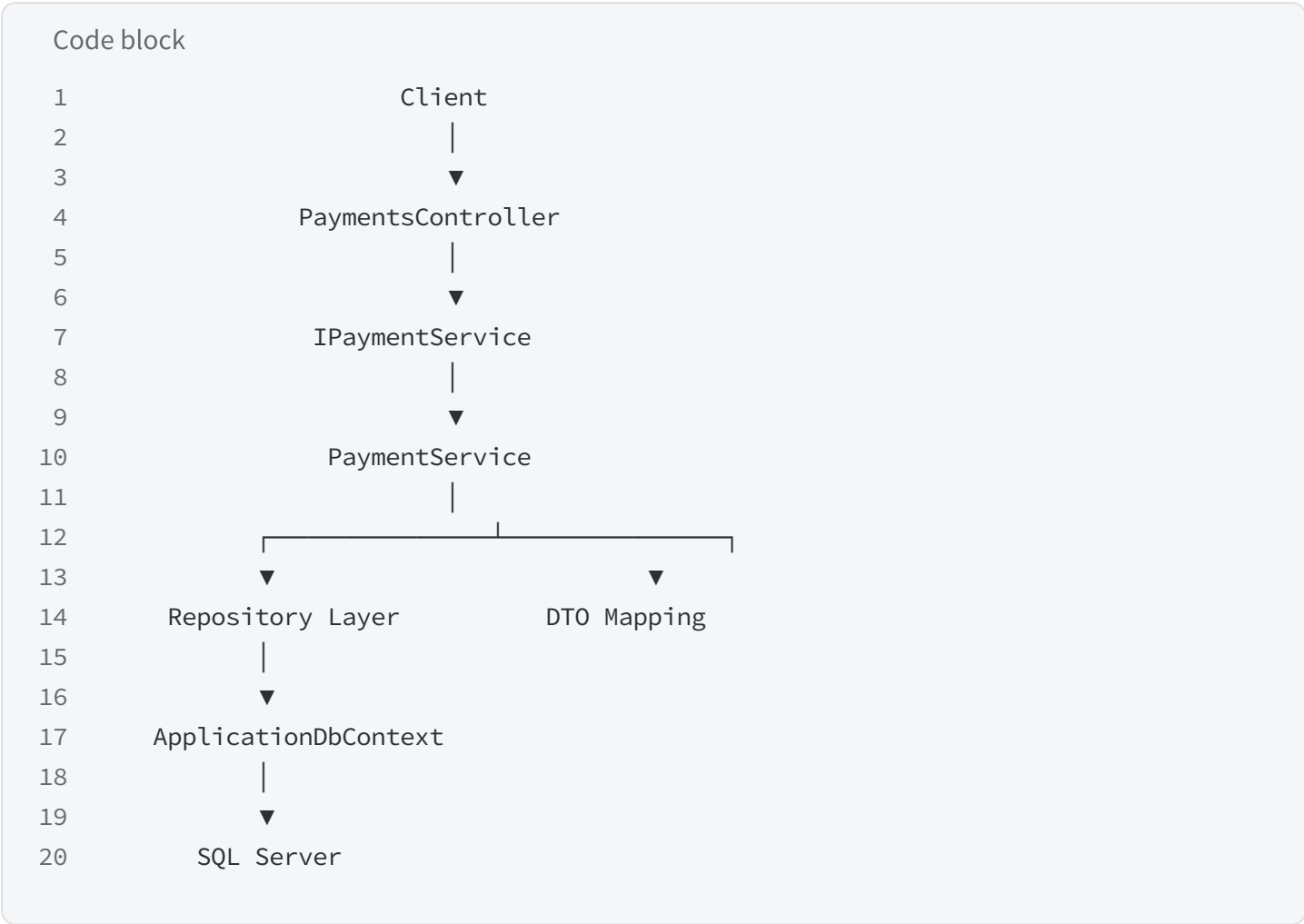
15.1 Introduction

The Payment Management Module is responsible for managing financial transactions within the NFS platform. It enables patients to pay for therapy services while maintaining accurate payment records associated with appointments and therapy sessions. By separating payment functionality into its own module, the system keeps financial operations independent from healthcare workflows, making the application easier to maintain and extend.

Based on the implementation analysis, the module is centered around the **PaymentsController**, which communicates with the **Application layer** through the **IPaymentService** interface. Business logic is implemented inside the service layer, while Entity Framework Core manages database operations in SQL Server.

15.2 Module Architecture

The Payment Management Module follows the same layered architecture used throughout the NFS backend.



The controller receives HTTP requests and delegates all payment-related processing to the Application layer, ensuring a clear separation of responsibilities.

15.3 Module Responsibilities

The Payment Management Module is responsible for managing all payment-related operations within the platform.

Its main responsibilities include:

- Creating payment records.
- Processing patient payments.
- Retrieving payment information.
- Viewing payment history.
- Updating payment status.

- Associating payments with appointments.
- Associating payments with therapy sessions.
- Validating payment requests.

These responsibilities ensure that financial transactions remain accurate, secure, and linked to the appropriate healthcare services.

15.4 PaymentsController Analysis

The **PaymentsController** acts as the presentation layer of the payment subsystem.

It follows [ASP.NET](#) Core REST API conventions using the following attributes:

- `[ApiController]`
- `[Route("api/[controller]")]`

The controller is intentionally lightweight and performs only request-handling operations.

Its responsibilities include:

- Receiving HTTP requests.
- Validating request models.
- Calling the Payment Service.
- Returning standardized HTTP responses.

All business rules and payment processing logic are delegated to the Application layer.

15.5 Payment Entity Analysis

The **Payment** entity represents a financial transaction within the platform.

Each payment record stores information related to a patient's payment for therapy services and is associated with several business entities.

The Payment entity is related to:

- Patient
- Appointment
- Session

Typical information stored includes:

- Payment ID
- Payment amount

- Payment date
- Payment status
- Transaction reference
- Related appointment
- Related therapy session

Keeping payment information in a dedicated entity improves normalization and simplifies future financial reporting.

15.6 Payment Processing Workflow

The payment process follows a structured workflow before a payment is stored in the database.

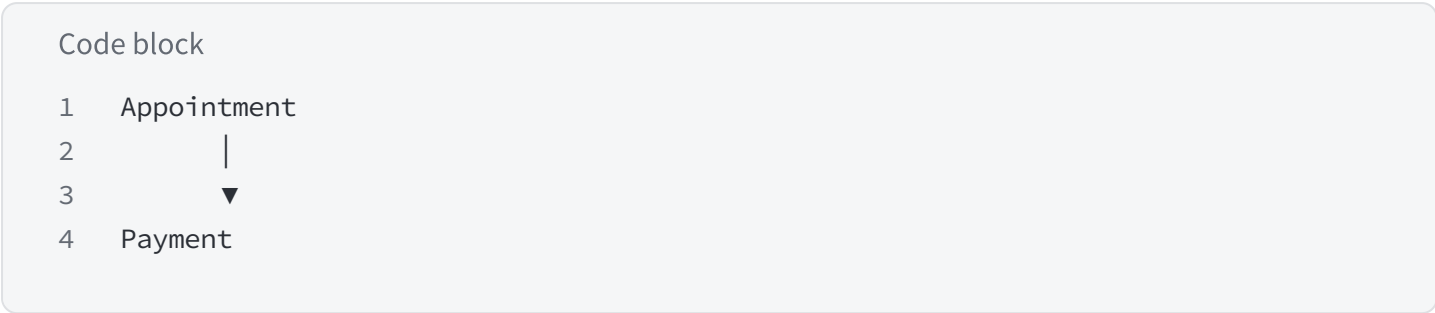
Code block

```
1  Patient
2  |
3  Initiates Payment
4  |
5  ▼
6  PaymentsController
7  |
8  ▼
9  PaymentService
10 |
11 ▼
12 Validate Request
13 |
14 ▼
15 Create Payment
16 |
17 ▼
18 Database
19 |
20 ▼
21 Return Response
```

The service validates the payment request before creating a new payment record, ensuring that only valid transactions are stored.

15.7 Payment and Appointment Relationship

Payments are directly associated with appointments to ensure that scheduled therapy sessions are financially tracked.



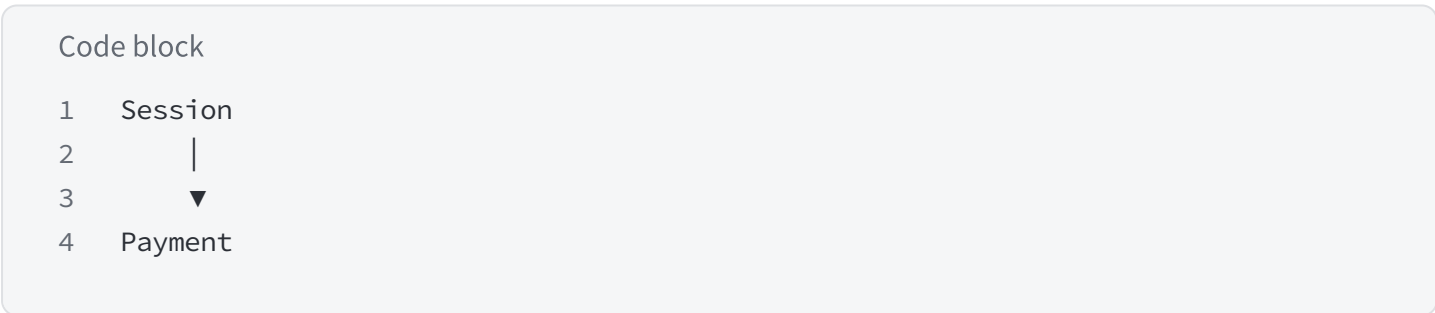
This relationship allows the platform to:

- Track payments for appointments.
- Verify whether appointments have been paid.
- Support financial reporting.
- Maintain payment history for scheduled consultations.

Separating appointment data from payment data keeps both business domains independent while preserving their relationship.

15.8 Payment and Session Relationship

The Payment Module is also integrated with therapy sessions.



Linking payments to sessions enables the system to associate completed therapy sessions with their corresponding financial transactions.

This separation maintains clear boundaries between healthcare records and financial information.

15.9 Payment Status Management

Each payment progresses through several possible states during its lifecycle.

Typical payment workflow:

Code block

```
1 Created
2   |
3   ▼
4 Pending
5   |
6   ▼
7 Completed
```

Other possible statuses include:

- Failed
- Cancelled
- Refunded

Managing payment status independently enables the system to monitor transaction progress without modifying the payment record itself.

15.10 DTO Usage

The module uses **Data Transfer Objects (DTOs)** to exchange payment information between the backend and frontend.

Typical payment DTOs include request and response models used during payment creation, retrieval, and updates.

Using DTOs provides several advantages:

- Protects internal entity structures.
- Reduces unnecessary data transfer.
- Improves API security.
- Simplifies frontend integration.
- Provides clear API contracts.

This approach keeps Entity Framework Core entities hidden from API consumers.

15.11 Validation Strategy

Before processing any payment operation, the module validates incoming requests.

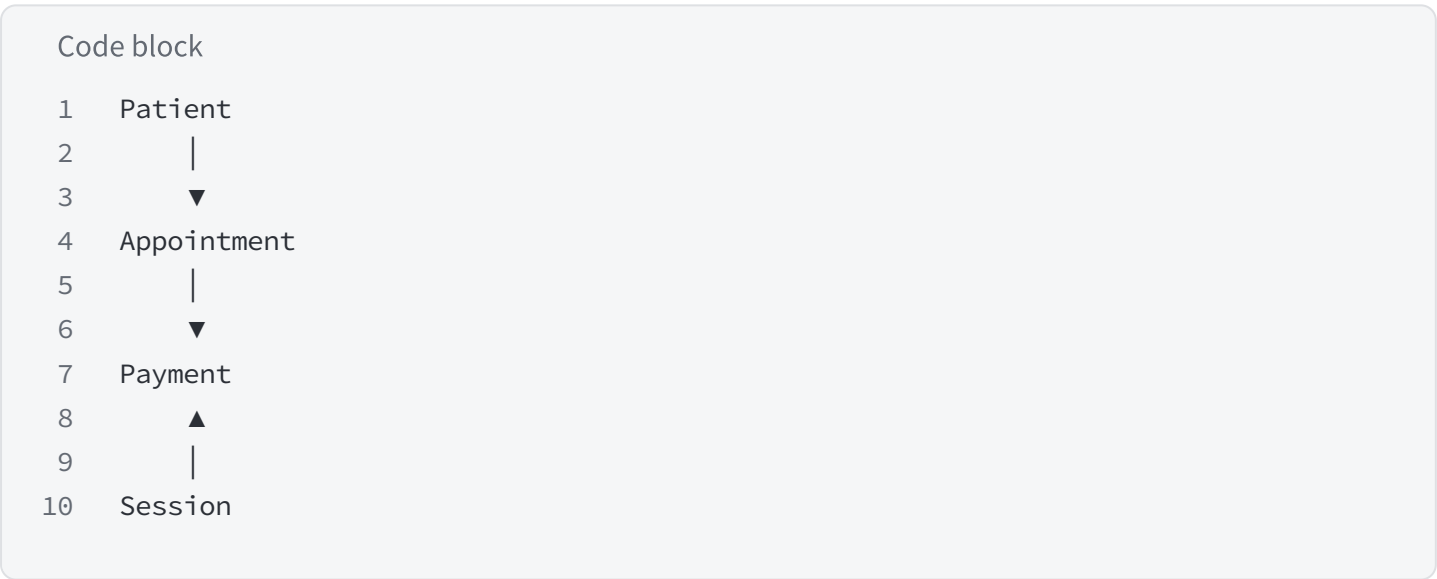
Validation includes:

- Model validation.
- Required field validation.
- Existing patient verification.
- Existing appointment verification.
- Existing session verification.
- Payment amount validation.
- Entity relationship validation.

Invalid requests are rejected before business logic execution, ensuring data integrity throughout the payment process.

15.12 Entity Relationships

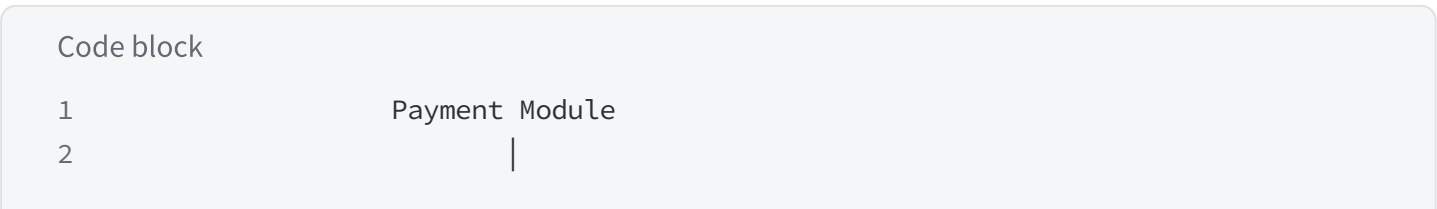
The Payment entity connects several important business entities within the platform.

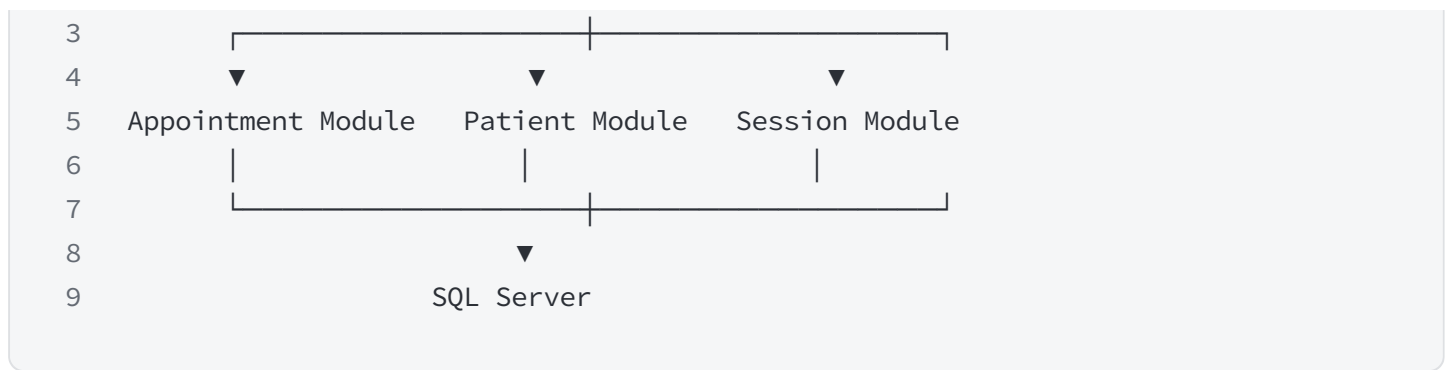


This relationship model enables the platform to associate financial transactions with both scheduled appointments and completed therapy sessions.

15.13 Integration with Other Modules

The Payment Management Module collaborates with several backend modules to support complete healthcare and financial workflows.





The module exchanges information with:

- **Appointment Module** to associate payments with booked appointments.
- **Patient Module** to identify the patient responsible for the payment.
- **Session Module** to link completed therapy sessions with financial records.
- **SQL Server** to store and retrieve payment information.

This integration ensures consistency between healthcare services and financial transactions.

15.14 Architectural Evaluation

The Payment Management Module demonstrates several architectural strengths.

- Clear separation between financial operations and healthcare services.
- Layered architecture with well-defined responsibilities.
- Lightweight RESTful controllers.
- Business logic centralized within the service layer.
- DTO-based communication between backend and frontend.
- Entity Framework Core integration for data persistence.
- Normalized entity relationships.
- Independent payment status management.
- Strong validation before data persistence.
- Easy integration with Appointment, Patient, and Session modules.

These architectural decisions improve maintainability, scalability, security, and future extensibility while ensuring reliable management of financial transactions throughout the NFS platform.

Chapter 16 – Real-Time Communication and Chat Module Implementation Analysis

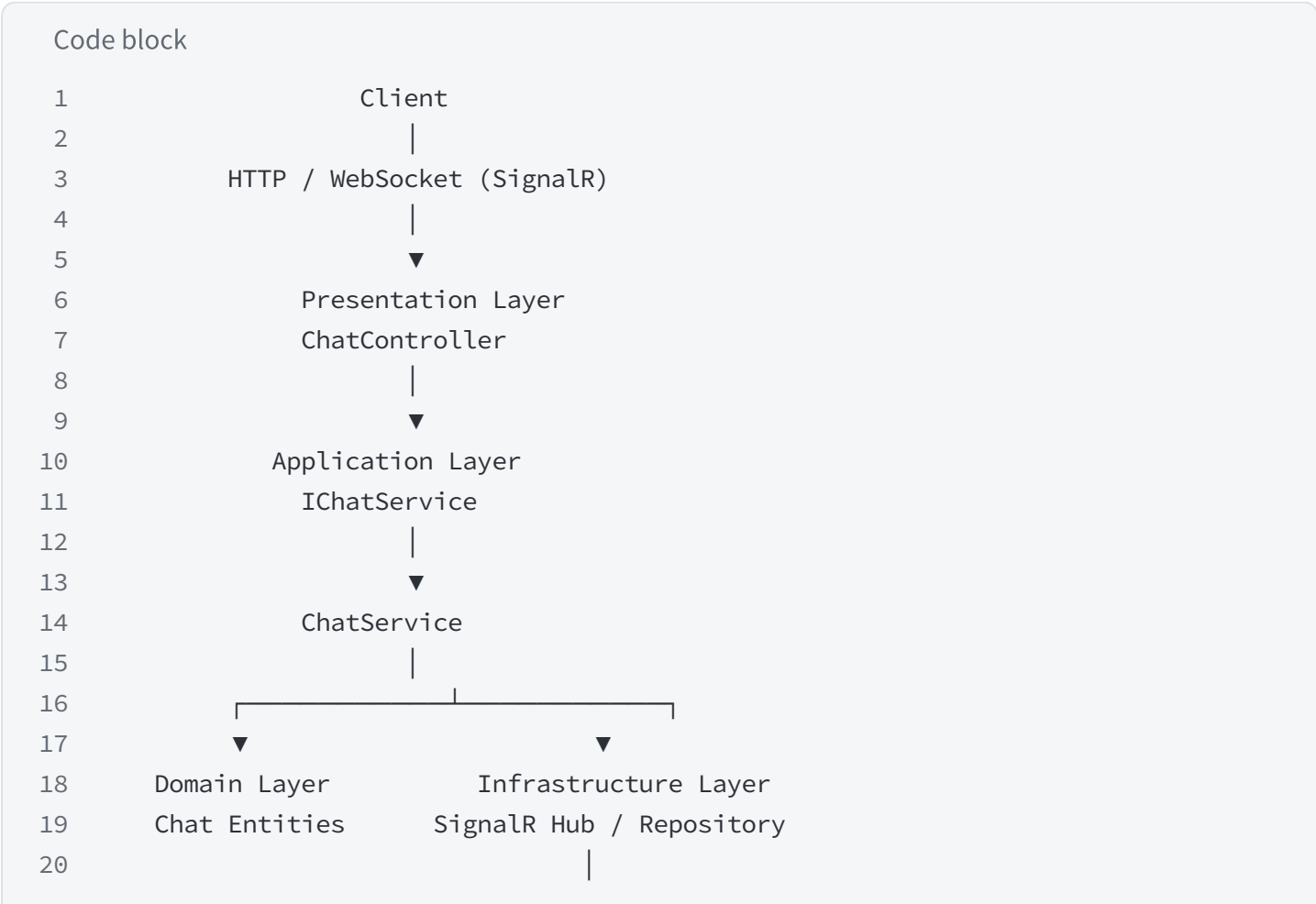
16.1 Introduction

The Real-Time Communication Module provides instant messaging capabilities within the NFS platform, allowing therapists and patients to communicate efficiently during the treatment process. Unlike traditional HTTP-based communication, this module enables bidirectional, real-time data exchange, ensuring that messages are delivered immediately without requiring continuous page refreshes.

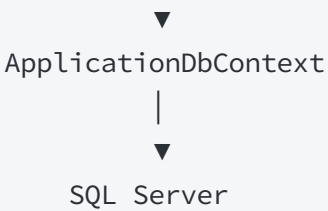
Based on the implementation analysis, the module is centered around the **ChatController** and integrates with a dedicated real-time communication infrastructure. Following the **Clean Architecture** design, the module separates presentation, application logic, domain rules, and infrastructure services, ensuring that real-time communication remains independent from implementation details while maintaining secure communication between authenticated users. The Chat Module plays an essential role in improving patient engagement, facilitating therapist-patient communication, and supporting continuous mental healthcare outside scheduled therapy sessions.

16.2 Module Architecture

The Chat Module follows the **Clean Architecture** adopted throughout the NFS platform while incorporating a real-time communication component.



```
21
22
23
24
25
```



```
graph TD
    ApplicationDbContext --> SQLServer[SQL Server]
```

The diagram illustrates a dependency flow from the `ApplicationDbContext` class to the `SQL Server`. A downward arrow points from `ApplicationDbContext` to `SQL Server`, indicating that the application context depends on the database server.

Unlike other modules, the Chat Module supports both REST API operations and real-time communication while maintaining the dependency flow required by Clean Architecture.

16.3 Module Responsibilities

Analysis of the implementation indicates that the Chat Module performs several communication-related operations.

Its responsibilities include:

- Managing conversations.
- Sending messages.
- Receiving messages.
- Retrieving chat history.
- Delivering messages in real time.
- Associating messages with authenticated users.
- Supporting therapist-patient communication.
- Maintaining conversation records.

These responsibilities enable continuous communication throughout the treatment process while keeping business rules isolated within the Application Layer.

16.4 ChatController Analysis

The **ChatController** serves as the API entry point for chat-related operations.

The controller follows [ASP.NET](#) Core REST API conventions using:

- `[ApiController]`
- `[Route("api/[controller]")]`

Its responsibilities include:

- Receiving chat requests.
- Validating request models.

- Invoking application services.
- Returning conversation data.

Following the principles of Clean Architecture, the controller contains no business logic. Instead, it delegates all processing to the **Application Layer**, which coordinates communication with the Domain and Infrastructure layers.

16.5 Real-Time Communication Workflow

Unlike ordinary API requests, chat messages require immediate delivery.

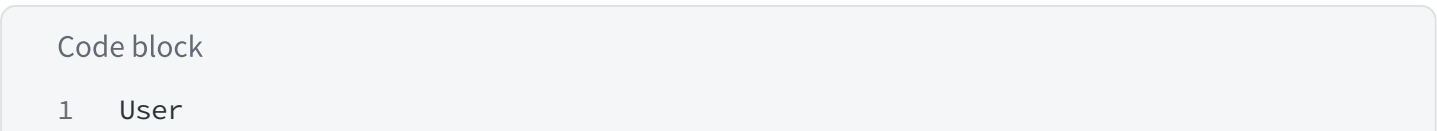
The communication workflow follows the sequence below.



When a therapist sends a reply, the same process occurs in the opposite direction, allowing both participants to communicate instantly.

16.6 Message Processing Workflow

The implementation processes each message through multiple stages before delivery.



```
2  |
3  ▼
4  Validate Request
5  |
6  ▼
7  Application Service
8  |
9  ▼
10 Store Message
11 |
12 ▼
13 SignalR Broadcast
14 |
15 ▼
16 Receiver
```

This workflow ensures that every message is validated and persisted before being delivered to the recipient.

16.7 Conversation Management

The implementation organizes communication into conversations rather than isolated messages.

Each conversation represents a communication channel between participants.

Typical conversation information includes:

- Conversation identifier.
- Participants.
- Creation date.
- Latest activity.
- Message history.

Grouping messages into conversations simplifies retrieval, improves user experience, and aligns with the domain-driven design adopted in Clean Architecture.

16.8 Message Storage

Messages exchanged between therapists and patients are stored in the database to preserve communication history.

Each stored message typically includes:

- Sender identifier.
- Receiver identifier.
- Message content.
- Timestamp.
- Conversation reference.
- Delivery status.

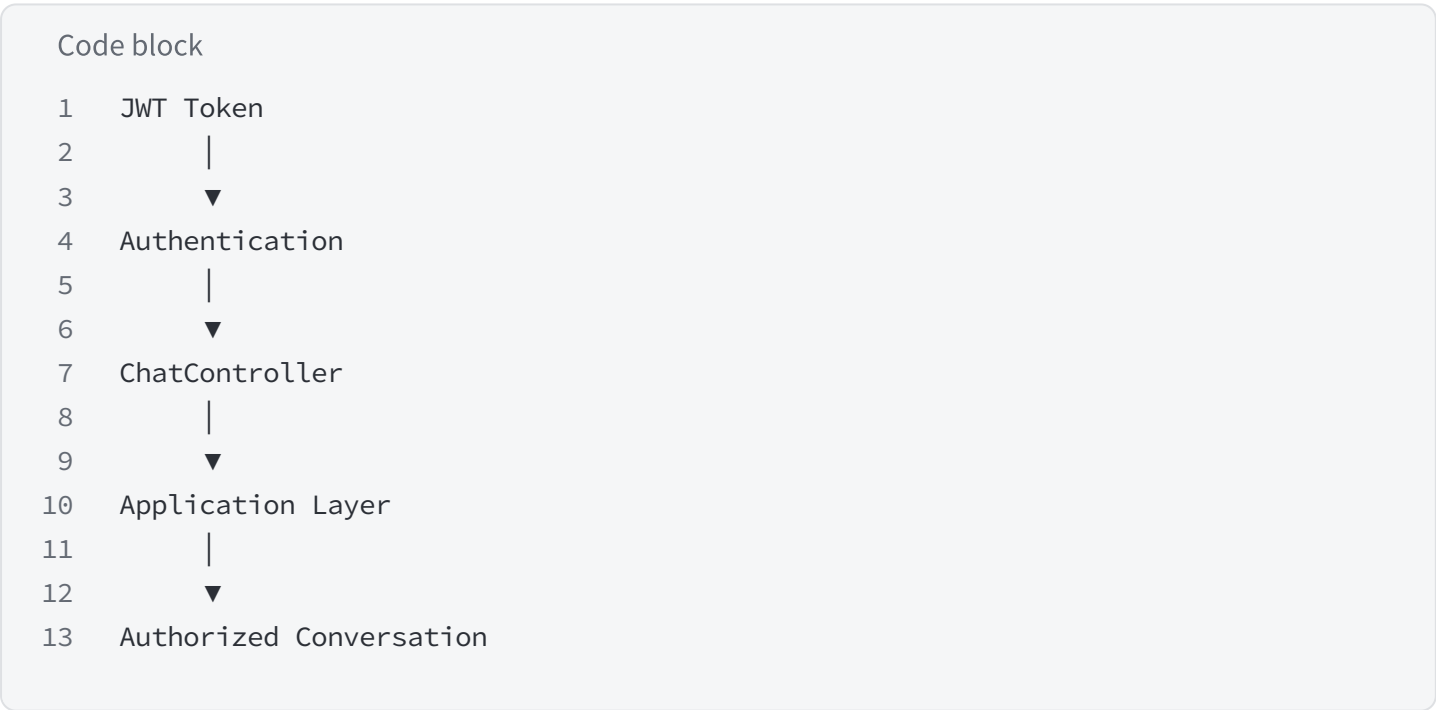
Within Clean Architecture, message persistence is handled through repository interfaces defined in the Application Layer and implemented by the Infrastructure Layer. This separation allows business logic to remain independent of database technologies.

Persisting chat history enables users to review previous discussions and supports continuity of care.

16.9 Integration with Authentication

The Chat Module integrates closely with the Authentication Module.

Only authenticated users are permitted to access chat functionality.



This integration ensures that messages are exchanged only between authorized users while preserving security boundaries across the system.

16.10 Integration with Patient and Therapist Modules

The Chat Module connects directly with the Patient and Therapist Management Modules.

Code block

```
1 Patient
2   |
3   ▼
4 Application Layer
5   |
6 Conversation
7   |
8   ▲
9 Therapist
```

Each participant is identified using authenticated user information, ensuring that conversations remain private and correctly associated with healthcare providers and patients.

16.11 DTO Usage

The implementation uses Data Transfer Objects (DTOs) to exchange chat information between the backend and frontend.

DTOs improve the architecture by:

- Hiding internal domain entities.
- Reducing response payload size.
- Improving validation.
- Simplifying frontend integration.
- Preventing unnecessary exposure of domain and database models.

This approach maintains consistency with the Clean Architecture adopted throughout the project.

16.12 Validation Strategy

Before processing chat requests, the implementation performs several validation steps.

These include:

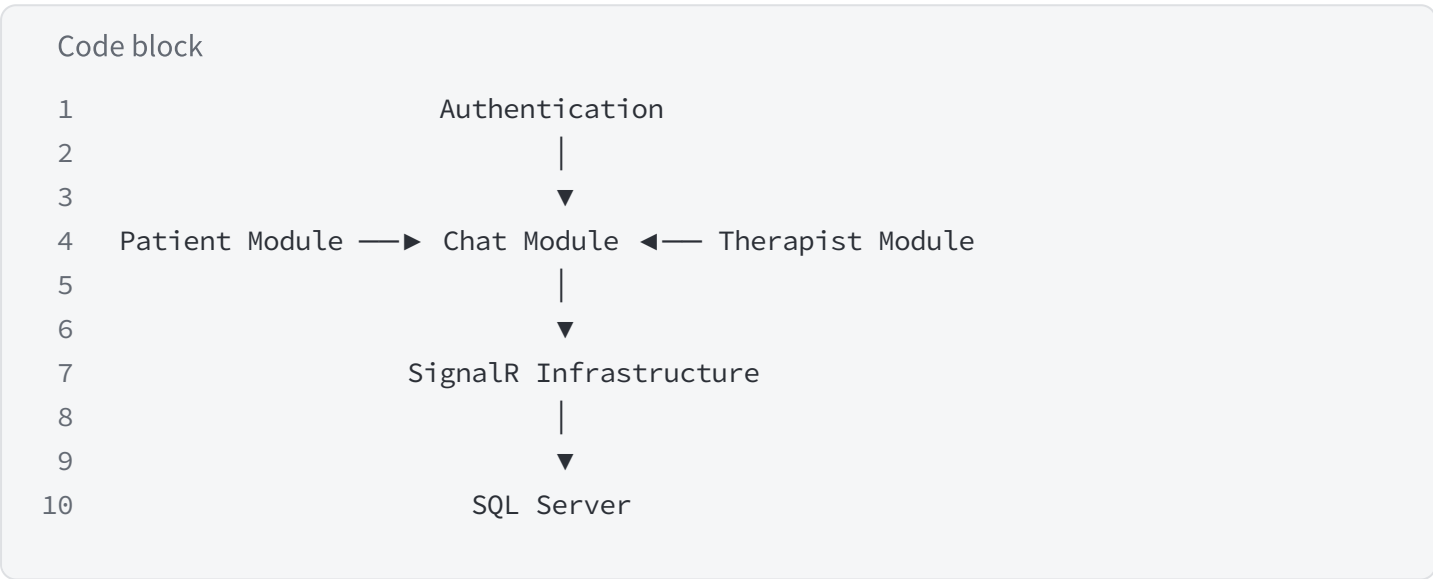
- Model validation.
- Authentication verification.
- Participant existence verification.

- Conversation validation.
- Required field validation.
- Message content validation.

These checks ensure that only valid messages are processed, stored, and delivered while enforcing business rules within the Application Layer.

16.13 Integration with Other Modules

The Chat Module communicates with several major components of the platform.



This integration allows chat functionality to operate as part of the overall mental healthcare ecosystem while maintaining clear separation between the Presentation, Application, Domain, and Infrastructure layers.

16.14 Architectural Evaluation

Analysis of the implementation demonstrates several architectural strengths.

- Compliance with Clean Architecture principles.
- Clear separation of Presentation, Application, Domain, and Infrastructure layers.
- Separation of REST APIs and real-time communication.
- Centralized business logic within the Application Layer.
- Secure communication through authentication and authorization.
- Persistent message storage through repository abstractions.
- DTO-based communication between client and server.

- Modular design that supports future expansion and maintainability.

These characteristics improve scalability, maintainability, and testability while providing a robust foundation for real-time communication within the NFS platform.

Chapter 18 – Database Design and Implementation Analysis

18.1 Introduction

The database is the backbone of the NFS platform, providing persistent storage for all business data generated by the application. Based on the implementation analysis, the backend utilizes Microsoft SQL Server as the primary relational database management system, while Entity Framework Core (EF Core) serves as the Object-Relational Mapping (ORM) framework that bridges the gap between the object-oriented domain model and the relational database.

The database is designed following normalization principles to eliminate redundancy, preserve data integrity, and support efficient querying. Each business entity defined in the **Domain Layer** is mapped to a corresponding database table, with relationships established through primary and foreign keys.

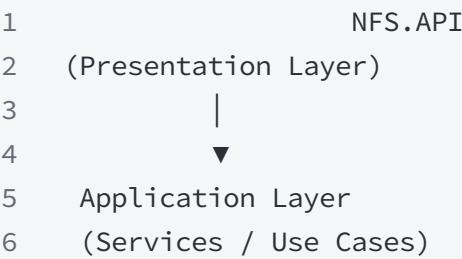
Following the principles of **Clean Architecture**, the database resides entirely within the **Infrastructure Layer**, while the Application Layer accesses data through repository abstractions. This separation ensures that business rules remain independent of database technologies.

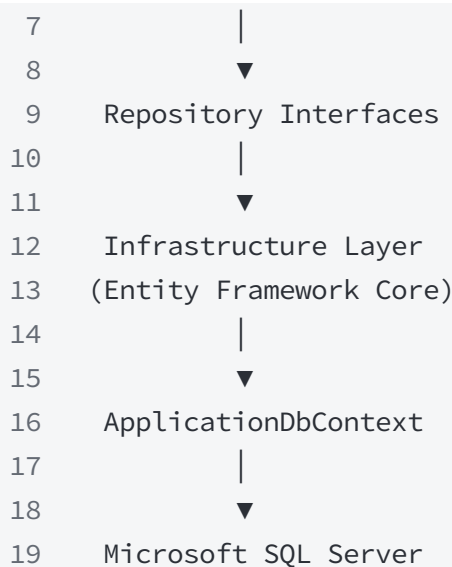
The database supports all major platform functionalities, including authentication, therapist management, patient records, appointments, therapy sessions, assessments, diary entries, chat, and payment processing.

18.2 Database Architecture

The implementation follows the **Clean Architecture** adopted throughout the NFS platform. The Application Layer communicates with SQL Server through repository interfaces implemented using Entity Framework Core.

Code block





This architecture abstracts database operations from business logic, making the application easier to maintain, test, and extend while preserving dependency inversion.

18.3 Database Technology Stack

The backend data layer is implemented using the following technologies:

- Microsoft SQL Server
- Entity Framework Core (EF Core)
- ApplicationDbContext
- Repository Pattern
- LINQ (Language Integrated Query)
- [ASP.NET](#) Core Identity (where applicable)

Together, these technologies provide a robust and scalable persistence layer while aligning with the principles of Clean Architecture.

18.4 Entity Framework Core Integration

Entity Framework Core is responsible for mapping C# domain entities to relational database tables.

The implementation uses **ApplicationDbContext** as the central database context that manages:

- Entity registration.
- Table mapping.
- Relationship configuration.

- Change tracking.
- Database transactions.
- Query execution.

Within Clean Architecture, **ApplicationDbContext** is implemented in the **Infrastructure Layer**, while the Application Layer interacts with it indirectly through repository interfaces. This abstraction allows developers to work with strongly typed C# objects instead of writing raw SQL for most operations.

18.5 Database Schema

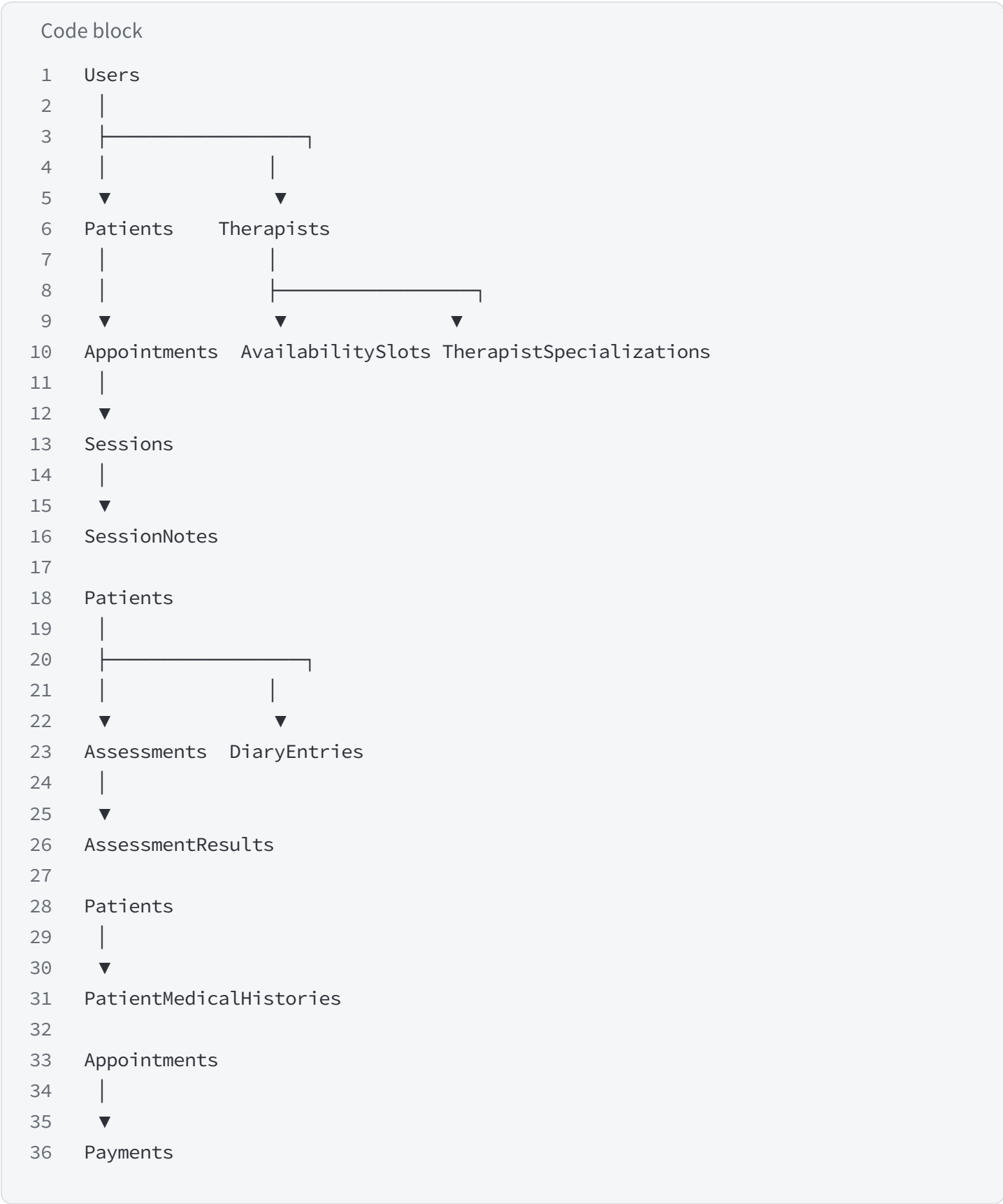
Based on the implementation analysis, the primary database tables correspond to the following domain entities:

- Users
- Patients
- Therapists
- TherapistSpecializations
- Appointments
- Sessions
- SessionNotes
- Assessments
- AssessmentResults
- PatientMedicalHistories
- DiaryEntries
- Payments
- Conversations
- Messages
- Notifications
- AvailabilitySlots

Each table represents a distinct business entity defined within the Domain Layer and contributes to the overall functionality of the platform.

18.6 Entity Relationships

The database schema establishes several relationships among entities.



These relationships ensure that business operations maintain referential integrity across the database while supporting the domain model defined in Clean Architecture.

18.7 Primary Keys and Foreign Keys

Each table uses a Primary Key (PK) to uniquely identify records.

Relationships between tables are maintained using Foreign Keys (FKs).

Examples include:

- Patient → User
- Therapist → User
- Appointment → Patient
- Appointment → Therapist
- Session → Appointment
- Assessment → Patient
- Payment → Appointment
- AvailabilitySlot → Therapist

Foreign key constraints help prevent invalid or orphaned records and preserve consistency throughout the database.

18.8 Database Normalization

The implementation follows database normalization principles by separating business concepts into independent tables.

Examples include:

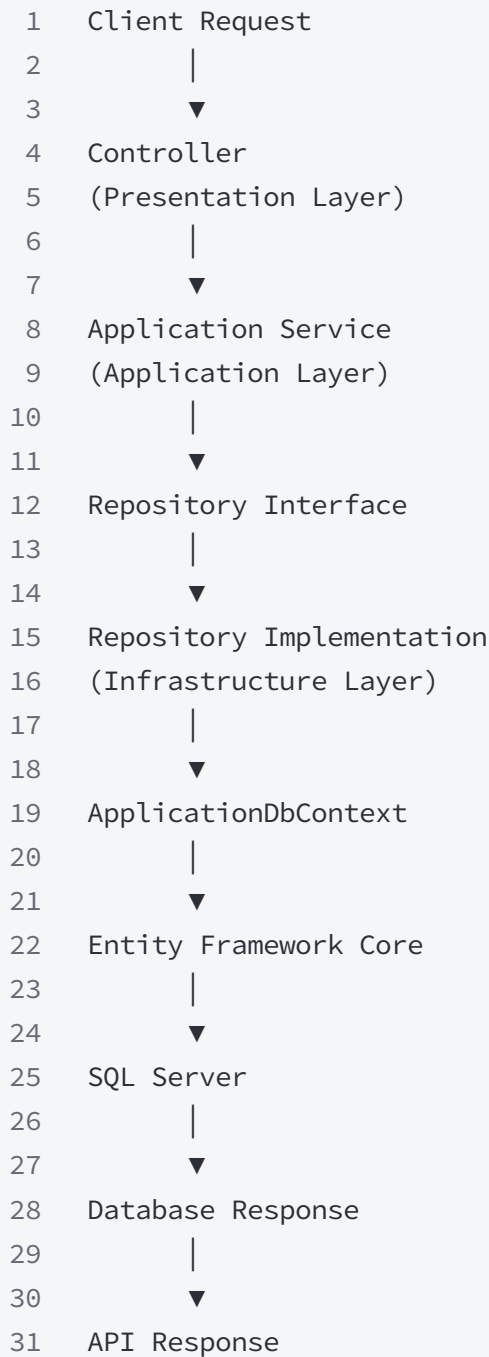
- Medical history is stored separately from patient information.
- Therapist specializations are managed through a junction table.
- Session notes are separated from therapy sessions.
- Assessment results are stored independently of assessment definitions.
- Payments are isolated from appointment records.

This normalized design reduces redundancy, improves maintainability, and supports the separation of responsibilities encouraged by Clean Architecture.

18.9 Data Access Workflow

The typical workflow for retrieving or updating data is illustrated below.

Code block



This workflow ensures that database operations remain encapsulated within the Infrastructure Layer while business rules remain inside the Application Layer.

18.10 Query Processing

The implementation uses LINQ (Language Integrated Query) to perform database operations.

LINQ provides:

- Strongly typed queries.
- Compile-time checking.
- Improved readability.

- Database abstraction.
- Automatic SQL generation.

Within Clean Architecture, LINQ queries are executed inside repository implementations, keeping the Application Layer independent of Entity Framework Core.

Using LINQ simplifies development while maintaining compatibility with SQL Server.

18.11 Data Integrity

The implementation maintains data integrity through multiple mechanisms.

These include:

- Primary key constraints.
- Foreign key constraints.
- Required fields.
- Entity validation.
- Model validation.
- Referential integrity.
- Transaction support.

These mechanisms reduce the possibility of inconsistent or invalid data while preserving the integrity of the domain model.

18.12 Database Security

The database layer incorporates several security practices.

These include:

- Parameterized queries through Entity Framework Core.
- JWT-based authentication before database access.
- Role-based authorization.
- Controlled access through application services.
- Prevention of SQL injection via ORM-generated queries.

Within Clean Architecture, all database access is controlled through repository implementations and application services, ensuring that sensitive healthcare information remains protected.

18.13 Performance Considerations

The implementation benefits from several performance optimizations provided by Entity Framework Core.

These include:

- Change tracking.
- Query optimization.
- Deferred execution.
- Navigation property loading.
- Efficient relationship management.

Future optimization opportunities include:

- Database indexing.
- Query caching.
- Read-only query optimization using `AsNoTracking()`.
- Stored procedures for complex reporting.
- Database partitioning for large datasets.

These optimizations can be incorporated within the Infrastructure Layer without affecting business logic, demonstrating one of the advantages of Clean Architecture.

18.14 Architectural Evaluation

Analysis of the database implementation reveals several strengths.

- Compliance with Clean Architecture principles.
- Well-normalized relational schema.
- Clear separation between Domain, Application, and Infrastructure layers.
- Entity Framework Core integration through repository implementations.
- SQL Server reliability.
- Scalable database design.
- Consistent use of primary and foreign keys.
- Decoupled data access through repository abstractions.
- Independent business logic unaffected by database implementation details.

These characteristics contribute to a maintainable, scalable, and testable persistence layer while supporting the long-term evolution of the NFS platform.

Chapter 19 – System Testing and Validation

19.1 Introduction

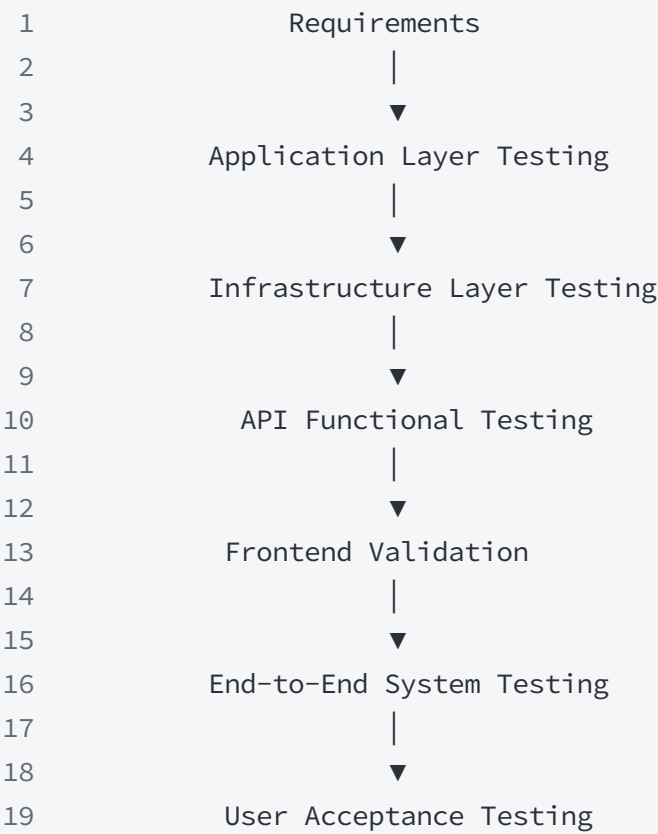
Testing is an essential phase in the software development life cycle, ensuring that the NFS platform satisfies its functional and non-functional requirements while maintaining reliability, security, and performance. Following the implementation of the backend, frontend, and database, comprehensive testing was conducted to verify that each module operates correctly both independently and as part of the integrated system.

The NFS platform was tested using multiple testing techniques, including API testing, database validation, frontend verification, authentication testing, and end-to-end functional testing. These activities helped identify defects, verify business logic, and ensure that all system components communicate correctly while validating the interactions between the Presentation, Application, Domain, and Infrastructure layers of the Clean Architecture.

19.2 Testing Strategy

The testing process followed a structured approach in which each architectural layer and system module was verified individually before testing the complete integrated system.

Code block



This strategy ensured that issues were detected early while verifying that the final system met the expected requirements and that each layer interacted correctly with the others.

19.3 Testing Environment

The NFS platform was tested using the following development environment.

- [ASP.NET](#) Core
- .NET
- Entity Framework Core
- Microsoft SQL Server
- Visual Studio
- Swagger UI
- Postman
- Angular
- Git

This environment closely matched the development environment used throughout the project.

19.4 API Testing

The REST APIs were tested to verify that each endpoint correctly handled client requests and returned the expected HTTP responses.

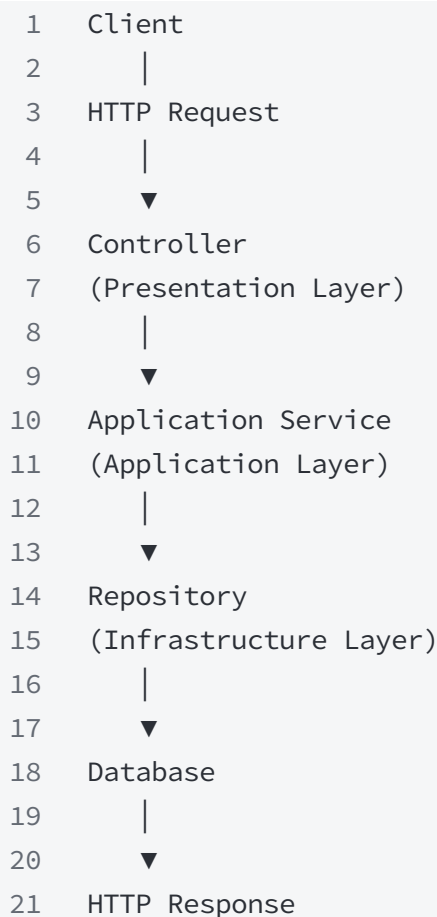
Testing focused on:

- Request validation.
- Response correctness.
- HTTP status codes.
- Authorization.
- Error handling.
- CRUD operations.
- DTO validation.

Swagger UI and Postman were used to send requests and inspect responses.

Example API verification process:

Code block



Each API endpoint was tested using both valid and invalid input data.

19.5 Authentication Testing

The Authentication Module underwent extensive testing to ensure secure user access.

The following scenarios were verified:

- User registration.
- User login.
- JWT token generation.
- Invalid login attempts.
- Unauthorized API access.
- Expired token handling.
- Role-based authorization.
- Protected endpoint access.

Successful authentication was confirmed by receiving a valid JWT access token and accessing protected resources.

19.6 Database Testing

Database testing verified that all entities were correctly mapped using Entity Framework Core.

The following aspects were validated:

- Entity relationships.
- Foreign key constraints.
- Data insertion.
- Data retrieval.
- Data updates.
- Data deletion.
- Cascade behaviors.
- Migration execution.

Since the project follows Clean Architecture, database operations were validated through repository implementations rather than direct access from the business layer.

Database consistency was confirmed after performing CRUD operations across multiple modules.

19.7 Frontend Testing

The frontend application was tested to verify that user interfaces functioned correctly and communicated successfully with the backend.

Testing activities included:

- Page navigation.
- Form validation.
- API integration.
- Authentication flow.
- Dashboard functionality.
- Appointment booking.
- Session management.
- Responsive layout verification.
- Error message display.

Each page was tested using different user scenarios to ensure consistent behavior.

19.8 Integration Testing

Integration testing ensured that all system modules worked together correctly.

The following interactions were verified:

- Authentication ↔ User Management
- Patient Module ↔ Appointment Module
- Therapist Module ↔ Appointment Module
- Appointment Module ↔ Payment Module
- Assessment Module ↔ Patient Module
- Chat Module ↔ Authentication Module
- Frontend ↔ Backend APIs
- Backend ↔ SQL Server

These tests confirmed that information flowed correctly between the Presentation, Application, Domain, and Infrastructure layers while preserving the separation of responsibilities defined by Clean Architecture.

19.9 Functional Testing

Functional testing verified that every implemented feature behaved according to the project requirements.

Major functional scenarios included:

- User registration and login.
- Viewing therapist profiles.
- Searching therapists.
- Booking appointments.
- Managing therapist availability.
- Conducting therapy sessions.
- Completing psychological assessments.
- Recording patient diary entries.
- Processing payments.

- Sending chat messages.

Each function was evaluated against its expected output.

19.10 Validation Testing

Input validation was tested to ensure that invalid data was rejected before reaching the business layer.

Validation scenarios included:

- Missing required fields.
- Invalid email addresses.
- Incorrect passwords.
- Invalid identifiers.
- Duplicate records.
- Invalid dates.
- Empty request bodies.

Validation was performed before executing business rules in the Application Layer, ensuring that invalid requests never reached the Infrastructure Layer.

The API correctly returned appropriate validation errors without affecting database integrity.

19.11 Security Testing

Security testing verified that sensitive healthcare information remained protected.

Testing focused on:

- JWT authentication.
- Authorization policies.
- Unauthorized endpoint access.
- SQL injection prevention through Entity Framework Core.
- Secure password handling.
- Protected API routes.

The results confirmed that unauthorized users could not access restricted resources and that data access remained controlled through the Application Layer.

19.12 Error Handling Testing

Exception handling was tested by intentionally generating invalid requests.

Examples included:

- Invalid entity identifiers.
- Missing resources.
- Incorrect request formats.
- Unauthorized access attempts.
- Database constraint violations.

The application consistently returned meaningful HTTP status codes and descriptive error messages without exposing internal implementation details.

19.13 Performance Testing

Basic performance testing was conducted to evaluate application responsiveness.

The following aspects were observed:

- API response time.
- Database query performance.
- Authentication speed.
- Frontend loading time.
- Page navigation.
- Concurrent request handling.

The separation of responsibilities introduced by Clean Architecture had no noticeable negative impact on application performance, and the system demonstrated acceptable responsiveness for the expected project workload.

19.14 Testing Results

The testing process confirmed that the implemented modules functioned correctly under normal operating conditions.

The overall results indicated:

- Successful API execution.
- Correct database operations.

- Proper authentication.
- Reliable frontend-backend communication.
- Accurate entity relationships.
- Stable appointment workflow.
- Correct payment processing.
- Functional real-time communication.

In addition, testing confirmed that the interactions between the Presentation, Application, Domain, and Infrastructure layers complied with the principles of Clean Architecture.

No critical defects preventing normal system operation were identified during the final validation phase.

19.15 Challenges Encountered During Testing

Several implementation challenges were encountered throughout the testing process.

These included:

- Resolving Entity Framework Core relationship configuration issues.
- Debugging authentication and authorization policies.
- Handling DTO validation errors.
- Verifying complex entity relationships.
- Synchronizing frontend requests with backend APIs.
- Managing database migrations during development.
- Verifying repository implementations and dependency injection configuration within the Clean Architecture.

Chapter 20 – Conclusion and Future Work

20.1 Introduction

This chapter concludes the development and implementation of the NFS Mental Health Platform. Throughout this project, a complete software solution was designed and implemented to provide an integrated environment that connects patients with qualified therapists while supporting appointment scheduling, therapy sessions, psychological assessments, secure communication, and payment management.

The project was developed using modern software engineering principles, including **Clean Architecture**, RESTful APIs, Entity Framework Core, SQL Server, JWT authentication,

dependency injection, and a modular frontend. The implementation focused on building a scalable, maintainable, and secure platform capable of supporting future enhancements in the field of digital mental healthcare.

20.2 Project Overview

The primary objective of this project was to develop a comprehensive mental healthcare platform capable of supporting the complete treatment lifecycle, from patient registration to therapy session completion.

The implemented system includes several integrated modules, including:

- Authentication and Authorization.
- Therapist Management.
- Patient Management.
- Appointment Scheduling.
- Therapy Session Management.
- Psychological Assessments.
- Payment Processing.
- Real-Time Chat.
- Personal Diary Management.
- Database Management.
- Responsive Frontend Application.

Each module was developed independently while maintaining seamless integration with the rest of the platform through RESTful APIs, repository abstractions, and a shared SQL Server database.

20.3 Achieved Objectives

The project successfully achieved the objectives defined during the analysis and design phases.

Major accomplishments include:

- Developing a secure authentication system using JWT.
- Implementing role-based authorization.
- Designing a normalized SQL Server database.
- Developing a modular [ASP.NET](#) Core Web API.

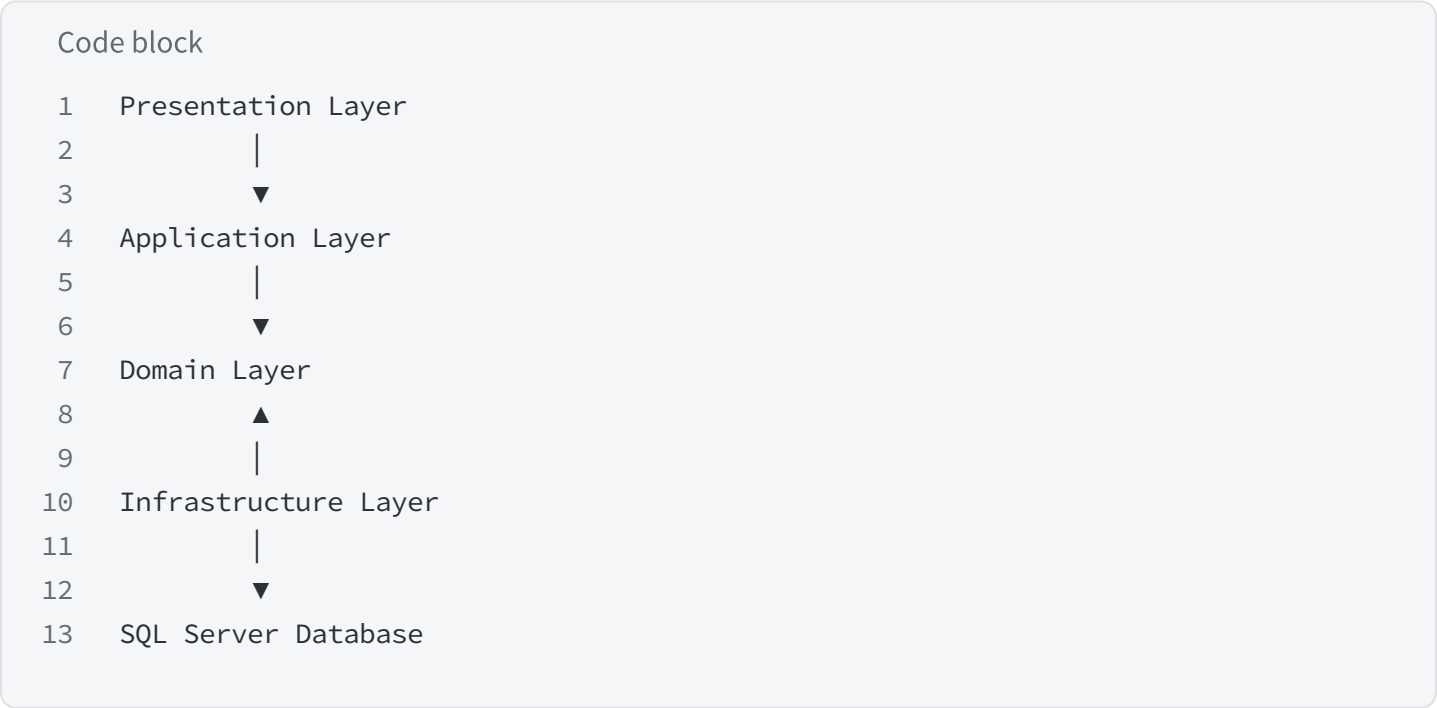
- Applying Clean Architecture principles.
- Implementing the Repository Pattern and Dependency Injection.
- Building reusable frontend components.
- Implementing appointment scheduling.
- Supporting therapist availability management.
- Managing therapy sessions.
- Implementing psychological assessment workflows.
- Supporting secure patient-therapist communication.
- Recording payment transactions.
- Maintaining patient medical records and diary entries.

These achievements demonstrate that the system satisfies the functional requirements established at the beginning of the project.

20.4 Architectural Achievements

The project adopted the **Clean Architecture** pattern, which separates responsibilities into independent layers while ensuring that business rules remain isolated from infrastructure concerns.

The architecture consists of:



Following the Dependency Rule of Clean Architecture, the Domain Layer remains independent of external frameworks and technologies, while the Infrastructure Layer implements data access

and other external services.

This architecture provided several advantages:

- Clear separation of concerns.
- Independent business logic.
- Improved maintainability.
- Better scalability.
- Easier unit testing.
- Loose coupling between application components.
- Simplified dependency management.
- Easier replacement of infrastructure technologies.
- Improved code reusability.

These characteristics make the application suitable for long-term maintenance and future expansion.

20.5 Technical Contributions

The implementation demonstrates the practical application of several modern software development technologies.

Technologies successfully integrated into the project include:

- [ASP.NET](#) Core Web API
- Clean Architecture
- C#
- Entity Framework Core
- SQL Server
- JWT Authentication
- Dependency Injection
- Repository Pattern
- LINQ
- Angular
- SignalR
- Git and GitHub

The integration of these technologies resulted in a robust full-stack healthcare application.

20.6 Challenges Encountered

During the development process, several technical challenges were encountered.

Some of the most significant challenges included:

- Designing a scalable database schema.
- Managing complex entity relationships.
- Configuring Entity Framework Core migrations.
- Implementing secure JWT authentication.
- Maintaining clear separation between the Presentation, Application, Domain, and Infrastructure layers.
- Designing repository interfaces and implementations.
- Synchronizing frontend and backend development.
- Handling API validation and exception management.
- Integrating multiple system modules.

Each challenge was addressed through iterative development, debugging, and continuous testing.

20.7 Lessons Learned

The project provided valuable practical experience in software engineering and full-stack application development.

Key lessons learned include:

- The importance of software architecture before implementation.
- Benefits of Clean Architecture in enterprise applications.
- Effective use of Entity Framework Core.
- Designing secure RESTful APIs.
- Applying the Repository Pattern and Dependency Injection.
- Managing database relationships efficiently.
- Building reusable frontend components.
- Applying authentication and authorization correctly.
- Performing systematic software testing.

- Using version control effectively in collaborative development.

These experiences strengthened both technical and software engineering skills.

20.8 Future Work

Although the current implementation satisfies the project's objectives, several enhancements can further improve the platform.

Artificial Intelligence

- AI-powered therapist recommendation.
- Intelligent chatbot for mental health support.
- Automated assessment interpretation.
- Mood prediction using machine learning.
- Personalized treatment recommendations.

Telemedicine

- Secure video consultation.
- Voice consultation.
- Screen sharing during therapy sessions.
- Digital whiteboard.
- Session recording with patient consent.

Mobile Application

- Native Android application.
- Native iOS application.
- Push notifications.
- Offline support.
- Biometric authentication.

Security Enhancements

- Multi-factor authentication (MFA).
- End-to-end encrypted messaging.
- Advanced audit logging.
- Data encryption at rest.

- Compliance with healthcare regulations.

Platform Improvements

- Multi-language support.
- Advanced analytics dashboard.
- Calendar synchronization.
- Email and SMS notifications.
- Cloud deployment.
- Microservices architecture.
- Automated backup and disaster recovery.
- AI-generated clinical reports.

These enhancements would significantly expand the platform's capabilities and improve the overall user experience while preserving the flexibility provided by Clean Architecture.

20.9 Overall Evaluation

Based on the implementation, testing, and system validation, the NFS platform successfully fulfills its intended purpose as an integrated mental healthcare management system.

The completed platform demonstrates:

- Clean and well-organized software architecture.
- Independent Presentation, Application, Domain, and Infrastructure layers.
- Modular backend implementation.
- Responsive frontend interface.
- Reliable database design.
- Secure authentication mechanism.
- Comprehensive healthcare workflow.
- Efficient communication between modules.
- Repository-based data access.
- Scalable and maintainable codebase.

The project illustrates how modern software engineering practices and Clean Architecture principles can be applied to develop a real-world healthcare application that addresses both functional and technical requirements.

20.10 Final Conclusion

The NFS Mental Health Platform represents a comprehensive full-stack software solution designed to improve the accessibility and management of mental healthcare services. Through the integration of secure authentication, therapist and patient management, appointment scheduling, therapy session management, psychological assessments, payment processing, real-time communication, and a normalized relational database, the platform provides a complete digital ecosystem for mental health support.

The implementation demonstrates the successful application of modern software engineering principles, including **Clean Architecture**, RESTful API design, Entity Framework Core, dependency injection, repository abstractions, and modular frontend development. Comprehensive testing confirmed the correctness and stability of the implemented features, while the project's architecture provides a scalable and maintainable foundation for future enhancements such as artificial intelligence, telemedicine, mobile applications, and advanced security mechanisms.

Overall, the project achieves its primary objectives and delivers a maintainable, extensible, secure, and well-structured healthcare platform capable of supporting future research, development, and real-world deployment in the field of digital mental health services.